



Developer Guide

for Motorola Enterprise Mobility Devices

Developer Guide for Motorola Enterprise Mobility Devices

72E-71161-02

Revision A

October 2007

© 2007 by Motorola, Inc. All rights reserved.

No part of this publication may be reproduced or used in any form, or by any electrical or mechanical means, without permission in writing from Motorola. This includes electronic or mechanical means, such as photocopying, recording, or information storage and retrieval systems. The material in this manual is subject to change without notice.

The software is provided strictly on an “as is” basis. All software, including firmware, furnished to the user is on a licensed basis. Motorola grants to the user a non-transferable and non-exclusive license to use each software or firmware program delivered hereunder (licensed program). Except as noted below, such license may not be assigned, sublicensed, or otherwise transferred by the user without prior written consent of Motorola. No right to copy a licensed program in whole or in part is granted, except as permitted under copyright law. The user shall not modify, merge, or incorporate any form or portion of a licensed program with other program material, create a derivative work from a licensed program, or use a licensed program in a network without written permission from Motorola. The user agrees to maintain Motorola’s copyright notice on the licensed programs delivered hereunder, and to include the same on any authorized copies it makes, in whole or in part. The user agrees not to decompile, disassemble, decode, or reverse engineer any licensed program delivered to the user or any portion thereof.

Motorola reserves the right to make changes to any software or product to improve reliability, function, or design.

Motorola does not assume any product liability arising out of, or in connection with, the application or use of any product, circuit, or application described herein.

No license is granted, either expressly or by implication, estoppel, or otherwise under any Motorola, Inc., intellectual property rights. An implied license only exists for equipment, circuits, and subsystems contained in Motorola products.

MOTOROLA and the Stylized M Logo and Symbol and the Symbol logo are registered in the US Patent & Trademark Office. Bluetooth is a registered trademark of Bluetooth SIG. Microsoft, Windows and ActiveSync are either registered trademarks or trademarks of Microsoft Corporation. All other product or service names are the property of their respective owners.

Motorola, Inc.
One Motorola Plaza
Holtsville, New York 11742-1300
<http://www.symbol.com>

Patents

This product is covered by one or more of the patents listed on the website: www.symbol.com/patents

Revision History

Changes to the original manual are listed below:

Change	Date	Description
Rev A	5/2006	Initial release.
Rev A	9/2007	Updated to the new corporate template.

Table of Contents

Patents.....	ii
Revision History.....	iii

About This Guide

Introduction.....	vii
What This Guide is Not.....	vii
Chapter Descriptions.....	vii
Notational Conventions.....	viii
Related Documents and Software.....	ix

Chapter 1: *Introduction*

A History of Innovation.....	1-1
Enterprise Mobility.....	1-1
SMDK.....	1-2

Chapter 2: *Developer Kit Overview*

Introduction.....	2-1
SMDK for C.....	2-2
Symbol Pocket Browser.....	2-4
SMDK for .NET.....	2-6
SMDK for Java.....	2-8
Selecting a Developer Kit.....	2-10
Alternate Development Tools.....	2-11
Odyssey.....	2-11
MCL.....	2-11
Wavelink.....	2-12

Chapter 3: *SMDK for C*

Introduction.....	3-1
Architecture.....	3-1

Platform SDKs	3-3
Programming Libraries	3-5
SMDK for C Components	3-7
Symbol Platform Integrator	3-8
CheckAPI	3-9
Install Requirements	3-10
Installation Rules	3-10
Adding a new WinCE 5.0 platform to a Visual Studio 2005 solution	3-11
Adding a New Platform	3-11
Removing a Platform	3-11
Hello Scan Sample Applications in C / C++	3-12
Device Updates	3-13
Installing Printer Components	3-13
Software Tools for Application Development	3-13
Microsoft eMbedded Visual C++ 4.0	3-14
Microsoft Visual Studio 2005	3-14
eVB	3-15
Tool Availability	3-16
Windows Mobile Developer Power Toys	3-16

Chapter 4: Symbol Pocket Browser

Introduction	4-1
Architecture	4-2
Symbol PocketBrowser Components	4-4
Usage Requirements	4-4
Symbol PocketBrowser Features	4-5
Hello Scan Sample Applications in JavaScript	4-7

Chapter 5: SMDK for .NET

Introduction	5-1
Architecture	5-1
Microsoft .NET Compact Framework	5-3
Programming Libraries	5-4
SMDK for .NET Components	5-5
Usage Requirements	5-6
Software Tools for Application Development	5-7
Microsoft® Visual Studio® .NET 2003	5-7
Microsoft Visual Studio 2005	5-7
Compatibility	5-7
Developing Applications	5-8
How to Use SMDK for .NET	5-8
Adding the BarcodeReader Design Time Control to the Toolbox	5-8
Application Deployment through Visual Studio.NET	5-8
Hello Scan Sample Application in C#/VB.NET	5-9
Device Updates	5-11
Application Mass Deployment for C#/VB.NET Applications	5-11

Chapter 6: *SMDK for Java*

Introduction	6-1
Architecture	6-1
Samples	6-3
Java Virtual Machine	6-3
J2ME JVM	6-3
J9 JVM	6-3
Programming Libraries	6-4
SMDK for Java Components	6-6
Usage Requirements	6-8
Configuring the Device for Java	6-8
Developing Applications	6-9
Remote Debugging	6-9
Hello Scan Sample Application in Java	6-10
Application Mass Deployment for Java Applications	6-12
Software Tools for Application Development	6-12
IBM WebSphere Studio Device Developer	6-12
Creating a new WSDD Project	6-12
Configuring a Build	6-15
Adding a SymbolPocketPC Device	6-17
Creating a Launch Configuration	6-18
JavaPOS Support	6-19
Supported Services	6-19
Architecture	6-20

Chapter 7: Deploying Applications

Introduction	7-1
Deployment Methods	7-1
ActiveSync	7-1
Removable Media	7-2
TCM	7-3
AirBEAM Smart	7-4
MSP	7-5
Third-Party Device Management Tools	7-7
Application Deployment for Mobile 5.0	7-8
Mobile 5.0 Deployment	7-8
Deployment	7-8
Image Update	7-8
XML Provisioning	7-8

Chapter 8: *Application Lock-down*

Introduction	8-1
Preventing User Access to Programs	8-1
AppCenter	8-1
Symbol PocketBrowser	8-2
Microsoft SHFullScreen	8-2

Chapter 9: Application Persistence

Introduction	9-1
Flash File System	9-1
RegMerge and CopyFiles	9-1
RegMerge	9-2
CopyFiles	9-2
Making the Application Persist	9-3
Making Application Data Persist	9-3
Packaging and Deployment	9-4
Running Applications from RAM	9-4
Automatic Startup of Applications	9-4
OS Launch Keys	9-4
Windows Startup Folder	9-5
Symbol Startup Program Keys	9-5
Symbol Startup Folder	9-6
Bypassing the Welcome Screen	9-6
Automatic CAB Installation	9-7
Windows Mobile 5.0	9-8
XML Provisioning vs. RegMerge and CopyFiles	9-8
RegMerge	9-8
CopyFiles	9-9

Chapter 10: *Advanced Programming*

Introduction	10-1
Advanced Programming	10-1
Writing a Bar Code Data Capture Application	10-1
Finding a Scanner Device	10-2
Opening a Scanner Device	10-2
Using Scanner Parameters	10-2
Starting a Read Request	10-3
Triggering	10-3
Cleaning Up	10-4
Combining Reads	10-4
Foreground, Background and Monitor Reads	10-4
Writing an Image Capture Application	10-5
Imaging Devices	10-5
Device Sharing	10-6
Image Acquisition	10-6
Image Formatting	10-6
Image Composition	10-6
Writing a WWAN Application	10-7
Line Device	10-7
Voice Call	10-7
Data Call	10-7
SMS	10-8
Radio/Network Information	10-8
Writing a Mobile Printing Application	10-8
How to Print	10-9

Appendix A: Additional Learning

Introduction	A-1
Additional Learning	A-1
SMDK for C	A-2
Topics	A-2
Training, Books, Learning Resources	A-2
MSDN - Visual Studio Team System Developer Center	A-2
MSDN - Mobile Developer Center	A-2
MSDN - Embedded Developer Center	A-2
MSDN - Embedded Visual C++	A-2
SMDK for .NET	A-3
Topics	A-3
Training, Books, Learning Resources	A-3
MSDN - Visual Studio Team System Developer Center	A-3
Microsoft Developer Network - .NET Compact Framework	A-3
Microsoft Developer Network - Microsoft Visual Studio Developer Center	A-3
OpenNETCF.org	A-3
Symbol Pocket Browser	A-4
Topics	A-4
Training, Books, Learning Resources	A-4
MSDN - Web Development	A-4
HTML	A-4
SMDK for Java	A-5
Topics	A-5
Training, Books, Learning Resources	A-5
IBM DeveloperWorks - New to Java Technology	A-5
Eclipse	A-5

Glossary**Index**



About This Guide

Introduction

This Developer Guide is intended for programmers who create applications for Motorola enterprise mobility devices. The information provided in this guide applies to devices based on a minimum OS version of Microsoft Windows CE v4.2, Microsoft Windows CE 5.0, Microsoft Windows Mobile 2003 for Pocket PC and Microsoft Windows Mobile 5.0.

Four programming models are described, each supported by a different Symbol Mobility Developer Kit (SMDK). This guide assists developers in deciding which programming model is right for them.

What This Guide is Not

This Developer Guide is not intended to teach:

- Windows CE or Pocket PC general programming techniques
- General Web programming techniques
- Computer languages such as Java, C, C++, C# or VB.NET
- Use of Motorola devices, such as the MC9000 or the PPT8800
- Use of Microsoft or IBM development tools
- Software design and development concepts.

Chapter Descriptions

Topics covered in this guide are as follows:

- [Chapter 1](#), Introduction provides a brief history of innovation at Symbol, a description of the enterprise mobility strategy, and an introduction to SMDK.
- [Chapter 2](#), Developer Kit Overview provides a summary of each of the four available SMDK (SMDK for C, SMDK for .NET, Symbol Pocket Browser and SMDK for Java).

- [Chapter 3, SMDK for C](#) provides a complete discussion of the Symbol Mobility Developer Kit for C, including its architecture, available libraries and usage in the creation of C and C++ applications.
- [Chapter 4, Symbol Pocket Browser](#) provides a complete discussion of the Symbol Pocket Browser, including its architecture, available libraries and usage in the creation of Web based applications.
- [Chapter 5, SMDK for .NET](#) provides a complete discussion of the Symbol Mobility Developer Kit for .NET, including its architecture, available libraries and usage in the creation of C# and VB.NET applications for the Microsoft .NET Compact Framework.
- [Chapter 6, SMDK for Java](#) provides a complete discussion of the Symbol Mobility Developer Kit for Java, including its architecture, available libraries and usage in the creation of Java applications for the IBM J9 JVM.
- [Chapter 7, Deploying Applications](#) describes various techniques for deploying applications to one or many Motorola enterprise mobility devices.
- [Chapter 8, Application Lock-down](#) describes various techniques for preventing user access to programs.
- [Chapter 9, Application Persistence](#) provides a complete discussion of the Flash File System architecture that supports application persistence.
- [Chapter 10, Advanced Programming](#) provides detailed guidance for writing specific types of applications such as those used for scanning, image capture, printing and wireless Wide Area Network.
- [Appendix A, Additional Learning](#) provides alternative sources of information related to Microsoft Windows CE and Pocket PC application development.
- [Glossary](#), provides definitions of relevant terms and acronyms.

Notational Conventions

The following conventions are used in this document:

- “device” refers to any Motorola enterprise mobility device.
- “User” refers to anyone using an application on the terminal.
- “You” refers to the End User, System Administrator or Technical Support person using this manual as a reference to install, configure, operate, maintain and troubleshoot the terminal.
- *Italics* are used to highlight the following:
 - chapters and sections in this and related documents
 - dialog box, window and screen names
 - drop-down list and list box names
 - check box and radio button names
 - icons on a screen.
- **Bold** text is used to highlight the following:
 - key names on a keypad
 - button names on a screen.

- Bullets (•) indicate:
 - action items
 - lists of alternatives
 - lists of required steps that are not necessarily sequential.
- Sequential lists (e.g., those that describe step-by-step procedures) appear as numbered lists.



NOTE This symbol indicates something of special interest or importance to the reader. Failure to read the note will not result in physical harm to the reader, equipment or data.



CAUTION This symbol indicates that if this information is ignored, the possibility of data or material damage may occur.



WARNING! This symbol indicates that if this information is ignored the possibility that serious personal injury may occur.

Related Documents and Software

The following documents and software provide more information about Motorola enterprise mobility devices.

- *Microsoft® Applications for Symbol Devices - User Guide*, p/n 72-68197-xx
- *Symbol Applications for Symbol Devices*, p/n 72-68901-xx
- *SMDK for C Help File*, p/n 72E-38880-xx
- Symbol Mobility Developer Kit for C (SMDK for C)
- Symbol Pocket Browser
- Symbol Mobility Developer Kit for Java (SMDK for Java)
- Symbol Mobility Developer Kit for .NET (SMDK for .NET)
- Device Configuration Packages (DCPs)
- ActiveSync® software, available at: <http://www.microsoft.com>.

For the latest versions of SMDKs and DCPs, go to: <http://support.symbol.com>.

For the latest version of this guide and all guides, go to: <http://support.symbol.com>.



Chapter 1 Introduction

A History of Innovation

Symbol Technologies was founded in 1975 and the first Symbolmaker generator film masters were produced. In 1980, the first handheld laser bar code scanner was introduced. By 1990, Spectrum One® became the first commercially available, transaction-oriented spread spectrum wireless LAN network. 1995 brought the introduction of Spectrum24®, the high-performance 2.4 GHz wireless network designed to comply with the IEEE 802.11 international standard for airwaves communications.

In 2004, continuing to find more efficient ways of working and engineering information technology into new ways of thinking, Symbol introduced RFID technology integrated with a mobile solution to expand and improve the general functionality and performance of the entire mobile application. Also in 2004, Symbol introduced the first in a new family of durable enterprise digital assistant (EDA) products specifically designed for business essential applications within the enterprise environment. Targeting the growing mobile enterprise market, the new Motorola enterprise mobility device was created for mobile workers within organizations that are seeking to capture, move and manage information at the point of business activity in order to increase productivity and efficiency, while improving customer responsiveness and satisfaction.

With the acquisition of Symbol in 2007, Motorola continues to invest in technologies that drive Enterprise Mobility solutions.

Enterprise Mobility

Today, when people, assets and information are in constant motion, companies across all industries are beginning to understand the competitive advantages that enterprise mobility solutions can deliver.

Having access to the right information at the right moment makes all the difference to a retailer monitoring inventory, a delivery person tracking a package or a doctor following a patient's progress. Without the latest data, progress can stall, delays can occur and costly mistakes can be made.

Motorola's enterprise mobility solutions continuously deliver real time answers to real world business problems, empowering people to make informed decisions that move business forward.

Motorola's enterprise mobility solutions enable some of the world's leading companies and ultimately:

- Enhance the retail experience by giving customers on-demand access to product and store information as they shop, and giving sales associates access to real-time customer and inventory data at the point of service.

- Boost worker productivity by linking managers and associates in the field, on the retail floor, in the warehouse and on the loading dock.
- Improve the ability of healthcare providers to make more informed decisions, by giving them access to medical charts, vitals and lab reports from patient's bedside.
- Reduce inventory shrinkage and out-of-date products by providing the most current information about customer buying patterns.
- Manage and control the entire enterprise mobility solution including security, upgrades, maintenance and performance from an easy-to-use, anytime, anywhere Web-based interface.

SMDK

SMDK (Symbol Mobility Developer Kit) is a family of developer tools used to create applications for Rugged, EDA and Micro-Kiosk devices. Using the SMDK, application developers can take advantage of the enterprise mobility features found on Motorola devices. These features include barcode scanning, image capture, RFID tag collection, wireless LAN communications, and many others.

Four developer kits, based on programming language are available

- *SMDK for C* - Create native applications in C and C++ that access the Symbol C API.
- *Symbol PocketBrowser* - Create browser-based applications in HTML and JavaScript that access the mobility features of Motorola enterprise mobility devices.
- *SMDK for .NET* - Create .NET managed applications in C# and VB.NET that run on the Microsoft .NET Compact Framework.
- *SMDK for Java* - Create Java applications that run on the IBM J9 Java Virtual Machine.



NOTE SMDK and Symbol Pocket Browser can be used to develop applications for Motorola enterprise mobility devices that contain the Symbol API. Other Motorola devices such as cell phones and two-way radios are not supported by these developer kits.



Chapter 2 Developer Kit Overview

Introduction

There are four developer kits available:

- SMDK for C
- Symbol Pocket Browser
- SMDK for .NET
- SMDK for Java.

The information provided in this chapter assists the reader in making an informed decision as to the appropriate development environment to choose.

For detailed information about each, see the appropriate chapter.

SMDK for C

SMDK for C provides all of the tools necessary to create C and C++ applications for Motorola enterprise mobility devices running Windows CE 4.2, Windows CE 5.0, Pocket PC 2003 and Window Mobile 5.0. This developer kit can be used with either Microsoft eMbedded Visual C++ 4.0 or Visual Studio 2005.



NOTE SMDK for C replaces SMDK for eVC4. No further updates are planned for SMDK for eVC4.

Table 2-1 lists the supported C API groups.

Table 2-1 SMDK for C API Groups

C API Group	Description
Audio	Provides the ability for applications to control sounds played through the device's beeper and/or speaker, depending on the specific product.
Audio Extension	Allows an application to configure the extensions to the audio subsystem. When applicable, the application can query and adjust the output device, microphone gain, and side tone. In addition, event notification can be configured and status retrieved. Because the audio subsystem does not support stereo output, stereo WAV files are combined into one channel.
Display	Provides the ability for applications to control display contrast and backlight. Functions are provided that allow the application to read the supported number of levels for both contrast and backlight.
Fusion WLAN	Allows an application to create and delete WLAN profiles. The application can also obtain information on the existing profiles.
Image Capture	Allows an application to capture still images. Supported functions include enumeration of available imaging devices, opening one or more of the available devices, querying for supported capabilities, modifying capabilities, enabling viewfinder and acquiring still images.
Keyboard	Allows an application to re-map the keyboard. Supported functions include moving a key to a new location, changing the scan codes and VK codes generated by keys.
MSR	Allows an application to access Magnetic Stripe Reader accessories connected to the device.
Notification	Provides the ability for applications to control various notification devices, such as LED, Pager and Beeper.
Printing	Provides the ability for applications to print bar codes, text, bitmaps and lines. Several mobile printing languages are supported, including Zebra, Comtec, O'Neil and Monarch.
Resource Coordinator	Provides the ability for applications to monitor trigger activity, query unique unit identification (UUID) and temperature information, and retrieve configuration data.

Table 2-1 SMDK for C API Groups (Continued)

C API Group	Description
RFID	Provides the ability for applications to access the tag information scanned by the RFID reader.
Scanning	Provides applications with the ability to read bar code data. This API supports 1D and 2D bar code scanning, image capture and signature capture. A variety of bar code scanning technologies are supported, including laser, contact wand, and CCD imaging. The API supports multi-read scanning, which allows multiple applications to share the use of a single scan engine.
Spectrum24	Allows an application to access the Driver and Wireless Adapter information in a local Mobile Unit (MU).

Symbol Pocket Browser

Symbol Pocket Browser provides all of the tools necessary to create advanced web-based applications for Windows Mobile and Windows CE devices.

Symbol PocketBrowser provides all of the functions necessary for creating rich enterprise web applications that target Motorola enterprise mobility devices. The included libraries expose enterprise mobility functionalities for developers developing web applications which output HTML and JavaScript.



NOTE Symbol Pocket Browser replaces SMDK for the Web. No further updates are planned for SMDK for the Web.

Table 2-2 lists the supported Symbol PocketBrowser Function Groups.

Table 2-2 *Symbol PocketBrowser Function Groups*

Function Group	Description
AirBEAM Smart	JavaScript access to the AirBEAM Smart APIs.
Battery & Power	Allows the developer to display a power meter and retrieve regular information about the status of the battery or if the device is consuming AC power.
Communications	Exposes access to read from the communications port.
Controls & Appearance	Allows the displaying of general navigation controls, the SIP button, command areas etc.
Device Control	Provides functions to initiate the device calibration routine, warm/cold boot sequence, synchronize the device clock, invoke notifications objects (vibrator, Led's, beeper).
Environment	Allows for adjusting the backlight, cursor position, sip position, screen orientation and text zoom factor.
File Management	Provides a mechanism for uploading and downloading files to and from the device via FTP or HTTP. It also allows to move/copy and delete files on the local file system.
Generic ActiveX	JavaScript access to show/hide the hourglass, invoke any meta tag, play wave files, access the registry, launch processes and initiate RAS dialup sessions.
Imager	Functionality to capture images from imager and send them to either an FTP or HTTP host.
Key Capture	Allows call-backs to be assigned to any key press or the trigger.
Logging	Provides access to log to either a local file server or to an HTTP server. Able to receive information and statistics on the performance and execution of Symbol Pocket Browser. This also allows user to add customized logged events using Javascript.
Magnetic Stripe Reader	Provides access to the MSR if attached.

Table 2-2 *Symbol PocketBrowser Function Groups (Continued)*

Function Group	Description
Offline Data Access (ODAX)	Allows storing of data locally on the device using either a flat file structure or XML. Combined with the File transfer tags, the ODAX control offers a viable solution for "Batch" style applications.
Printing	Provides access to the APD (Adaptive Printer Driver) to allow the user to print to most mobile printers.
Push Navigate	Reduces network traffic, saves battery resources and delivers instant information and messages for faster response times and increased productivity.
Radio	Displays a signal meter and allows periodic updates on radio data.
Registry	Provides the ability to write to the registry backing up the value in a .reg file in the non-volatile ROM.
RFID	Provides a mechanism to capture data from the RFID reader.
Scanner	Provides a mechanism to capture data from the scanner and allows for each decoder to be configured.

SMDK for .NET

The SMDK for .NET provides all of the tools necessary to develop C# and VB .NET managed applications for Motorola enterprise mobility devices. These tools include class libraries, sample applications and the associated documentation. SMDK for .NET allows Microsoft .NET Compact Framework (see [Microsoft .NET Compact Framework on page 5-3](#)) developers to programmatically access the enterprise mobility features of the devices.

[Table 2-3](#) lists the supported .NET Namespaces.

Table 2-3 SMDK for .NET Class Namespaces

Namespace	Description
Audio	Provides the ability to control sounds played through the device's beeper and/or speaker, depending on the specific product.
Barcode	Provides the ability to read bar code labels. This namespace fully supports 1D and 2D bar code scanning using a variety of bar code scanning engines, including laser and CCD scanners. The barcode class library also provides capabilities for allowing multiple applications to share the use of a single scan engine.
BarcodeForms	Provides standard forms for viewing and modifying barcode scanner parameters.
Display	Provides the ability to configure display contrast and backlight settings. A .NET application can read the supported number of contrast levels and adjust the device screen contrast.
Fusion	Provides a set of classes that can be used to obtain version information of Fusion components and diagnostic report.
Fusion.WLAN	Provides a set of classes that can be used to create and manage WLAN profiles and obtain WLAN information.
Imaging	Provides the ability to display a viewfinder window and capture images on devices that contain imaging hardware.
Keyboard	Provides a set of classes that can be used to register for keypress notification for modifier keys (such as SHIFT, CAPS_LOCK, FUNC, etc.) as well as set the state of these keys. The current KeyLite configuration can also be modified using classes provided by this assembly.
MagStripe	Provides the ability to obtain swiped data from an attached Magnetic Stripe Reader (MSR).
MKSeries	Provides access to functionality found ONLY on MK Series products (such as MK2000). This functionality includes button configuration, inactivity manager setup, as well as the capability of setting the MK product into continuous trigger mode.
Notification	Provides the ability to control device notification components such as LEDs, pager/vibrator and beeper.
Printing	Provides the ability to print bar codes, lines, and text to the mobile printers supported by the Print 'C' API.

Table 2-3 SMDK for .NET Class Namespaces (Continued)

Namespace	Description
Resource Coordination	Provides the ability to monitor trigger activity, query unique unit identification, and retrieve configuration data.
RFID	Provides the ability to read RFID tags using an RFID reader attached to the device. This class library has been deprecated and is planned for removal in future versions. Please start using the RFID2 Class library.
RFID2	Provides the ability to read RFID tags using a common RFID interface for MC9090, RD5000 and XR400 RFID devices.
StandardForms	Provides common controls and dialogs used to view and modify classes that have been derived from the Symbol.API class.
WirelessLAN	Provides the ability to obtain statistical and configuration information from the Wireless LAN adapter.

SMDK for Java

The SMDK for Java provides a set of tools to develop Java applications for Motorola enterprise mobility devices running the IBM J9 JVM. These tools include class libraries, sample applications and the associated documentation. SMDK for Java allows J2ME developers to programmatically access the enterprise mobility features of their devices.

[Table 2-4](#) lists the supported Java Class Libraries.

Table 2-4 SMDK for Java Class Libraries

Class Library	Description
Audio	Provides the ability to control sounds played through the device's beeper and/or speaker, depending on the specific product. While the Audio class works best with Motorola enterprise mobility devices that have physical beepers, a WAV file can be played through a standard audio codec and speakers.
Display	Provides the ability to configure display contrast and backlight settings. The programmer can use this class to turn backlight on or off, read the supported number of backlight intensity levels and query or adjust the current backlight intensity. Similarly, the programmer can use this class to read the supported number of contrast levels and query or adjust the current display contrast. The contrast attribute is usually used on monochrome displays only.
Fusion	Provides applications with the ability to access wireless networking. (Currently supports WLAN only.)
Imager	Provides applications with the ability to access the imaging device(s) on a system for acquiring still images.
Keyboard	Provides the ability to configure or query keyboard state and parameters, as well as register for notification of keyboard state changes.
MSR	Provides the ability to read data from an MSR (Magnetic Stripe Reader).
Notification	Provides the ability to control device notification components such as LEDs, Pager/Vibrator and Beeper.
Power	Provides the ability to obtain the battery status information as well as the power state of a particular device.
PrintierJob	Provides the capability to print bar codes, text, bitmaps and lines.
RFID	Provides applications with the ability to perform RFID related features on devices equipped with an RFID Reader.

Table 2-4 *SMDK for Java Class Libraries (Continued)*

Class Library	Description
Scanner	Provides the ability to read 1D and 2D bar code labels using a variety of scanning engines, including laser and CCD imagers.
Spectrum24	Provides the ability to obtain statistical and configuration information from the Spectrum24 Wireless LAN adapter.
Trigger	Provides the ability to register for trigger event notifications and get status for any available trigger.

Selecting a Developer Kit

The developer kits detailed in this manual allow you to write applications that take advantage of the capture, move and manage capabilities of Motorola enterprise mobility devices. Several programming models are supported. Use [Table 2-5](#) to identify the kit that best fits your development needs.

Check the individual download page for each developer kit to see which devices and operating systems are supported.

Table 2-5 *Selecting a Developer Kit*

	SMDK for eVC4	SMDK for C	SMDK for .NET	SMDK for Java	Pocket Browser
Coding Language	C and C++	C and C++	C# and VB.NET	Java	JavaScript / HTML
Development Tools	Microsoft eMbedded Visual C++ 4.0 with SP 4	Visual Studio 2005 / Microsoft eMbedded Visual C++ 4.0 with SP 4	Microsoft Visual Studio.Net 2003/ Visual Studio 2005	IBM WebSphere Studio Device Developer	Any web authoring tool such as Microsoft FrontPage
Additional SDK	Microsoft Windows Mobile SDK or a Symbol Windows CE SDK	Microsoft Windows Mobile SDK or a Symbol Windows CE SDK	None	None	None
Runtime on Device	None	None	Symbol .NET class libraries and Microsoft .NET Compact Framework	Symbol Java class libraries and IBM J9 Java Virtual Machine	Symbol Pocket Browser runtimes
CE 4.2	Yes	Yes	Yes	Yes	Yes
CE 5.0	Yes	Yes	Yes	Yes	Yes
Mobile 2003	Yes	Yes	Yes	Yes	Yes
Mobile 5.0	No	Yes	Yes	Yes	Yes
Make your selection	To create native applications for Motorola enterprise mobility devices, written in C or C++, select SMDK for eVC4	To create native applications for Motorola enterprise mobility devices, written in C or C++, select SMDK for C	To create managed .NET CF applications for Motorola enterprise mobility devices, written in C# or VB.NET, select SMDK for .NET	To create Java applications for Motorola enterprise mobility devices and the J9 JVM, select SMDK for Java	To create Web applications for Motorola enterprise mobility devices, in HTML and JavaScript, select Pocket Browser

Alternate Development Tools

While use of SMDKs and full-featured programming environments from Microsoft and IBM are powerful, they can also be complicated and may require professional programmers with a higher level of training and experience. Some third-party vendors provide simpler application development environments, for situations when speed-to-market and developer productivity are of primary importance. These tools are referred to as Rapid Application Development or RAD tools. RAD can be defined as any techniques that make developing applications easier than the traditional approach. For example, a tool that accepts a description of the requirements and then automatically generates the code. The following companies provide RAD tools for Motorola enterprise mobility devices.

Odyssey

Odyssey Software delivers mobile and wireless application infrastructure and application development tools for fast and effective mobile enterprise application development and deployment. You can implement powerful distributed enterprise applications with true interoperability among a wide range of mobile, desktop and server-class platforms. The Odyssey products available include CFCom, for adding COM support to CE; ViaXML, for Web services; and CEFusion, for rapidly building and deploying rich mobile enterprise applications.

For more information go to: <http://www.odysseysoftware.com/>.

MCL

MCL-Collection is an intuitive, high-productivity software tool used to create, integrate, and deploy enterprise, multimodal mobile worker applications quickly and easily. From barcode scanning and data capture on terminals to ODBC, WMS, or SAP R/3 connectivity on the host, MCL-Collection provides seamless integration from mobile computer to host application.

- *MCL-Designer V3* offers easy development of complex wired, wireless LAN and WAN applications with limited programming knowledge.
- *MCL-Net V3* offers wireless communication to and from a centralized point and real-time connection back to enterprise systems. This exchange of files and/or data records between a host system and Mobile Devices uses various communication methods, like 802.11, Internet (PPP via GSM), Ethernet, and GPRS.
- *MCL-Link V3* offers wired exchange of files and/or data records and is used for sequential communication with one device at a time. (Point to Point)
- *MCL-Client V3* provides a thick client architecture, making the terminal an intelligent independent device allowing for always connected, casually connected, or standalone operation.

For more information go to: <http://www.mcl-collection.com/>.

Wavelink

Wavelink offers tools for wireless mobility development. Wavelink Studio COM is a powerful collection of development libraries, server-side software and client applications for devices. The clients run on the device and are the bridge between the server-side application and the end user. The client is specific to a particular device, but supports Wavelink Studio COM development libraries.

The Wavelink libraries reduce the time it takes you to create a wireless application by allowing you to separate the business functions from the presentation of the applications. Note that Wavelink is not the development environment for the business logic; instead, Wavelink libraries can be used with C/C++ and Java.

Wavelink also provides terminal emulation clients for many Motorola Windows CE-based devices.

For more information go to: <http://www.wavelink.com/>.



Chapter 3 SMDK for C

Introduction

This chapter provides a complete discussion of the SDK architecture for developers who are programming using the SMDK for C. This developer kit supports all mobile devices starting with those based on Windows CE 4.2 and or Windows Mobile 2003 for Pocket PC. It requires Microsoft Visual Studio 2005 or eMbedded Visual C++ 4.0 SP4, plus at least one installed Platform SDK. SMDK for C provides access to enterprise mobility functionality from within C, C++ and MFC applications. (see [Install Requirements on page 3-10](#))

Architecture

C Language programmers access Motorola enterprise mobility features through a C API. The C API is implemented on the device as Dynamic Link Libraries (DLLs). Each hardware device has an associated library which allows programmatic access to the device driver for that particular device. For example, there is a Symbol scan API, implemented in a scan DLL, that allows access to the scan driver functionality.

Most of these DLLs are pre-installed on the device at the factory. Some, such as the print DLLs, must be installed by the developer.

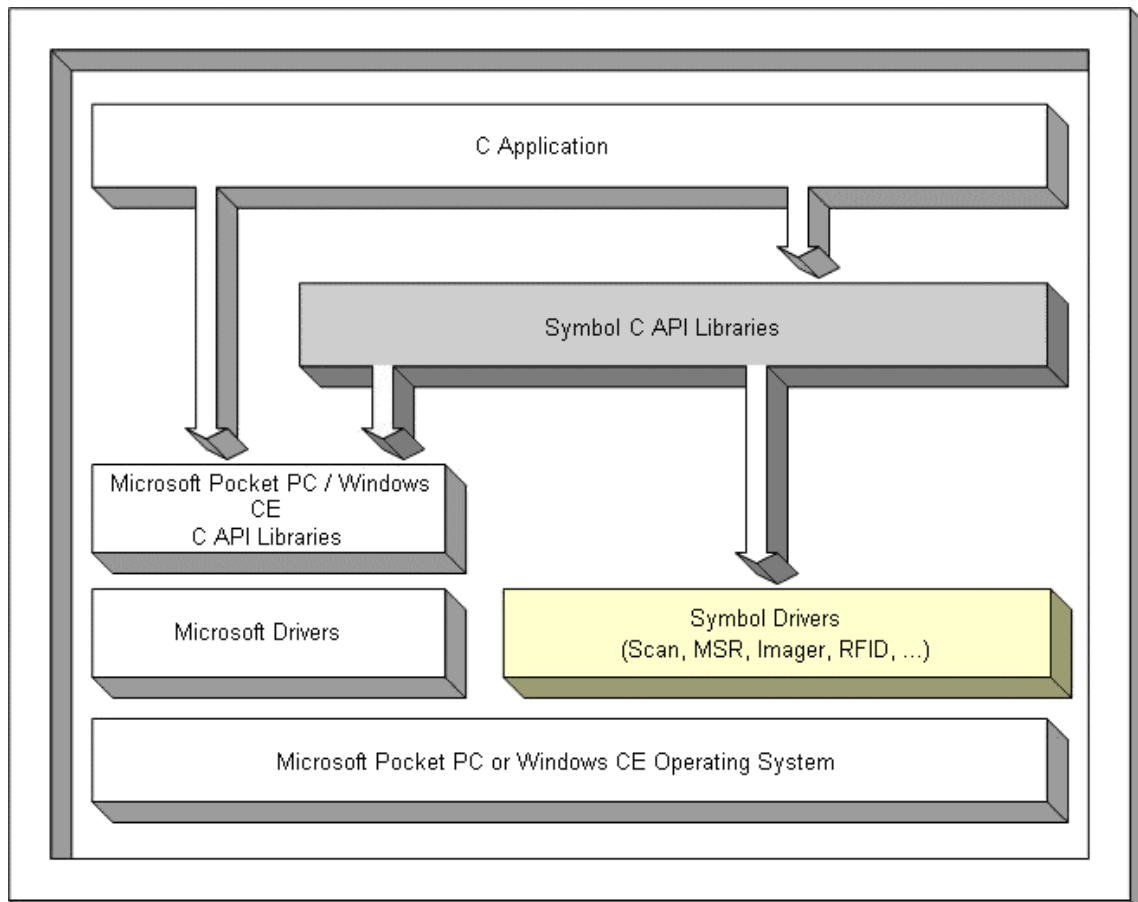


Figure 3-1 SMDK for C Architecture

Platform SDKs

The Platform SDK (PSDK) is created using Microsoft's Platform Builder tool and is provided by Motorola for Windows CE devices. The PSDK for a particular device can be downloaded from the Support Central <http://support.symbol.com>.

For Windows Mobile devices, an equivalent SDK is available from Microsoft as a free download. Use the Pocket PC 2003 SDK or the Windows Mobile 5.0 SDK to create applications for Windows Mobile devices.

Each installed SDK integrates within the Integrated Development Environment (IDE) of eVC4 or Visual Studio 2005 to provide a new target device for which applications can be built. Once installed, the new device type is available in the *Active WCE Configuration* field of Microsoft eMbedded Visual C++ 4.0 (eVC4). For Visual Studio 2005, the newly installed platform must be added to your projects using the Configuration manager within the IDE.

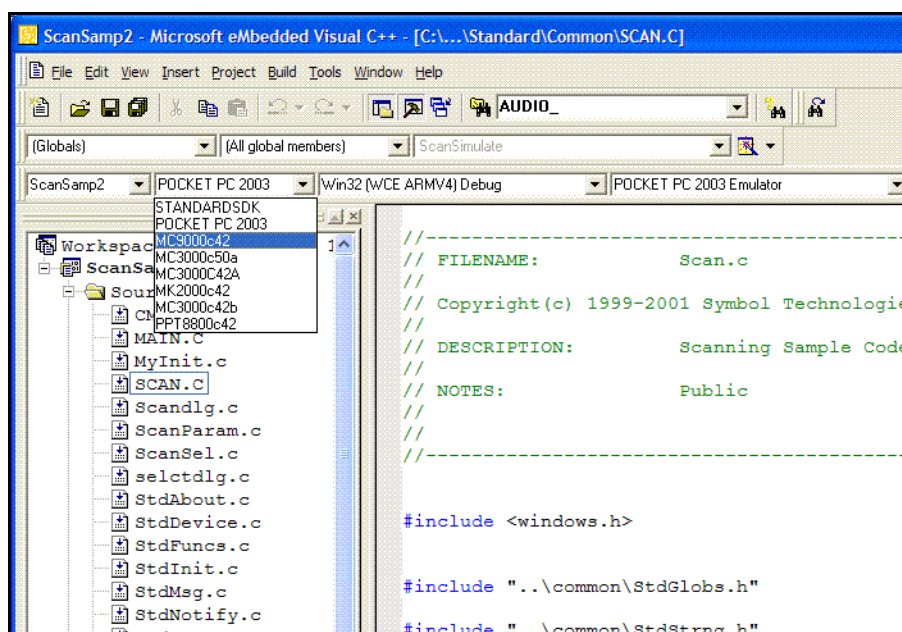


Figure 3-2 eVC4 Active WCE Configuration

An installed PSDK adds a program group to the Start menu providing easy access to the release notes. For example the Windows CE 5.0 version of the MC3000 PSDK would appear as *Windows CE Platform SDK v1.0 for MC3000c50B*.

For Visual Studio 2005, the newly installed platform must be added to your project using the Configuration manager within the IDE. For the sample projects to build correctly, you must select *Windows Mobile 5.0 Pocket PC SDK (ARMV4I)* in the *Copy settings from:* field.

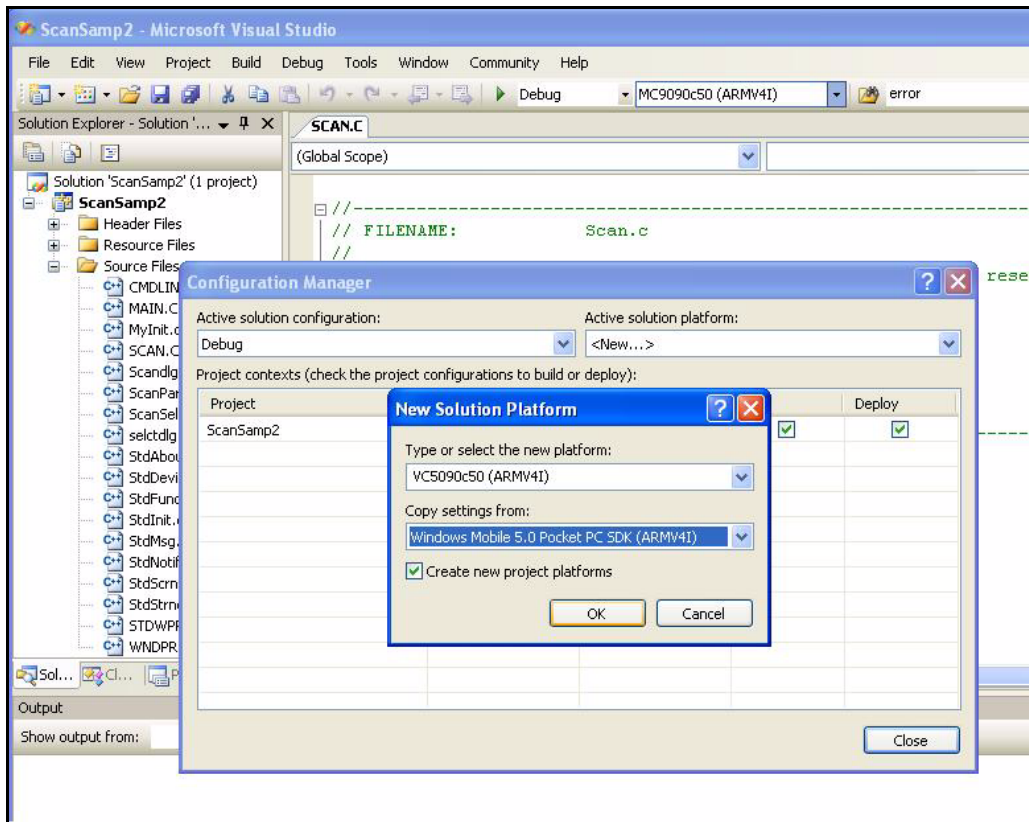


Figure 3-3 Visual Studio 2005 Configuration Manager

Programming Libraries

The SMDK for C supports 13 C libraries. The APIs constitute the standard Symbol Application Programming Interface (API). API definitions in the SMDK Help file illustrate how to interact with a given function. Prototypes, parameters, return values and requirements are provided for each API. With the exception of the Printer API, all APIs for the SMDK for C are installed on the device at the factory. (For more information about the Printer API, see [Installing Printer Components on page 3-13.](#))

Table 3-1 lists the APIs supported by SMDK for C.

Table 3-1 SMDK for C API Groups

C API Group	Description
Audio	Provides the ability for applications to control sounds played through the device's beeper and/or speaker, depending on the specific product.
Audio Extension	Allows an application to configure the extensions to the audio subsystem. When applicable, the application can query and adjust the output device, microphone gain, and side tone. In addition, event notification can be configured and status retrieved. Because the audio subsystem does not support stereo output, stereo WAV files are combined into one channel.
Display	Provides the ability for applications to control display contrast and backlight. Functions are provided that allow the application to read the supported number of levels for both contrast and backlight.
Fusion WLAN	Allows an application to create and delete WLAN profiles. The application can also obtain information on the existing profiles.
Image Capture	Allows an application to capture still images. Supported functions include enumeration of available imaging devices, opening one or more of the available devices, querying for supported capabilities, modifying capabilities, enabling viewfinder and acquiring still images.
Keyboard	Allows an application to re-map the keyboard. Supported functions include moving a key to a new location, changing the scan codes and VK codes generated by keys.
MSR	Allows an application to access Magnetic Stripe Reader accessories connected to the device.
Notification	Provides the ability for applications to control various notification devices, such as LED, Pager and Beeper.
Printing	Provides the ability for applications to print bar codes, text, bitmaps and lines. Several mobile printing languages are supported, including Zebra, Comtec, O'Neil and Monarch.
Resource Coordinator	Provides the ability for applications to monitor trigger activity, query unique unit identification (UUID) and temperature information, and retrieve configuration data.

Table 3-1 SMDK for C API Groups (Continued)

C API Group	Description
RFID	Provides the ability for applications to access the tag information scanned by the RFID reader.
Scanning	Provides applications with the ability to read bar code data. This API supports 1D and 2D bar code scanning, image capture and signature capture. A variety of bar code scanning technologies are supported, including laser, contact wand, and CCD imaging. The API supports multi threaded scanning, which allows multiple applications to share the use of a single scan engine.
Spectrum24	Allows an application to access the Driver and Wireless Adapter information in a local Mobile Unit (MU).

SMDK for C Components

The SMDK for C installation package provides all of the files necessary for creating C and C++ applications.



NOTE Although SMDK for C was designed to work with all Motorola enterprise mobility devices running Windows Mobile and Windows CE, it should only be used to develop applications for approved devices. Refer to the product download page for a complete listing of approved devices.

Once installed, the SMDK for C components can be easily accessed using the "Symbol Mobility Developer Kit for C" program group on the Windows *Start* menu. This program group provides access to the Help file, Readme file, CheckAPI utility, Platform Integrator utility and the Sample applications source code.

If the default install location is not changed, the components can be found on the development PC at the locations specified in [Table 3-2](#).

Table 3-2 SMDK for C Installed Components

Component	Description	Locations
Readme file	An HTML file that includes release notes and usage information. This file should always be read prior to development.	\Program Files\Symbol Mobility Developer Kit for C\X.X\
Help file	API function reference guide.	\Program Files\Symbol Mobility Developer Kit for C\X.X\
Samples	C language source code used for quick-start development. Each sample includes the full source code files and project files required to rebuild the application.	\Program Files\Symbol Mobility Developer Kit for C\X.X\Samples\
Headers	Header files (.h) with API prototypes and structures.	\Program Files\Windows CE Tools\wce420\<Platform Name>\Include\armv4 \Program Files\Windows CE Tools\wce500\<Platform Name>\Include\Armv4i
Libraries	Import Library files (.lib)	\Program Files\Windows CE Tools\wce420\<Platform Name>\Lib\armv4 \Program Files\Windows CE Tools\wce500\<Platform Name>\Lib\Armv4i
Platform Integrator	Utility used to add all Symbol header and library files to installed Symbol Platform SDKs and the Microsoft Pocket PC SDK.	\Program Files\Symbol Mobility Developer Kit for C\X.X\Integrator
Check API	Produces a report of the available Symbol API functions for a device.	\Program Files\Symbol Mobility Developer Kit for C\X.X\CheckAPI

Symbol Platform Integrator

Symbol Platform Integrator is a component of the SMDK that copies the Symbol library and header files from the SMDK to any installed Symbol PSDKs and to any installed Microsoft Windows Mobile SDKs. Copying the library and header files allows developers to call Symbol API functions without changing any of their project settings.

The Platform Integrator program runs automatically whenever the SMDK is installed, and whenever a Symbol PSDK is installed.

If a Microsoft Windows Mobile SDK is installed after the SMDK, Platform Integrator must be manually run from the program group on the Windows start menu. If this step is not taken, compiler errors are generated for calls to Symbol API functions.

Platform Integrator operates in a way that it can be used to repair corrupt platform SDK folders. If for some reason, Symbol lib and header files are changed or deleted, the original files can be recovered by manually launching Platform Integrator from the start menu.



warning! The lib and header files delivered with the SMDK must not be changed. Doing so may cause unpredictable results when building applications for any of the installed platforms.

When the application completes, the *Symbol Platform Integrator* window appears.



Figure 3-4 Symbol Platform Integrator Window

CheckAPI

CheckAPI is a utility that produces a report of the available C API functions on a mobile device. The Symbol export libraries provided with SMDK for C provide access to the latest set of API functions.

Use the CheckAPI utility provided in this developer kit to determine which API functions are present on the device. Make an ActiveSync connection and launch CheckAPI from the Windows Start menu. A report is produced, detailing all of the API functions that are available on the device. The report also lists some important system version information.

CheckAPI utility is useful when developing applications for an older device that may not implement all of the functions listed in the API help file. Although the compilation of the source code would not produce errors, the output program may not run in an older device that does not support all the functions listed in the API. Calling a function that exists in the export library but does not exist in the DLL on the device causes a failure when the application is launched. This generates an error message that states "Not a valid Windows CE application".



NOTE To avoid receiving "Not a valid Windows CE application" messages, use LoadLibrary and GetProcAddress within the application to call functions. This allows you to determine at runtime if an API function is implemented.

Install Requirements

The following software must be installed prior to using the SMDK for C. Most are available for download directly from Microsoft websites.

- Microsoft Windows 2000 or Windows XP Operating System.
- Microsoft ActiveSync 4.1 or higher.
- If developing applications for Windows Mobile 2003 or Windows CE 4.2.
 - Microsoft eMbedded Visual C++ 4.0.
 - Microsoft eMbedded Visual C++ 4.0 Service Pack 4.
- If developing applications for Windows Mobile 5.0, Windows Mobile 2003 or Windows CE 5.0.
 - Microsoft Visual Studio 2005.
- One or more of the following Platform SDKs:
 - Microsoft SDK for Windows Mobile 2003-based Pocket PCs.
 - Microsoft Windows Mobile 5.0 SDK for Pocket PC.
 - Windows CE Platform SDK for MC9090c50.
 - Windows CE Platform SDK for MC9000c50.
 - Windows CE Platform SDK for MC3000c50a.
 - Windows CE Platform SDK for MC3000c50b.
 - Windows CE Platform SDK for PPT8800c42.
 - Windows CE Platform SDK for MC9000c42.
 - Windows CE Platform SDK for MC3000c42a.
 - Windows CE Platform SDK for MC3000c42b.
 - Windows CE Platform SDK for MC1000c42.
 - Windows CE Platform SDK for MK2000c42.
 - Windows CE Platform SDK for VC5090C50.
 - Windows CE Platform SDK for WT4090C50.
 - Windows CE Platform SDK for MC1770c50B.



NOTE New Platform SDKs are released throughout the year.

Installation Rules

Please read these rules carefully. Failure to follow them could cause problems:

1. To install "SMDK for eVC4" and "SMDK for C" on the same PC, ensure that "SMDK for eVC4" is installed first, followed by "SMDK for C". Both SMDK packages install a version of the Platform Integrator with its associated library files. Installing SMDK for C last, ensures that the latest library files are being used with all of developments.
2. To ensure recognition of Windows CE SDKs (or Platform SDKs) by Visual Studio 2005, install the Windows CE SDKs after Visual Studio 2005 is installed.
3. The Windows Mobile 5.0 SDK installs only if Visual Studio 2005 is already installed.

4. The Microsoft Windows Mobile SDKs should be installed before the SMDK for C. This ensures that the Symbol Platform Integrator adds the Symbol Include and Library files to this Microsoft SDK. If the Microsoft SDKs for Windows Mobile are installed after the SMDK for C, run the Symbol Platform Integrator manually using the shortcut in the SMDK for C Start Menu program group.
5. Installing an older version of the SMDK for C onto a PC that already has a newer version installed is not recommended. If a roll back to an older version is required, then uninstall the newer version before installing the older version.

Adding a new WinCE 5.0 platform to a Visual Studio 2005 solution

SMDK vX.X for C includes various sample applications that provide examples of how to interface with the Symbol API functions. Each sample project contains a Visual Studio 2005 solution file. When opened in Visual Studio, the solution file allows developers to rebuild the sample for different target platforms. By default, each solution includes settings for targeting Pocket PC 2003 and Windows Mobile 5.0. When building a project for a Windows CE device; such as the MC3000, the Platform SDK (or PSDK) for that device must be installed and a new platform configuration must be added to the solution.

Adding a New Platform

A new platform configuration is added to the solution as follows:

1. Open the SLN file for the project.
2. In the *Build* menu, select *Configuration Manager*.
3. In the *Active Solution Platform* field, select *New*.
4. In the *Type or select the new platform:* field, select the desired platform (such as *MC3000c50b (ARMV4I)*).
5. In the *Copy settings from:* field, ensure to select *Windows Mobile 5.0 Pocket PC SDK(ARMV4I)*.
6. Check the *Create new project platforms* box.
7. Press the **OK** button in the *New Config* dialog.
8. Press the **Close** button in the *Configuration Manager* dialog.
9. Ensure to save the SLN and VCPROJ using the **SaveAll** button on the toolbar.

Removing a Platform

If by mistake the wrong platform is selected, from which to copy the settings, then remove the inappropriate platform before adding the correct platform. A platform can be removed as follows:

1. In the *Build* menu, *Select Configuration Manager*.
2. In the *Active Solution Platform* field, select *Edit*.
3. In the *Platforms:* field, select the desired platform and press the **Remove** button.
4. Press the **Yes** button in the *Are you sure you want to remove* message box.
5. Press the **Close** button in the *Configuration Manager* dialog.

Hello Scan Sample Applications in C / C++

Any good programming language reference guide includes a "Hello World" program to illustrate the most simplified example of a program written in a particular language. "Hello World" usually consists of a set of instructions that display the string "Hello World" to the console or screen device. This Developer Guide is no exception, and includes a variation on the "Hello World" program, called "Hello Scan." "Hello Scan" is the most simplistic form of a bar code scanning application, which implements the following operations:

- Open the scanner.
- Enable the default bar codes.
- Submit a read.
- User presses scan trigger.
- Display the data in a message box.
- Disable the scanner.
- Close the scanner.



NOTE The code examples presented in this chapter are for illustration purposes only and are not guaranteed to compile and run.

```
#include <windows.h>
#include <ScanCAPI.h>

int WINAPI WinMain(HINSTANCE hInstance,
                  HINSTANCE hPrevInstance,
                  LPWSTR lpszCmdLine,
                  int nCmdShow)
{
    HANDLE          hScanner      = NULL;
    LPSCAN_BUFFER   lpScanBuf     = NULL;
    DWORD           dwScanSize    = 7095;    // default scan buffer size

    SCAN_Open(TEXT("SCN1:"), &hScanner);
    SCAN_Enable(hScanner);
    lpScanBuf = SCAN_AllocateBuffer(TRUE, dwScanSize);
    SCAN_ReadLabelWait(hScanner, lpScanBuf, 0);
    MessageBox(NULL, (LPTSTR)SCNBUF_GETDATA(lpScanBuf), TEXT("HelloScan"), MB_OK);
    SCAN_Disable(hScanner);
    SCAN_DeallocateBuffer(lpScanBuf);
    SCAN_Close(hScanner);
    return 0;
}
```

Figure 3-5 Sample Application in 'C'



NOTE For detailed programming information, see [Chapter 10, Advanced Programming](#).

Device Updates

With the exception of the Printer API, all APIs for the SMDK for C are deployed in the device at the factory. Other than software downloads that may be necessary to update features, no device updates or CAB file installs are required.

Installing Printer Components

The Symbol Printer Driver Delivery Package is available in a zip file for installation from a host computer, or as a CAB file for installation on a device from AirBEAM or MSP. Both files are similar except that the CAB file does not contain documentation. The Symbol Printer Driver Delivery Package contains:

- Printer Driver CAB file which includes registry files
- Deployment Registry file that provides the registry settings necessary to install the Printer Driver CAB file upon a cold boot
- Terminal Default Settings Registry file which includes the registry settings that define the printing defaults for the device (if the user settings are not defined)
- User Default Settings Registry file which includes the registry settings that defined the user printer settings
- Printer Driver Copy file (printerdriver.cpy)
- Load Command file (Wceldcmd.exe)
- User Documentation (zip file only)
- Demos.

The Printer Driver Delivery Package zip file is installed on a host PC and then files are copied to the device using ActiveSync. The Printer Driver Delivery Package CAB file is copied onto a device using ActiveSync, AirBEAM or MSP and then it installs onto a device.

For detailed information about the Printer Driver Delivery Package, such as installing and removing printer drivers, preparing and modifying a user default registry file refer to the *Using the Printer Driver Delivery Package Guide* (p/n 72-70973-xx). This guide is included in the Symbol Printer Driver Delivery Package zip file.

For additional information about the Printer API, refer to the SMDK Help file and the guide, *Using Symbol Printing API* (p/n 72-70972-xx).

Software Tools for Application Development

SMDK for C is an extension to the Integrated Development Environments provided by Microsoft eMbedded Visual C++ 4.0 and Microsoft Visual Studio 2005. These tools must be installed prior to using SMDK for C.

Microsoft eMbedded Visual C++ 4.0 can be used to target SDK's for Windows CE 4.2, Windows CE 5.0 and Pocket PC 2003.

Microsoft Visual Studio 2005 can be used to target SDK's for Windows CE 5.0, Windows Mobile 5.0 and Pocket PC 2003.



NOTE While you may be able to write an application using eMbedded Visual C++ 4.0 and get it to run on a Windows Mobile 5.0 device, certain limitations exist. You will not be able to call new Windows Mobile 5.0 API functions and you will not be able to debug your application on a Windows Mobile 5.0 device.

Microsoft eMbedded Visual C++ 4.0

The Microsoft® eMbedded Visual C++ 4.0 tool delivers a complete desktop development environment for creating applications and system components for Windows® CE .NET-powered devices.

With new capabilities like C++ exception handling, Run Time Type Information (RTTI), include STL library components and a plethora of new debugger functionality, eMbedded Visual C++ 4.0 provides you with the power and flexibility to create the most advanced applications for Windows CE .NET.

SMDK for C is validated for use with Microsoft eMbedded Visual C++ 4.0 and service pack 2 which can be downloaded from

<http://msdn.microsoft.com/mobility/windowsmobile/downloads/default.aspx>

Microsoft Visual Studio 2005

Microsoft's Visual Studio 2005 replaces Microsoft's eVC4 and Visual Studio .NET 2003. Use Visual Studio 2005 to create managed and native applications for Motorola enterprise mobility devices. Visual Studio 2005 allows the developer to use one tool when writing applications in C, C++, C# and VB.NET. In addition, Visual Studio 2005 contains .NET Compact Framework 2.0.

Visual Studio 2005 includes an automatic project conversion wizard for applications created using Visual Studio.NET 2003 and eMbedded Visual C++ 4.0. With the release of Visual Studio 2005, the developer can now write C, C# and VB.NET applications for Motorola enterprise mobility devices using a single Integrated Development Environment from Microsoft.

eVB

eMbedded Visual Basic (eVB) is not supported by Microsoft. This continues to be true for Visual Studio 2005, as well as Windows Mobile 5.0 and Windows CE 5.0.

The following information regarding eMbedded Visual Basic (eVB) development comes directly from the Microsoft Web site.

Microsoft eMbedded Visual Basic (eVB) development is no longer supported on Pocket PC 2003 platforms. Microsoft continues to support the eVB run-time in Pocket PC 2003 devices as a RAM installable component so that older eVB applications can run on the new device. However, new eVB development for Pocket PC 2003 devices is not supported, leaving way for more powerful and robust development experiences with Visual Basic® .NET, Visual C#® .NET and the Microsoft® .NET Compact Framework.

Based on customer feedback, the Pocket PC 2003 devices includes the .NET Compact Framework in ROM. The .NET Compact Framework is a subset of the full .NET Framework that is specifically designed for smart devices. It is a far more comprehensive, language neutral application engine than the eVB specific runtime that was provided in the Pocket PC 2000 and Pocket PC 2002 devices.

This raises the question of how to migrate from eVB to Visual Basic .NET. The move from eVB to Visual Basic .NET is not easy but the benefits of migrating far outweigh the costs. The benefits are:

- Richer data types - eVB has only a 16-byte VARIANT; Visual Basic .NET uses the .NET Framework common type system.
- Faster execution - eVB is interpreted; Visual Basic .NET is JIT (just in time) compiled to native code prior to execution.
- Better error handling - eVB only supports "On Error"; Visual Basic .NET supports structured exception handling.
- Support for structures - not in eVB; Visual Basic .NET supports namespaces, classes and structures.
- Object-oriented - eVB is procedural; Visual Basic .NET supports full OOP (object orientated programming).
- First-class citizen - eVB is always playing catch-up with MFC; Visual Basic .NET is core .NET.
- Native XML Support - not in eVB; Visual Basic .NET has native support for XML and XML Web services.
- Better data model - ADOCE is far surpassed by ADO.NET; our best data model yet.
- Safer, more reliable execution - eVB is a scripting language; Visual Basic .NET is managed code.

There are a wide variety of resources to help developers make this transition, from porting labs to technical articles. One example is a technical article, ["Moving from eMbedded Visual Basic to Visual Basic .NET"](#) from Microsoft. The article outlines the advantages of moving from eVB to Visual Basic .NET and how it can be accomplished.

Tool Availability

While eMbedded Visual C++ 4.0 is available as a free download from the Microsoft website, Visual Studio 2005 must be purchased from an authorized dealer. It is also available to qualified MSDN Subscribers as a free download and as part of the disk set. Check the MSDN web site for availability.

Windows Mobile Developer Power Toys

The "Windows Mobile Developer Power Toys" is a set of utilities that help with the development and testing of Windows Mobile applications. While these utilities are not officially supported by Microsoft, they still provide some valuable help with application development.

One of the more frequently used tools in the Power Toy collection is the ActiveSync Remote Display program. This utility allows the developer to interact with a mobile device using the screen, mouse and keyboard of a connected PC. This is especially useful for testing applications on a device with no touch screen.

The Remote Display Control application is a way to remote the display and keyboard/mouse/touch screen of a Windows CE device to a desktop PC. The application uses TCP/IP and can work over ActiveSync connections to the Host Desktop (no routing off the host), via Ethernet, to any reachable Desktop or dialup (this does not include support for proxy servers, etc.). Remote Display Control works on any platform running Windows CE version 2.11 or later.

"Windows Mobile Developer Power Toys" is a free download from the Microsoft web site. At the time of this writing, the download was available at:

<http://www.microsoft.com/downloads/details.aspx?familyid=74473fd6-1dcc-47aa-ab28-6a2b006edfe9&displaylang=en>.

Chapter 4 Symbol Pocket Browser

Introduction

This chapter provides a complete explanation of the SDK architecture for developers who are programming for the Symbol PocketBrowser.

Symbol PocketBrowser is a browser specifically designed for Enterprise Applications running on Motorola enterprise mobility devices. Symbol PocketBrowser occupies the entirety of the device screen which gives the advantages of greater screen real estate and security by blocking the user from accessing the operating system. Symbol PocketBrowser is based on Microsoft Internet Explorer for Windows CE based devices and Microsoft Pocket IE for Windows Mobile based devices; using this model, developers can utilize all their favorite development tools from Notepad to Visual Studio or Dream Weaver.

Symbol PocketBrowser is supported on most Motorola enterprise mobility devices based on Windows CE Professional 4.2 or greater and Windows Mobile 2002 or greater.

Symbol PocketBrowser does not normally ship on the device from the factory and must be installed prior to use.

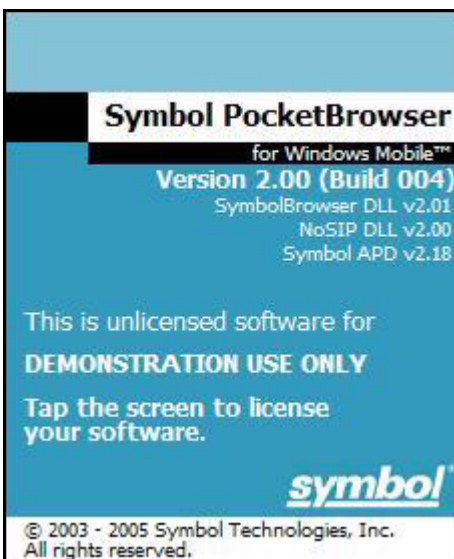


Figure 4-1 Symbol PocketBrowser Splash Screen

Architecture

Symbol PocketBrowser's unique functionality is implemented using a combination of META tags, ActiveX controls, JavaScript and device-configuration.

The META tags either activate functionality on the device, such as displaying the battery strength indicator or setup a call-back function (either a JavaScript function or URL) to execute when certain conditions are met, such as the ScannerNavigate tag which invokes the specified call-back when the scanner successfully decodes a barcode.

The ActiveX controls supplied with Symbol PocketBrowser allow for additional functionality such as reading from the device registry or dialling up a remote access connection.

Using a specific method on the SymbolBrowser.Generic ActiveX control, developers can invoke any of the Symbol PocketBrowser META tags using JavaScript (SPB 2.0 or greater only) allowing for greater control of the device.

Unique configurations allow for functionality to be defaulted into the device to reduce the amount of code required in each web page.

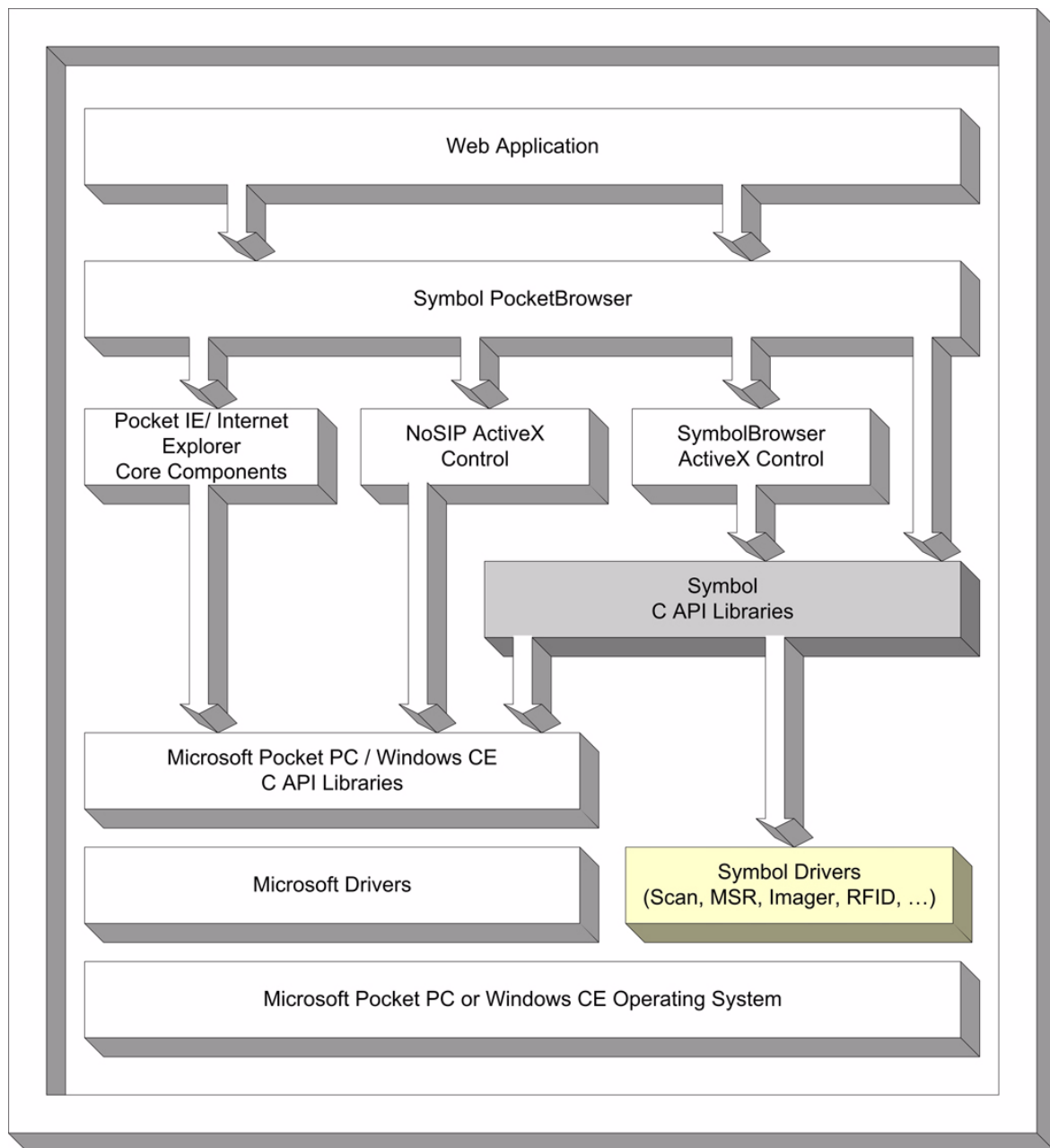


Figure 4-2 *Symbol PocketBrowser Architecture*

Symbol PocketBrowser Components

Symbol PocketBrowser provides all of the functions necessary for creating rich enterprise web applications that target Motorola enterprise mobility devices. The included Metatags and ActiveX controls expose enterprise mobility functions to developers who are creating web applications using HTML and JavaScript.

Although Symbol PocketBrowser is designed to work with all Motorola enterprise mobility devices running Windows Mobile or Windows CE Professional, it should only be used to develop applications for approved devices. Refer to the product download page for a complete listing of approved devices.

Symbol PocketBrowser installs into the memory of the device via an ActiveSync installer. A copy of the installation is placed in the \Application folder of the device which is a non-volatile folder. This copy allows for retention of the application even if a cold boot is performed. A help file is provided in the downloadable Zipped archive.

If the default install location is not changed, the components included in [Table 4-1](#) are installed in the folders noted.

Table 4-1 Symbol PocketBrowser Components

SPB Components	Description	Location
SymbolPB.exe (v1.0) SPB20_WM.exe (v2.x WM) SPB20_CE.exe (v2.x CE)	The main executable of Symbol PocketBrowser. The executable file contains all the META functionality. Users should launch this file when accessing Symbol PocketBrowser applications.	\program files
SymbolBrowser.DLL	Provides access to the Generic, NarrowBand (Symbol APD - Advanced Printer Driver)	\windows
NoSIP.DLL	Provides an alternative to the standard input box with additional functionality for SIP control and interactivity events	\windows

Usage Requirements

Symbol PocketBrowser can be installed on either a Windows 2000 or Windows XP PC. There are no other usage requirements.

Symbol PocketBrowser Features

Table 4-2 *Symbol PocketBrowser Features*

SPB Feature	Description
AirBeam ActiveX	Interface to AirBeam APIs and instruct device to perform updates from JavaScript.
Alarms	Specifies an action to be taken at a specific time and date.
Backlight	Adjust backlight level from the website.
Barcode Scanning	The barcode scanner can be configured to output the scanned data as keyboard presses or pass the data to a JavaScript function. Symbolologies can be configured on a page-by-page basis exposing the full functionality of the scanner engine.
Battery and Signal Strength Indicators	A battery power indicator and signal strength indicator, similar to that on a mobile phone, can be displayed.
Calibrate	Calls the screen alignment routine.
Extensive logging Capabilities	Flexibility to easily and centrally monitor mobile devices, reducing management and support effort.
Full Screen	Allows full screen mode on the various screen sizes of Motorola enterprise mobility devices, such as the MK2000 and MC3000.
Hot-Key support	All keys can be trapped and assigned to a JavaScript function or a URL. This makes selecting from a menu or list more intuitive.
HTTP/FTP File Transfer	Enables transfer of files (including signature capture and digital photos) for immediate proof of delivery, package condition and more.
Imager	Capture images to the device via SPB viewfinder window and send them to server over HTTP.
Key Remapping	A new VKCode for a key can be specified and the browser performs the translation, which is ideal for devices that have no TAB key.
Key State	Displays icons for Shift, Caps, Alt, Control and Function (where appropriate) and new orientation tags to support new visualizations.
Keyboard Mode	Specify the mode the keyboard should be in, i.e., Shift, Numlock, Caps, Function, etc. (where available).
Lock Stylus Input	Stops the stylus input from working, which is useful for applications where only keyboard input is required (does not support the CE PDT8800).
Logging	Output error messages, information and detailed debugging information to a file on the device or to a Server using MSP or SNMP.

Table 4-2 *Symbol PocketBrowser Features*

SPB Feature	Description
META ActiveX	Allows invoking any of the SPB 2.0's META Tags via JavaScript at any time through the application, not just at page load, i.e., enable/disable the scanner per entry field; reposition the SIP to accommodate entry fields; update a registry setting the warm boot the device to apply a system setting.
Minimize	The application can be programmed to minimize to allow access to the OS or another application without losing the state in the web application.
Offline Storage	Provides the ability to save captured data to a database.
OS Lock-Out	Completely locks the user out of the operating system (unless explicitly specified by the designer). Removes the address bar and navigation bar, normally visible in Pocket IE. Disables and hides the "Start" bar, so the user cannot select other applications, such as Solitaire. A designer can include the "Quit" META tag on a page. When this page is loaded, the application exits.
Patent Pending Push Navigation	Reduces network traffic, saves battery resources and delivers instant information and messages for faster response times and increased productivity.
Portable Printing	Using the Symbol Adaptive Printer Driver (APD), can print to several popular portable printers such as the Zebra Cameo and the O'Neil PrintPad.
RFID META Tag and ActiveX control	Provides full support for RFID Tag scanning without the need for third party software or wedge style applications.
Screen Rotation	The screen on Windows Mobile 2003SE devices can be rotated, allowing for improved screen layout and control.
Signature Capture ActiveX	Provides the ability to capture signatures from a touch screen.
Signature Capture Capabilities	Enables real-time proof of delivery applications with minimal development time and costs.
SNTP	Update the device clock from a network time server.
Suspend Device	Send the device into suspend mode from the website.
Trap Power-on	On each page, a URL or JavaScript function can be specified which the browser invoke when the unit is switched back on from suspend mode, i.e. for security purposes, the browser could be configured to navigate to a login screen.
Volume	Adjust the volume from the Website.

Hello Scan Sample Applications in JavaScript

Any good programming language reference guide includes a "Hello World" program to illustrate the most simplified example of a program written in a particular language. "Hello World" usually consists of a set of instructions that display the string "Hello World" to the console or screen device. This Developer Guide is no exception, and includes a variation on the "Hello World" program, called "Hello Scan." "Hello Scan" is the most simplistic form of a bar code scanning application, which implements the following operations:

- Open the scanner.
- Enable the default bar codes.
- Submit a read.
- User presses scan trigger.
- Display the data in a message box.
- Disable the scanner.
- Close the scanner.

✓ **NOTE** The code examples presented in this chapter are for illustration purposes only and are not guaranteed to execute.

```
<html>
<meta http-equiv="scanner" content="enabled">
<meta http-equiv="scanner" content="Javascript:doScan('%s');">
<script>
function doScan(barcode)
{
    alert("Barcode: " + barcode);
}
</script>
</body>
</html>
```

Figure 4-3 Sample Application in JavaScript

✓ **NOTE** For detailed programming information, see [Chapter 10, Advanced Programming](#).



Chapter 5 SMDK for .NET

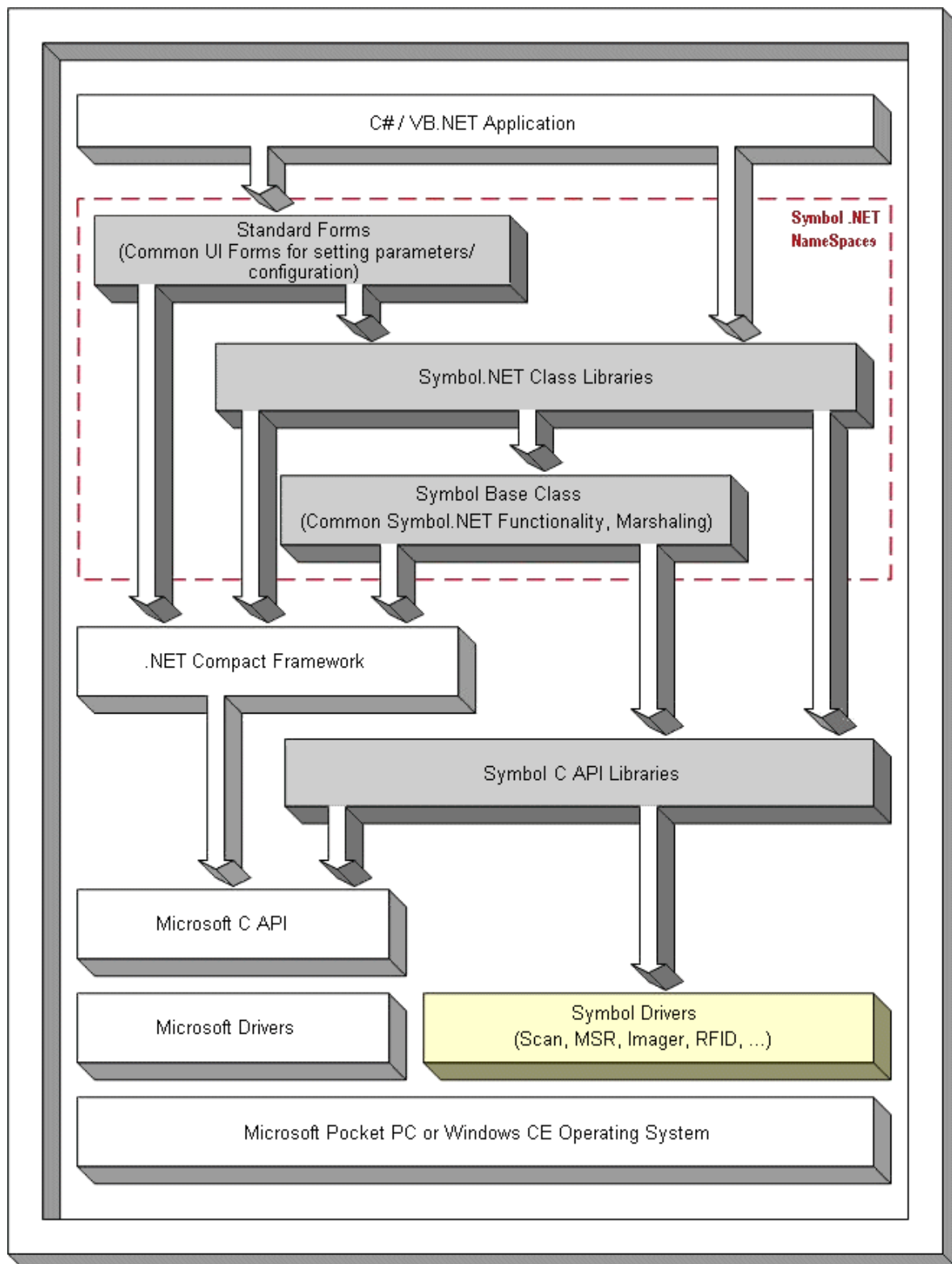
Introduction

This chapter provides a complete discussion of the SDK architecture for developers who are programming using the SMDK for .NET. SMDK for .NET provides all of the tools necessary to develop C# and VB.NET managed applications for Motorola enterprise mobility devices. These tools include class libraries, sample applications, and associated documentation. SMDK for .NET allows developers who are writing applications for the Microsoft® .NET Compact Framework to programmatically access the enterprise mobility features of the device.

Architecture

.NET programmers access Motorola enterprise mobility features through a set of class libraries. These class libraries provide a set of methods and properties that can be called from C# and VB.NET programs. The class libraries are implemented on the device as Dynamic Link Libraries (DLLs). These Symbol .NET class libraries accomplish their tasks by calling into Symbol C API functions as well as the Microsoft .NET Compact Framework functions. Each feature of the device has an associated library which allows programmatic access to the device driver functionality of that feature. For example, there is a class library for scanning that allows access to the scan API, which in turn provides access to the scan driver.

The Symbol .NET class libraries and the Microsoft .NET Compact Framework do not normally ship on the device from the factory and must be installed prior to running C# and VB.NET programs.

**Figure 5-1** SMDK for .NET Architecture

Microsoft .NET Compact Framework

The Microsoft .NET Compact Framework is a version of .NET Framework designed for smart devices. The .NET Compact Framework is a hardware-independent environment for running programs on resource-constrained computing devices, encompassing personal data assistants (PDAs) such as the Pocket PC, mobile phones, set-top boxes, automotive computing devices, and custom-designed embedded devices built with the Windows CE .NET operating system.

The .NET Compact Framework is a subset of the .NET Framework class library and also contains classes exclusively designed for it. It inherits the full .NET Framework architecture of the common language runtime and managed code execution.

The .NET Compact Framework provides the following key functionalities:

- Runs programs that are independent of hardware and operating systems.
- Supports common network protocols and connects seamlessly with XML Web services.
- Provides developers with a model for targeting their applications and components to either a wide range or specific category of devices.
- Provides benefits of design and optimization of limited system resources.
- Obtains optimal performance in generating native code using just-in-time (JIT) compilation.

The .NET Compact Framework uses the same class library documentation as the full .NET Framework.

SMDK for .NET supports devices running PocketPC 2003, Windows Mobile 5.0, Windows CE 4.2 and Windows CE 5.0.



NOTE Refer to the Microsoft Developer site to see comparisons between the .NET Framework and .NET Compact Framework, and to determine supported classes and members and classes exclusive to the .NET Compact Framework.

Programming Libraries

Table 5-1 lists the Class Libraries supported by SMDK for .NET.

Table 5-1 *Class Library Support*

Class Library	Description
Audio	Provides the ability for applications to control sounds played through the device's beeper and/or speaker, depending on the specific product.
Barcode	Provides applications with the ability to read bar code labels. This API supports 1D and 2D bar code scanning, image capture and signature capture. A variety of bar code scanning technologies are supported, including laser, contact wand, and CCD imaging
BarcodeForms	Displays a dialog box to view and modify bar code decoder parameters and scan parameters including symbologies, scan types and local feedback.
Display	Provides the ability to control the contrast and backlight display attributes
Fusion	Provides the ability to create and manage WLAN profiles, obtain WLAN information and diagnostic report.
Imaging	Allows an application to access the imaging device(s) on a system for the purpose of acquiring still images
Keyboard	Allows the keys on a device's keyboard to be mapped from one location to another.
MagStripe	Allows programmatic access to MSR 3000 API compliant accessories.
MKSeries	Provides the ability to configure MK Series related device settings including raster mode scanning, continuous trigger, keycode mappings to button ids, and protected mode activities.
Notification	Provides the ability for applications to control the device's notification devices such as LEDs, pager and beeper.
Printing	Provides the ability for applications to print bar codes, text, bitmaps and lines.
ResourceCoordination	Provides the ability for applications to monitor trigger activity, query unique unit identification and temperature information and retrieve configuration data.
RFID	Provides the ability to read RFID tags. This API supports reading Class 0 and Class 1 tags. This class library has been deprecated and is planned for removal in future versions. Please start using the RFID2 Class library.

Table 5-1 *Class Library Support (Continued)*

Class Library	Description
RFID2	Provides the ability to read RFID tags using a common RFID interface for MC9090, RD5000 and XR400 RFID devices.
StandardForms	Provides an easy and quick way for developers to create user interfaces for viewing and modifying parameters defined in the SMDK namespaces. Also provides a dialog box for displaying and selecting a list of available device objects.
WirelessLAN	Provides the ability to obtain WLAN information including the radio status, ESS ID, Signal, MAC Address, etc.

SMDK for .NET Components

The SMDK for .NET installation package contains all of the files necessary for creating .NET applications for Motorola enterprise mobility devices.

Once installed, the SMDK for .NET components can be easily accessed using the "Symbol Mobility Developer Kit for .NET" program group on the Windows Start menu. This program group provides access to the Help file, Readme file and the Sample applications.

If the default install location is not changed, the components included in [Table 5-2](#) are installed in the folders noted.

Table 5-2 *SMDK for .NET Installed Components*

Components	Description	Location
Class library assemblies	Dynamic Link Library (DLL) implementations of the Symbol class libraries.	For Visual Studio .NET 2003: \Program Files\Microsoft Visual Studio .NET 2003\CompactFrameworkSDK\v1.0.5000\Windows CE For Visual Studio 2005: \Program Files\Microsoft Visual Studio 8\SmartDevices\SDK\CompactFramework\2.0\v2.0\WindowsCE
Help file	Provides a reference for the Symbol Class Library namespaces, detailing all methods and properties.	\Program Files\Symbol Mobility Developer Kit for .NET\vX.X\Windows CE\Help Files

Table 5-2 SMDK for .NET Installed Components (Continued)

Components	Description	Location
Readme file	Includes release notes, usage information, and any late additions to the documentations. This file should always read prior to development.	\Program Files\Symbol Mobility Developer Kit for .NET\vX.X\Windows CE\Help Files
Sample applications	C# and VB.NET sample applications, provided with full source code.	\Program Files\Symbol Mobility Developer Kit for .NET\vX.X\Windows CE\Samples
Class library assemblies	Dynamic Link Library (DLL) implementations of the Symbol class libraries.	For Visual Studio .NET 2003: \Program Files\Microsoft Visual Studio .NET 2003\CompactFrameworkSDK\v1.0.5000\Windows CE For Visual Studio 2005: \Program Files\Microsoft Visual Studio 8\SmartDevices\SDK\CompactFramework\2.0\v2.0\WindowsCE

Usage Requirements

Microsoft Visual Studio.NET 2003 or Visual Studio 2005 must be installed on the development PC before installing SMDK for .NET. If neither version of Visual Studio is found, an error is displayed and the installation is aborted.

Install Requirements for Visual Studio.NET 2003:

- Microsoft® Windows 2000 or Microsoft® Windows XP
- Microsoft® Visual Studio.NET 2003
- Microsoft® Visual Studio .NET 2003 Add-on Pack
- Microsoft ActiveSync 4.0 or higher

Install Requirements for Visual Studio 2005:

- Microsoft® Windows 2000 or Microsoft® Windows XP
- Microsoft® Visual Studio 2005
- Microsoft® Windows Mobile 5.0 SDK for PocketPC
- Microsoft ActiveSync 4.0 or higher



NOTE The installed version of Visual Studio 2003/2005 must support Mobile device development. Express versions of Visual Studio do not support Mobile device development.

Software Tools for Application Development

A version of Microsoft Visual Studio is required to develop .NET applications. There are two versions of the tool supported: Microsoft® Visual Studio® .NET 2003 and Microsoft's Visual Studio 2005.

SMDK for .NET supports using Visual Studio 2003 and Compact Framework 1.0 to target devices running PocketPC 2003, Windows CE 4.2 and Windows CE 5.0.

SMDK for .NET supports using Visual Studio 2005 and Compact Framework 2.0 to target devices running PocketPC 2003, Windows Mobile 5.0, Windows CE 4.2 and Windows CE 5.0.

Microsoft® Visual Studio® .NET 2003

Microsoft® Visual Studio® .NET 2003 allows the Visual Basic .NET or C# programmer to create managed code applications (i.e., those applications that target the .NET common language runtime) for the Pocket PC 2000, Pocket PC 2002 and Pocket PC 2003 devices. In order to develop managed code applications for Pocket PC 2000 and Pocket PC 2002 devices, use Visual Studio .NET 2003 out of the box. Visual Studio .NET 2003 automatically installs the .NET Compact Framework into RAM on these devices during development. The .NET Compact Framework is already in ROM in all Pocket PC 2003 devices and is supported by Visual Studio .NET 2003 via the Pocket PC 2003 SDK.

Microsoft Visual Studio .NET 2003 allows you to create managed code. Useful for much more than just device development, Visual Studio .NET enables you to select from traditional Web and client models to properly architect complete systems, including mobile applications.

Visual Studio .NET doesn't restrict you to device, Web services or Windows programming-it does all three. This enables you to rapidly build a broad range of applications so you can reduce IT operating costs and integrate more easily and quickly with the latest applications, systems and devices.

Visual Studio .NET 2003 is included in MSDN Universal subscriptions. For those who do not have an MSDN subscription, the tool can be purchased from Microsoft or a Microsoft dealer.

Microsoft Visual Studio 2005

Compatibility

Visual Studio 2005 includes an automatic project conversion wizard for project files created with Visual Studio .NET 2003 and embedded Visual C++ 4.0.

With the release of Visual Studio 2005, the developer can write C, C++, C# and VB.NET applications for Motorola enterprise mobility devices with a single tool.



NOTE If developing applications for Windows Mobile 5.0, Visual Studio 2005 is required.

Developing Applications

How to Use SMDK for .NET

Once SMDK for .NET is installed, creating .NET Compact Framework applications is fairly simple. Follow the steps below to create a new Symbol enabled .NET application.

1. Create a new "Smart Device Application" project that uses either the Microsoft Visual Basic.NET or Microsoft Visual C# languages.
2. From the *Project* pull-down menu select *Add Reference...*
3. From the list of .NET assemblies, select the "Symbol" assembly as well as the particular SMDK assembly that matches the functionality of the application being developed. For example, for bar code scanning applications select *Symbol* and *Symbol.Barcode*.
4. Begin programming the application. Refer to the *SMDK .Net Help Documentation* for information on the methods and properties of each class.

Adding the BarcodeReader Design Time Control to the Toolbox

The Design Time Controls, feature of SMDK for .NET, can be dragged onto a form.

When installed for use with Visual Studio 2005, SMDK for .NET automatically installs three design time controls (Barcode, MagStripe and Imaging) to the Toolbox.

Only the BarcodeReader design time control is available for use with Visual Studio .NET 2003. Before the BarcodeReader Design Time Control can be used, it must be added to the Visual Studio. NET 2003 toolbox.

To add BarcodeReader Design Time Control to the Visual Studio .NET 2003 toolbox:

1. Activate the *Toolbox* window (*View - Toolbox*) from Visual Studio .NET.
2. Within the Toolbox, right click the tab in which to put the BarcodeReader control and select *Add/Remove Items...* from the pop-up menu.
3. In the *Customize Toolbox* dialog, select the *.NET Framework Components* tab.
4. Click **Browse** to locate the *Symbol.Barcode.Design.dll*. This DLL can be found at the following location:
`\Program Files\Microsoft Visual Studio.NET 2003\CompactFrameworkSDK\v1.0.5000\Windows CE\Designer\.`
5. Open *Symbol.BarcodeDesign.dll*. The BarcodeReader control appears in the *Customize Toolbox* dialog.
6. Click **OK** to close the dialog.

Application Deployment through Visual Studio.NET

During deployment, Visual Studio automatically copies the .NET Compact Framework and SMDK CAB files to the device, if they were not already installed. The application is then deployed onto the device.

For devices with .NET Compact Framework or SMDK class libraries installed, Visual Studio downloads the application only.

Hello Scan Sample Application in C#/VB.NET

Any good programming language reference guide includes a "Hello World" program to illustrate the most simplified example of a program written in a particular language. "Hello World" usually consists of a set of instructions that display the string "Hello World" to the console or screen device. This Developer Guide is no exception, and includes a variation on the "Hello World" program, called "Hello Scan". "Hello Scan" is the most simplistic form of a bar code scanning application, which implements the following operations:

- Open the scanner.
- Enable the default bar codes.
- Submit a read.
- User presses scan trigger.
- Display the data in a message box.
- Disable the scanner.
- Close the scanner.



NOTE The code examples presented in this chapter are for illustration purposes only and are not guaranteed to compile and run.

The following code assumes the presence of a Windows Form "Form1" and demonstrates basic steps to make this form scan enabled. *Form1_Load* and *Form1_Closing* are functions called upon respectively loading and closing of the form.

```
private Symbol.Barcode.Reader MyReader = null;
private Symbol.Barcode.ReaderData MyReaderData = null;

private void Form1_Load(object sender, System.EventArgs e)
{
    MyReader = new Symbol.Barcode.Reader();
    MyReaderData =
        new Symbol.Barcode.ReaderData(Symbol.Barcode.ReaderDataTypes.Text,
        Symbol.Barcode.ReaderDataLengths.DefaultText);
    MyReader.ReadNotify += new EventHandler(MyReader_ReadNotify);
    MyReader.Actions.Enable();
    MyReader.Actions.Read (MyReaderData);
    return;
}

private void Form1_Closing(object sender, System.ComponentModel.CancelEventArgs e)
{
    MyReader.Actions.Flush();
    MyReader.Actions.Disable();
    MyReader.Dispose();
    MyReaderData.Dispose();
    return;
}

private void MyReader_ReadNotify(object sender, EventArgs e)
{
    System.Windows.Forms.MessageBox.Show(MyReaderData.Text, "HelloScan");
    MyReader.Actions.Read(MyReaderData);
    return;
}
```

Figure 5-2 Sample Application in 'C#'

The following code assumes the presence of a Windows Form "Form1" and demonstrates basic steps to make this form scan enabled. Form1_Load and Form1_Closing are functions called upon respectively loading and closing of the form.

```
Private MyReader As Symbol.Barcode.Reader = Nothing
Private MyReaderData As Symbol.Barcode.ReaderData = Nothing
Private MyEventHandler As System.EventHandler = Nothing

Private Sub Form1_Load(ByVal sender As System.Object, ByVal e As System.EventArgs) Handles MyBase.Load
    MyReader = New Symbol.Barcode.Reader
    MyReaderData =
        New Symbol.Barcode.ReaderData(Symbol.Barcode.ReaderDataTypes.Text,
            Symbol.Barcode.ReaderDataLengths.DefaultText)
    MyEventHandler = New System.EventHandler(AddressOf MyReader_ReadNotify)
    AddHandler MyReader.ReadNotify, Me.MyEventHandler
    MyReader.Actions.Enable()
    MyReader.Actions.Read(MyReaderData)
End Sub

Private Sub Form1_Closing(ByVal sender As Object, ByVal e As
System.ComponentModel.CancelEventArgs) Handles MyBase.Closing
    MyReader.Actions.Flush()
    MyReader.Actions.Disable()
    MyReader.Actions.Dispose()
    MyReaderData.Dispose()
End Sub

Private Sub MyReader_ReadNotify(ByVal o As Object, ByVal e As EventArgs)
    System.Windows.Forms.MessageBox.Show(MyReaderData.Text, "HelloScan")
    MyReader.Actions.Read(MyReaderData)
End Sub
```

Figure 5-3 Sample Application in VB.NET



NOTE For detailed programming information, see [Chapter 10, Advanced Programming](#).

Device Updates

Before using the SMDK for .NET with a Motorola enterprise mobility device, the native code drivers may need to be updated. It is strongly recommended that you update the device with the latest DLLs or registry entries to avoid incompatibilities.

Each released SMDK for .NET is tested for compatibility with a wide range of Motorola devices. Refer to the *Device Compatibility* section of the SMDK for .NET download page for a list of these devices. Refer to the individual device product pages for a list of the latest software updates available for each devices. All of this information can be found on the Support Central website located at <http://support.symbol.com>.

✓ **NOTE** A cold boot must be performed after any update to ensure that the files were installed into the system.

Application Mass Deployment for C#/VB.NET Applications

Table 5-3 lists the Mass Deployment directories provided by SMDK for .NET.

Table 5-3 Application Mass Deployment - SMDK for .NET

Devices OS/CABs	Location
Windows CE 4.2, Windows CE 5.0, Windows Mobile 2003 or Windows Mobile 5.0	\Program Files\Symbol Mobility Developer Kit for .NET\vX.X\Windows CE\MassDeployment\CE.NETorWM2003
Windows CE 4.2	\Program Files\Symbol Mobility Developer Kit for .NET\vX.X\Windows CE\MassDeployment\MKSeries

The directories listed in Table 5-3 include files (.cab, .cpy, and .reg) that can be used to copy and install the .NET CF and SMDK CAB files from an Application folder into the Windows folder on a cold boot. The CAB files are installed using Startup.exe to launch wceload.exe (standard CAB installation method), or wceldcmd.exe (UI-less install) with the command line that contains the name of the CAB file.

✓ **NOTE** NET CF CAB files are not provided in the MassDeployment directory. These files need to be manually copied from the Visual Studio directory to the Application folder located on the Motorola enterprise mobility device. In Visual Studio .NET 2003, the .NET CF CAB files can be found in the folder "Arm" or "Armv4" under "\Program Files\Microsoft Visual Studio .NET 2003\CompactFrameworkSDK\v1.0.5000\Windows CE\<Platform>". In Visual Studio 2005, the .NET CF CAB files can be found in the folder "Armv4" or "Armv4i" under "C:\Program Files\Microsoft Visual Studio 8\SmartDevices\SDK\CompactFramework\2.0\v2.0\WindowsCE\<Platform>".



Chapter 6 SMDK for Java

Introduction

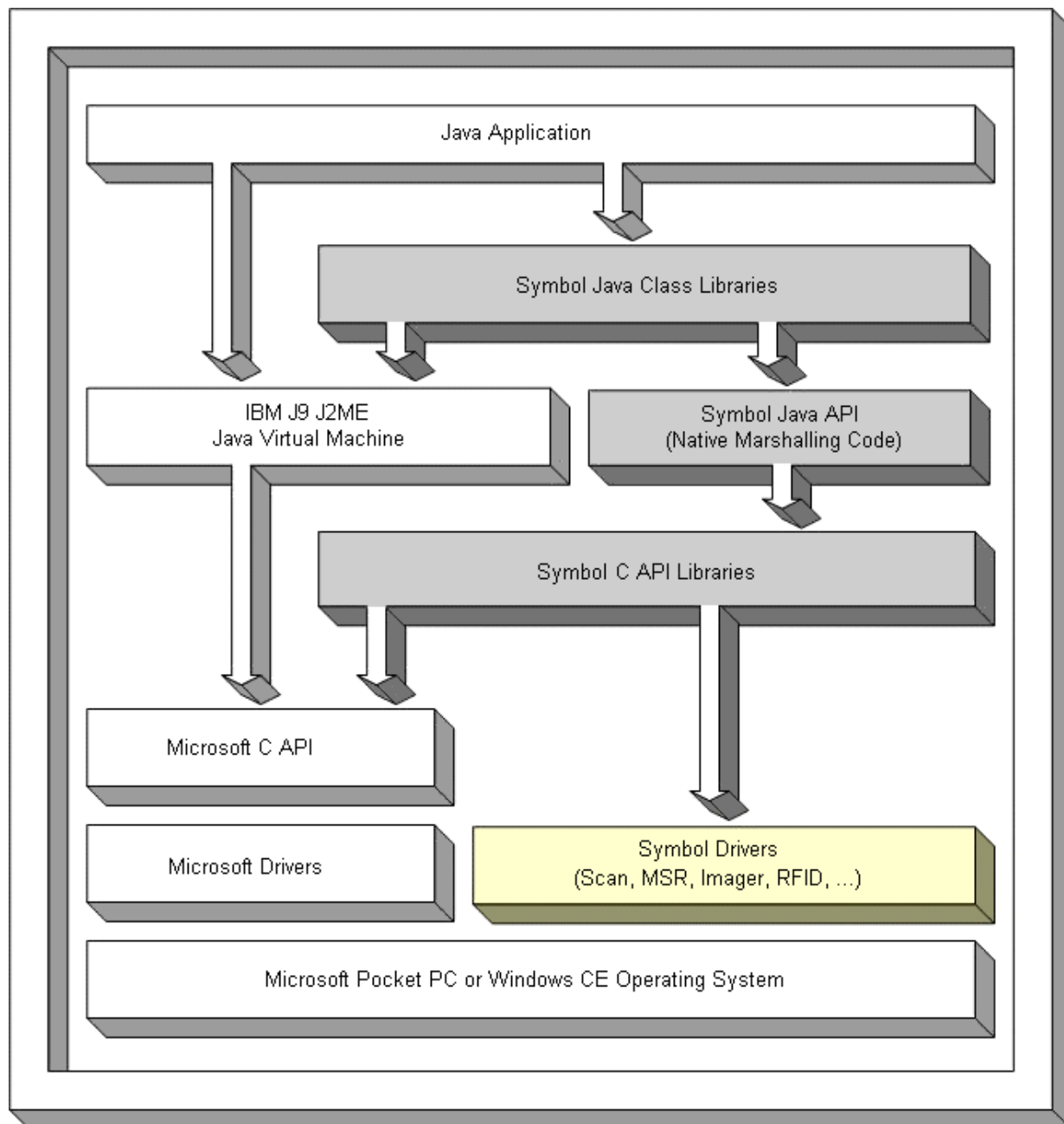
This chapter provides a complete discussion of the native SDK architecture for developers who are programming using the SMDK for Java. SMDK for Java provides a set of tools necessary to develop Java applications for Motorola enterprise mobility devices. These tools include class libraries, sample applications for each class library type and the associated documentation. SMDK for Java allows J2ME developers to programmatically access the enterprise mobility features of their devices, such as the barcode scanner and the MSR (Magnetic Stripe Reader).

The developer kit includes source code for the Java API, native DLLs for marshalling Java data types to and from native data types, sample applications and CAB files for the installation of the Symbol class libraries.

The Symbol Java class libraries and the IBM J9 JVM do not normally ship on the device, from the factory and must be installed prior to running Java programs.

Architecture

Java programmers access Motorola enterprise mobility features through a set of class libraries. These class libraries provide a set of fields, functions and interfaces that can be used in Java programs. The class libraries are implemented on the device as Java Classes with Java Native Interface (JNI) DLLs. These Java class libraries accomplish their tasks by calling into Symbol API marshalling code as well as the IBM J9 JVM functions. Each hardware device has an associated library, which allows programmatic access to the device driver functionality of that device. For example, there is a class library for scanning that allows access to the scan API, which in turn accesses the scan driver.

**Figure 6-1** SMDK for Java Architecture

Samples

The SMDK for Java includes a Symbolsamples.jar file that contains compiled sample applications for Audio, Display, Keyboard, MSR, Notify, Power, Print, Scan, WLAN etc.

Java Virtual Machine

In order to run Java applications such as the provided samples, a Java Virtual Machine (JVM) is required. The JVM is a platform-independent execution environment that converts Java bytecode into machine language and executes it. SMDK for Java is validated with IBM J9 Personal Profile 1.0.

J2ME JVM

The Java 2 Platform, Micro Edition is the edition of the Java platform that is targeted at small, standalone or connectable consumer and embedded devices. The J2ME technology consists of a java virtual machine and a set of APIs suitable for tailored runtime environments for these devices. The J2ME technology has two primary kinds of components - configurations and profiles. J2ME configurations have profiles associated to them. For example, a Connected Device Configuration (CDC) is available in two profiles: Foundation and Personal while Connected Limited Device Configuration (CLDC) is available in Mobile Information Device Profile (MIDP).

J9 JVM

J9 JVM is a J2ME Java Virtual Machine from IBM. Competing J2ME JVMs are Sun PersonalJava, Insignia Jeode, and NSIcom CrEme.

Programming Libraries

Table 6-1 lists the Java Classes supported by SMDK for Java.

Table 6-1 *Java Class Support*

Class Library	Description
Audio	Provides the ability for applications to control sounds played through the device's beeper, and speaker if available.
Display	Provides the ability for applications to control the display contrast and backlight.
Fusion	Provides applications with the ability to access wireless networking. (Currently supports WLAN only.)
Imager	Provides applications with the ability to access the imaging device(s) on a system for the purpose of acquiring still images.
Keyboard	Provides the ability for applications to control the keyboard state (CAPSLOCK, CONTROL, FUNCTION, NUMLOCK...etc.) as well as remap keys to new values.
KeyLight	Provides the ability for applications to control the backlight for the keypad.
Misc	Provides the ability for applications to get the configuration information of the mobile device, such as the UUID (Unique Unit ID) and the temperature of the unit.
MSR	Provides the ability for applications to read data from a card with a standard magnetic stripe.
Notification	Provides the ability for applications to control the device's notification devices, such as LEDs, pager, and beeper.
Power	Provides the ability for applications to get the battery status as well as get and set the state of power managed devices.
Printer	Provides the ability for applications to print bar codes, text, bitmaps and lines.
RFID	Provides applications with the ability to perform RFID related features on devices equipped with an RFID Reader.
Scanner	Provides applications with the ability to read bar code labels.
Spectrum24	Provides applications with the ability to get statistical information for the Wireless LAN radio.

Table 6-1 *Java Class Support (Continued)*

Class Library	Description
Trigger	Provides applications with the ability to monitor the scan trigger buttons on the mobile device. Applications can register for trigger event notifications and get status for any available trigger.
WAN	Provides applications with the ability to access wide area networking such as telephone calls, SMS etc.
Wedge	Provides the ability for applications to redirect data from one medium to another.

SMDK for Java Components

The SMDK for Java components can be easily accessed using the "Symbol Mobility Developer Kit for Java" program group on the Windows *Start* menu. This program group provides access to the Help file, Readme file and the sample applications.

If the default install location is not changed, the components included in [Table 6-2](#) can be found at the locations specified.

Table 6-2 SMDK for Java Installed Components

Component	Description	Location
FusionInterface.dll	Fusion Public API wrapper DLL	\Program Files\Symbol Mobility Developer Kit for Java\X.X\bin
SmdkJavaSamLauncher.htm	SMDK for Java sample launcher	\Program Files\Symbol Mobility Developer Kit for Java\X.X
symbolclasses.jar	Symbol SMDK for Java compiled class libraries.	\Program Files\Symbol Mobility Developer Kit for Java\X.X\bin
symbolsamples.jar	Compiled versions of the Symbol sample applications: J_AudioSample1, J_DisplaySample1, J_KeyboardSample1, J_MSRSample1, J_NotifySample1, J_PowerSample1, J_PrintSample1, J_ScanSample1, J_WLANSample1, etc.	\Program Files\Symbol Mobility Developer Kit for Java\X.X\bin
symboljavaapi.dll	JNI wrapper DLL	\Program Files\Symbol Mobility Developer Kit for Java\X.X\bin
symbol.java.arm.cab	Symbol class library installation	\Program Files\Symbol Mobility Developer Kit for Java\X.X\cabs
symbol.jsamples.arm.cab	Symbol sample application installation	\Program Files\Symbol Mobility Developer Kit for Java\X.X\cabs
symbol.jpos.arm.cab	Symbol JavaPOS service installation	\Program Files\Symbol Mobility Developer Kit for Java\X.X\cabs
symbol.jpos.jsamples.arm.cab	Symbol JavaPOS samples installation	\Program Files\Symbol Mobility Developer Kit for Java\X.X\cabs
Updates to native drivers	Drivers and wrapper DLL updates	\Program Files\Symbol Mobility Developer Kit for Java\X.X\DriverUpdates
*.java	SMDK sample application source code	\Program Files\Symbol Mobility Developer Kit for Java\X.X\samples\smdk

Table 6-2 *SMDK for Java Installed Components (Continued)*

Component	Description	Location
*.java	JavaPOS sample source code	\Program Files\Symbol Mobility Developer Kit for Java\X.X\samples\javapos
*.java	Source files of SMDK for Java and JavaPOS Services	\Program Files\Symbol Mobility Developer Kit for Java\X.X\src
*.cpy	Copyfiles for auto-install of CAB files	\Program Files\Symbol Mobility Developer Kit for Java\X.X\cabs
Readme.htm	Documentation on how to use product	\Program Files\Symbol Mobility Developer Kit for Java\X.X
WSDD5.7.1-HowTo.pdf	Whitepaper on how to use SMDK with WSDD	\Program Files\Symbol Mobility Developer Kit for Java\X.X
Index.htm	JavaDoc API documentation	\Program Files\Symbol Mobility Developer Kit for Java\X.X\JavaDocs
Eclipse resources	Eclipse plug-in and SWT (Standard Widget Toolkit) libraries	\Program Files\Symbol Mobility Developer Kit for Java\X.X\eclipse

Usage Requirements

The SMDK for Java is designed for installation on PCs with the Microsoft Windows 2000 and Microsoft Windows XP operating systems. The following software is recommended to obtain full use of the SMDK for Java:

- Microsoft ActiveSync 3.7 or higher
- IBM® WebSphere Studio Device Developer (WSDD) 5.7.1 or higher

Configuring the Device for Java

The following step-by-step instructions explain how to install the SMDK for Java components on a Motorola enterprise mobility device. Once these steps are complete, Java applications that target enterprise mobility features, such as scanning, can run.

1. Install the SMDK for Java on the development PC.
2. Connect a Symbol mobile device to the development PC.
3. Open the ActiveSync File Explorer.
4. Use the drag and drop feature to copy the CAB files listed below onto the device. The files are located in `\Program Files\Symbol Mobility Developer Kit for Java\v1\cabs`.
 - symbol.java.arm.cab
 - symbol.jsamples.arm.cab.

✓ **NOTE** Due to the IBM JVM package size (greater than 8 MB), this transfer may take some time. For Windows Mobile 2003, the IBM J9 JVM CAB file is provided with WSDD5.7.1. JVMs for other platforms such as CE4.2, CE5.0 and WM5.0 evaluation versions can be downloaded from IBM web site. See the readme.htm of SMDK for Java for more information.

Table 6-3 defines where files are placed on the device.

Table 6-3 File Locations

Component	Location on Device
FusionInterface.dll	\Windows
IBM J9 JVM	\Program Files\J9\PPR010
Sample Application Shortcuts	\Windows\Start Menu\Programs\Java Samples\
Sample Deployment Location	\JavaSamples\
symbolclasses.jar	\Java\
symboljavaapi.dll	\Windows\
symbolsamples.jar	\Java\

5. Once all CAB files are copied to the device, use the device's File Explorer to find each CAB file and launch each file to begin the installation. Follow the on screen instructions.

6. Test the installation by running the sample applications. Tap *Start - Programs - Java Samples* on the device. The SMDK for Java shortcuts displays.

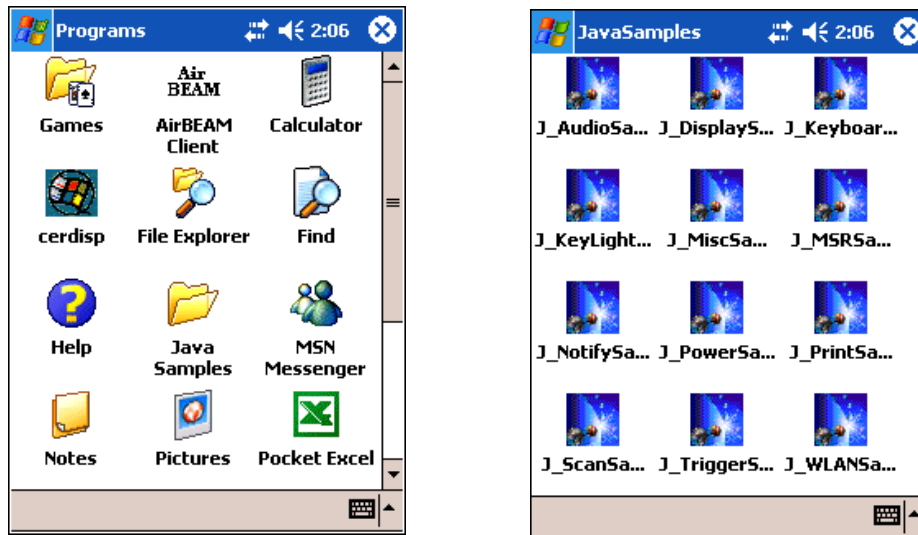


Figure 6-2 Java Shortcuts

7. Run the associated sample application for the feature to verify.

Developing Applications

To develop Java applications using the SMDK for Java:

1. Use the Javadocs documentation as a reference during development.
2. Compile your applications using any standard* v1.1 or higher Java compiler. WSDD5.7.1 is recommended.
3. Place the symbolclasses.jar file in the classpath during the build as well as when you run your application.

✓ **NOTE** IBM's J2ME JVMs do not support all java classes (such as the java.swing.* classes). Care should be taken that only available J2ME Connected Device Configuration with Foundation and Personal Profile classes are used within your program.

Remote Debugging

Use IBM's Websphere Studio Device Developer for remote debugging. Refer the *WSDD5.7.1-HowTo.pdf* document for detailed information on how to use the SMDK for Java with WebSphere 5.7.1. This document is located in the root directory of the SMDK for Java ("Program Files\Symbol Mobility Developer Kit for Java\X.X").

Hello Scan Sample Application in Java

Any good programming language reference guide includes a "Hello World" program to illustrate the most simplified example of a program written in a particular language. "Hello World" usually consists of a set of instructions that display the string "Hello World" to the console or screen device. This Developer Guide is no exception, and includes a variation on the "Hello World" program, called "Hello Scan." "Hello Scan" is the most simplistic form of a bar code scanning application, which implements the following operations:

- Open the scanner.
- Enable the default bar codes.
- Submit a read.
- User presses scan trigger.
- Display the data in a message box.
- Disable the scanner.
- Close the scanner.



NOTE The code examples presented in this chapter are for illustration purposes only and are not guaranteed to compile and run.

```

import symbol.*;

public class HelloScan implements ScannerListener
{
    Scanner scanner;

    public HelloScan()
    {}

    public static void main(String argv[])
    {
        (new HelloScan()).go();
    }

    public synchronized void go()
    {
        try
        {
            System.out.println("Hello Scan");
            ScannerDevice[] devList = Symbol.getScannerDeviceList();
            scanner = new Scanner(devList[0]);
            scanner.enable();
            scanner.read(null, this);
            wait();
            scanner.disable();
            scanner.dispose();
        }
        catch (Exception e)
        {
            System.out.println(e.getMessage());
        }
    }

    // On scan complete, 'readNotify' is called
    public synchronized void readNotify(ScanReadInfo result)
    {
        System.out.println(((TextData)result.data).text);
        notify();
    }
}

```

Figure 6-3 *Sample Application in Java*



NOTE For detailed programming information, see [Chapter 10, Advanced Programming](#).

Application Mass Deployment for Java Applications

To automatically install CAB files on cold boot of the device, CPY files and CAB files (located in the "\Program Files\Symbol Mobility Developer Kit for Java\vX.X\Cabs" directory) can be copied to the non-volatile "\Application" folder on the device. During a cold boot, the CAB files are copied to the "\Windows\Startup" folder and installed.

Software Tools for Application Development

IBM WebSphere Studio Device Developer is required to develop Java applications for Motorola enterprise mobility devices. WSDD is available for purchase from IBM or an authorized IBM dealer. A trial version is available directly from IBM.

IBM WebSphere Studio Device Developer

IBM WebSphere Studio Device Developer (WSDD) provides an integrated development environment (IDE) for building, testing, and deploying Java™ 2 Micro Edition (J2METM) applications that run on wireless devices such as cellular telephones, personal digital assistants (PDA), and handheld computers.

Creating a new WSDD Project

✓ **NOTE** Existing projects can also be modified to access the Symbol Java Classes. Updating an existing project to access SMDK for Java classes is simply a matter of accessing the same dialogs discussed in this section.

1. Select *New Project...* from the *File* pull down menu to create a new J2ME Project.

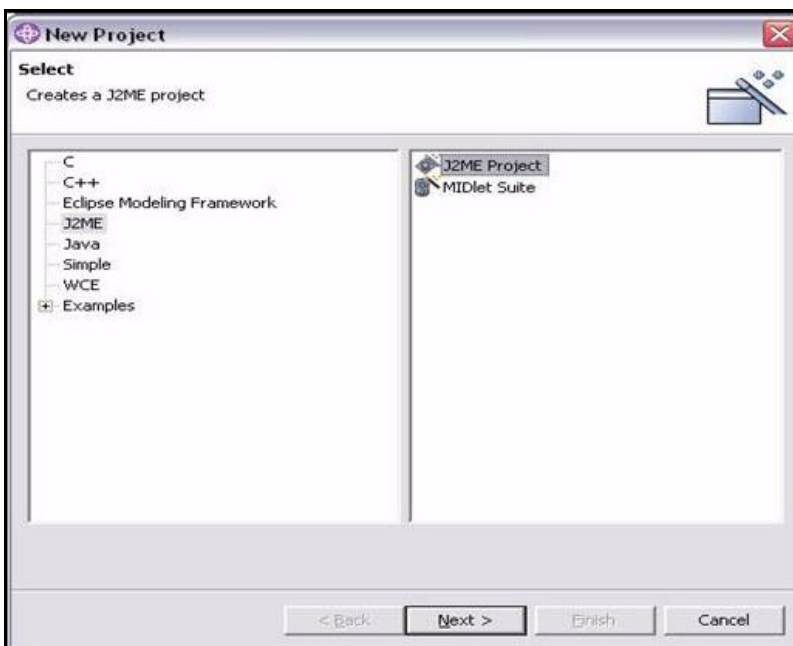


Figure 6-4 New Project Window

2. Click **Next**. The *J2ME Project* window appears.

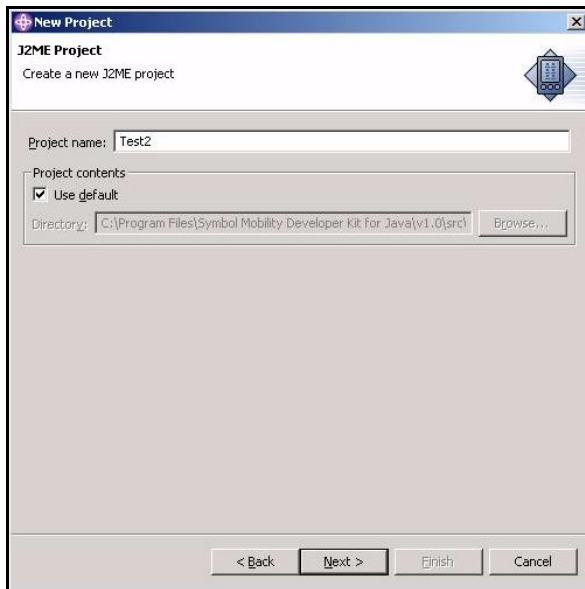


Figure 6-5 *J2ME Project Window*

3. Enter a name for the project in the *Project Name*: text box. In [Figure 6-5](#), the new project name entered is *Test2*.
4. Click **Next**. The *Class Library Selection* window appears

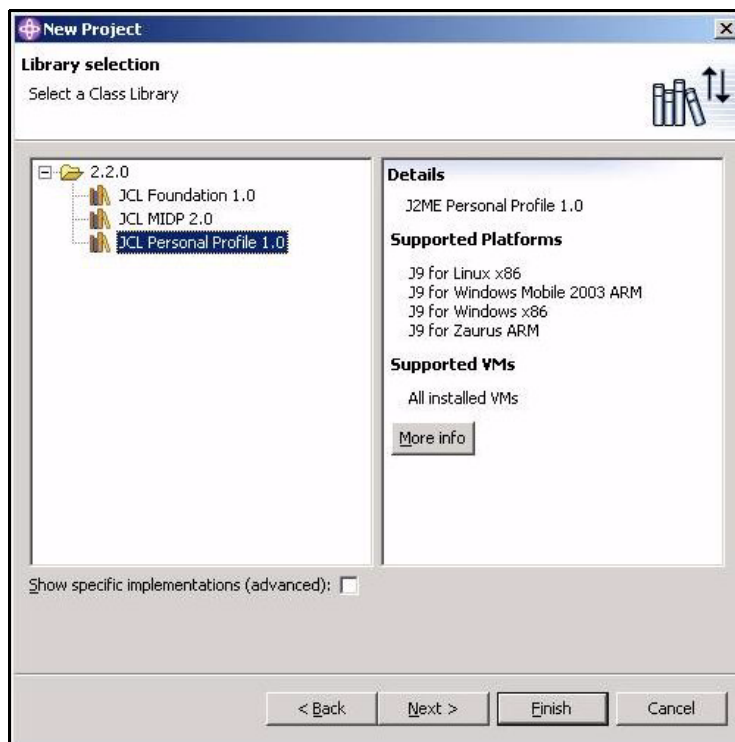


Figure 6-6 *Class Library Selection Window*

5. On the *Class Library Selection* window, select *JCL Personal Profile 1.0* as the target class library. This configuration provides the Personal Profile classes.

6. Click **Next**. The *Define the Java build settings* dialog appears.

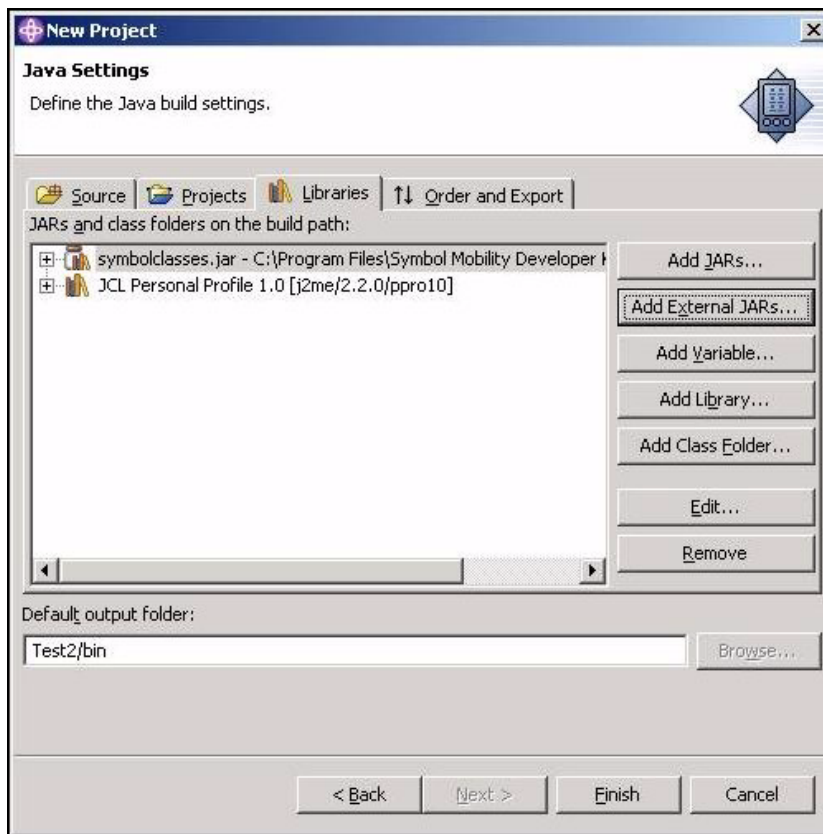


Figure 6-7 Define the Java Build Settings Dialog

7. On the *Define the Java build settings* dialog click **Add External JARs...**
8. Select and open the `symbolclasses.jar` file from the "`\Program Files\Symbol Mobility Developer Kit for Java\X.X\bin`" directory.
9. Expand the `symbolclasses.jar` file tree view and browse to the source code and JavaDocs. Attaching source code is recommended so that the class libraries can be stepped into. Attaching JavaDocs is an excellent way to view help for the Symbol Classes within WSDD. The source code is located at "`\Program Files\Symbol Mobility Developer Kit for Java\X.X\src`" and the JavaDocs are located at "`\Program Files\Symbol Mobility Developer Kit for Java\X.X\Documents`."



NOTE This dialog can also be invoked for existing projects by right clicking on the project in Package Explorer and selecting Properties- Java Build Path.

10. Click **Finish**.
11. Configure a build.

Configuring a Build

1. From the *Project* drop-down menu, select *Device Developer Builds*.

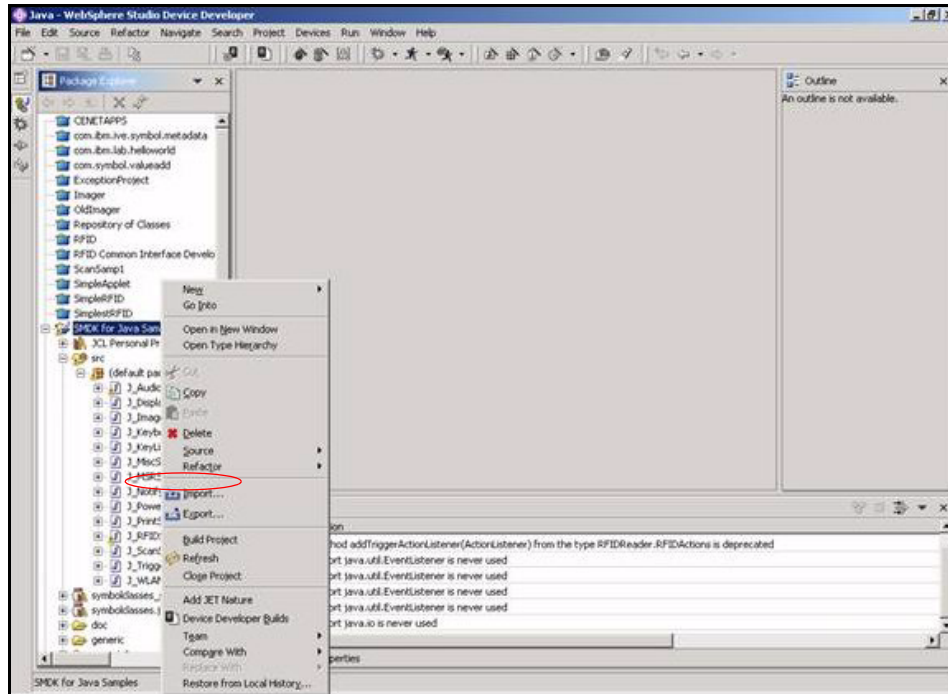


Figure 6-8 Device Developer Builds

2. The *Configure builds* dialog appears.

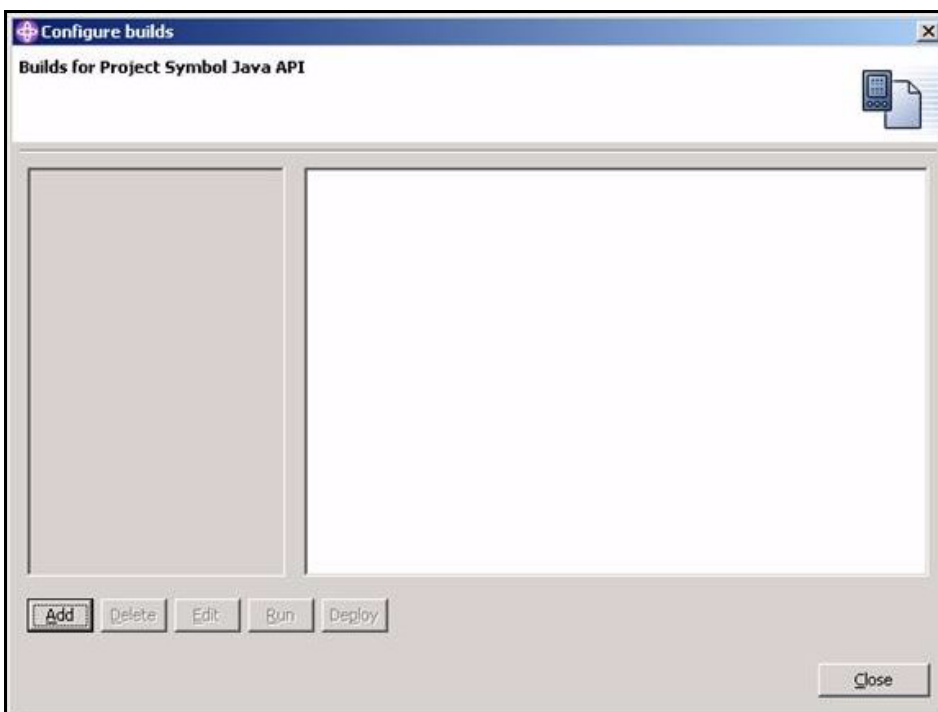


Figure 6-9 Configure Builds Dialog

3. From *Configure builds* dialog, click **Add** to add a new build. The *Build List* dialog appears.

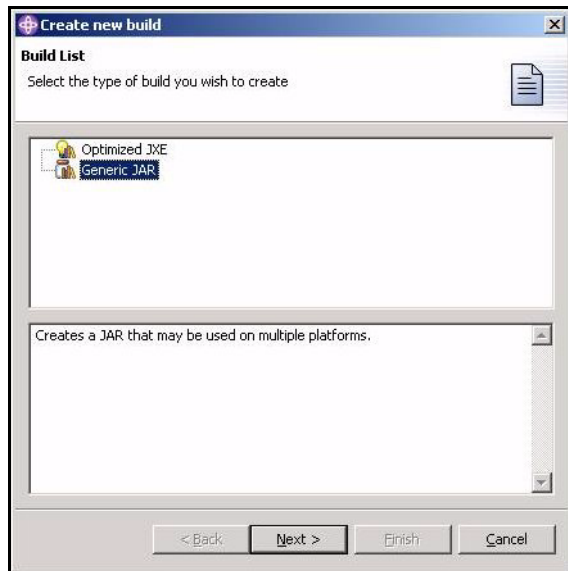


Figure 6-10 *Build List Dialog*

4. Select the desired Build List option (Optimized JXE or Generic JAR).
5. Click **Next**. The *General build settings* dialog appears.



Figure 6-11 *General Build Settings Dialog*

6. From the *General build settings* dialog, ensure that *Remove unused classes* is not selected (checked).



CAUTION If this option is selected, it may break notification methods that are not explicitly called by managed Java classes (those methods may be removed by the smart linker).

- Click **Next**. The *Contents* dialog appears.

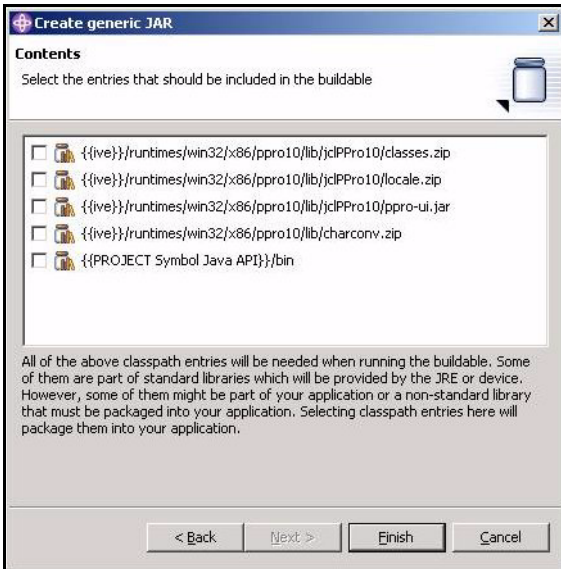


Figure 6-12 *Contents Dialog*

- From the *Contents* dialog, do not add any class libraries to be included in the build (keep all class libraries unchecked). These classes should be located on the device in jar and/or zip file format. This reduces the size of application file by a large margin and reduces any chance of the smart linker incorrectly removing methods that are located inside of these class libraries.
- Click **Finish**.
- Add a Symbol Pocket PC as a target device.

Adding a SymbolPocketPC Device

- From the *Devices* pull down menu, select *Configure...*. The *Create and manage device configurations* dialog appears.

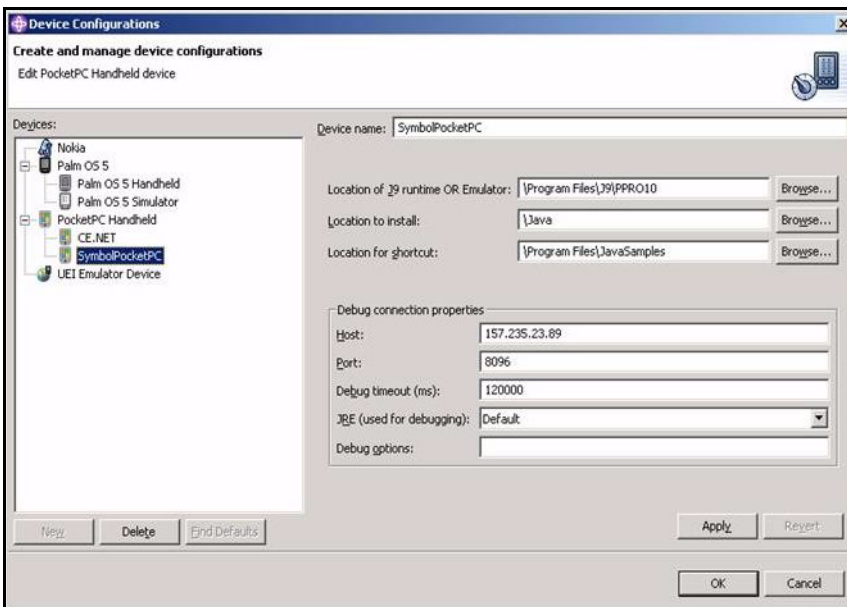


Figure 6-13 *Create and Manage Device Configurations Dialog*

2. From *Create and manage device configurations* dialog - *Devices*: tree, select *PocketPC Handheld*.
3. Click **New** to add a new device.

✓ **NOTE** Although all options on this dialog are important for proper remote debugging and run session to work, all text boxes except *Device name*: and *Locations* are filled in correctly by default. There are instances where it is desired to customize the location to install and/or the location for shortcut.

4. Enter *SymbolPocketPC* in the *Device Name*: text box.
5. Set the locations to point to the root folder where the WSDD J9-JVM is located in the device.

✓ **NOTE** The correct J9-JVM for WSDD must be present on the device. WSDD 5.7 provides a version of the J9-JVM for Pocket PC Handhelds. It is this JVM that must be present on the device. Other versions of the JVM may not be compatible.

6. Click **OK** when complete.
7. Add a new launch configuration.

Creating a Launch Configuration

1. Access the *Run* pull down menu and select either the *Run...* or *Debug...* option. The *Create, manage, and run configurations* dialog appears.

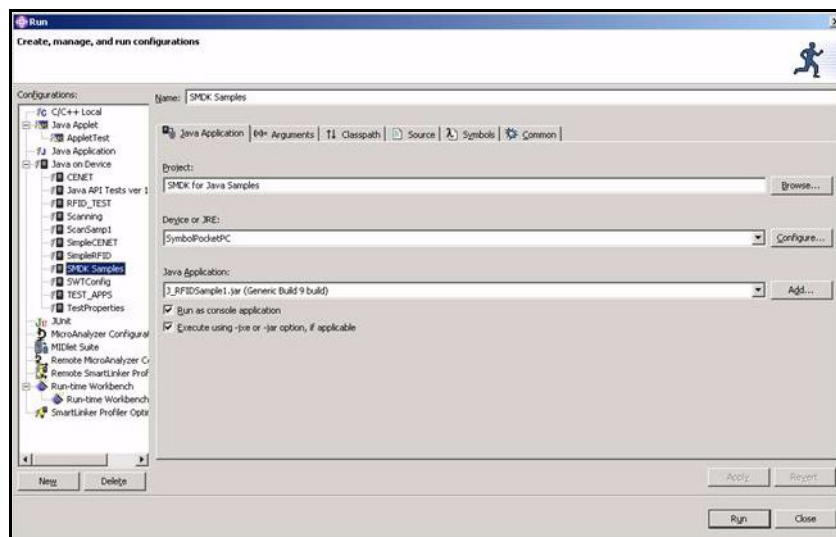


Figure 6-14 *Create, Manage, and Run Configurations Dialog*

2. From the *Create, manage, and run configurations* dialog, create a new “Java on Device” launch configuration. Select the project to run, the target device (SymbolPocketPC), and the Java application to run or debug.
3. **IMPORTANT:** On the *Arguments* tab, add the following lines to the VM arguments.
 - a. For WSDD 5.7.1 (PocketPC Devices):


```
"-jcl:ppro10" "-classpath" "\Java\symbolclasses.jar;\Java\symbolsamples.jar"
```
 - b. For WSDD 5.7.1 (Windows CE Devices):


```
"-jcl:foun"
"-Xbootclasspath/a:\Java\lib\jclPPro\ppro-ui-win.zip;\Java\symbolclasses.jar;\Java\symbolsamples.jar"
```



NOTE The arguments above are for running the samples only. The <Application Name>.jar file and path may have to be added to the argument string. For example, a typical argument line may resemble the following:

```
$ "-jcl:ppro10" "-classpath" "\\Java\\symbolclasses.jar;\\Java\\J_RFIDSample1.jar"
  "J_RFIDSample1"
```

JavaPOS Support

JavaPOS (JPOS) is an operating system independent Point of Service (POS) peripheral device standard for systems running Java applications. JavaPOS defines a set of device classes that conform to specific well defined interfaces to allow generic control from any JavaPOS compliant application.

Supported Services

JPOS drivers for the following enterprise mobility devices are implemented:

- Scanner
- Magnetic Stripe Reader (MSR)

- iii. *Low level Device Specific Driver*: Directly communicates with the device in order to control its operation.
- Service Discovery Infrastructure: Provides a class factory infrastructure whereby instances of the service classes can be instantiated through a lookup process in the System JPOS Database.

Chapter 7 Deploying Applications

Introduction

This chapter provides information for an Application Developer who may also play the role of Integrator. The chapter discusses the various methods available to package and deploy the applications written for Motorola enterprise mobility devices. While the described methods apply to most Motorola enterprise mobility devices, consult the documentation for an individual deployment method to see if your device is supported.

Deployment Methods

The deployment methods available for Motorola enterprise mobility devices are:

- ActiveSync
- Removable Media
- TCM (Terminal Configuration Manager)
- AirBEAM Smart
- MSP
- Third-Party Device Management Tools.

All of these deployment methods are generally independent of the programming language used to develop an application.

ActiveSync

Microsoft ActiveSync™ allows the establishment of a connection between a PC and a mobile device via a cable, cradle, or other "serial" link. Once an ActiveSync connection is established, data can be synchronized between the mobile device and the PC and files can be copied between the file systems of the mobile device and the PC using a "drag-and-drop" user interface.

ActiveSync performs file conversions when necessary. For example, a Microsoft Word document, copied from the PC to a device, is converted to a Pocket Word document. Some content may be lost after the conversion.

To install ActiveSync on a PC, download the software from the Microsoft web site at: <http://www.microsoft.com>. Microsoft recommends installing ActiveSync on the PC before connecting the device. Note that not all versions of ActiveSync support all versions of Windows CE and/or Windows Mobile. Consult the Microsoft web site for information on the specific version of ActiveSync required to support the OS on a given target mobile device. Note also that only one version of ActiveSync can be present on any one PC, hence it may NOT be possible to support all device OS versions on a single PC at once.

Using ActiveSync as a method of deployment has the following advantages:

- There is no additional per-device license fee for using ActiveSync.
- All Windows CE and Windows Mobile devices generally support ActiveSync.
- There is no Server or Motorola-supplied software required to use ActiveSync.

Using ActiveSync as a method of deployment has the following disadvantages:

- Motorola may not be able to provide direct support or corrective action if ActiveSync fails to operate as desired - this may need to be obtained from Microsoft.
- File transfer requires human interaction for each mobile device to manually drag and drop the desired files, which may be tedious, time-consuming, and error-prone.
- ActiveSync may experience difficulties when transferring large quantities of data and/or when connecting and disconnecting devices frequently.
- Only one mobile device can be connected to any one PC at a time using ActiveSync.

Removable Media

If a mobile device has a Removable Media slot, then it may be possible to deploy user applications to the mobile device via Removable Media. The following steps would generally be followed to accomplish this:

1. Equip the PC with a reader/writer that allows it to read and write the required type of Removable Media.
2. Insert the Removable Media into the PC reader/writer.
3. Use Windows Explorer to copy the desired application files (typically one or more CAB files) to the Removable Media.
4. Remove the Media from the PC and insert it into the Media slot of the mobile device.
5. Use the File Explorer on the device to locate the desired application files on the Removable Media and execute them and/or copy them to a desired target directory on the mobile device.
6. Alternately, create and place an "auto-run" file onto the Removable Media to perform the above execute or copy automatically when the Removable Media is inserted into the mobile device.
7. Remove the Media from mobile device and use it to deploy additional mobile devices.

Using Removable Media as a method of deployment has the following advantages:

- There is no additional per-device license fee for using Removable Media.
- Removable Media can usually be used to deploy very large files and/or many files with reasonable performance.
- There is no Server or Motorola-supplied software required to use Removable Media.

Using Removable Media as a method of deployment has the following disadvantages:

- A reader capable of reading and writing the appropriate type(s) of Removable Media must be available on the PC. Note that a mobile device connected via ActiveSync COULD be used to provide such a read and write capability.
- One or more Removable Media of suitable capacity must be acquired and “asset managed”.
- Depending on the device, inserting and removing Removable Media into mobile devices during deployment may be a tedious and/or time consuming process.
- If manual execution or copying is performed, the process may be tedious, time-consuming, and error-prone.

TCM

The Terminal Configuration Manager (TCM) is a PC utility supplied as part of the Device Configuration Packages (DCPs) for many Motorola mobile devices that can be used to build customized flash partitions (\Application and \Platform). The most common use is to create a custom \Application partition hex file that contains one or more applications. In addition to building hex files, TCM can also be used to transfer hex files to a mobile device so the device-resident Initial Program Loader (IPL) can receive it and burn it into the flash memory of the mobile device. TCM is generally supported on mobile devices running Windows CE and mobile devices running Windows Mobile versions prior to Windows Mobile 5.0.

TCM provides a drag-and-drop user interface for creating scripts that describe the entire folder structure and file content to be built into a flash partition. TCM presents a pair of directory windows to the user, one displaying the current “content” described by the script and the other displaying the files resident on the PC. A script can be thought of as a list of commands specifying the files and folders to be created within the partition, the files to be stored into those folders within the partition and the source locations on the PC from which they are obtained.

For mobile devices that support TCM and IPL, the DCP for the mobile device includes the TCM executable and a set of standard scripts used by Motorola to build the standard factory partitions shipped on the mobile device. The standard scripts can be used as a starting point, with files and/or folders added or deleted as required.

Using TCM as a method of deployment has the following advantages:

- Motorola can provide direct support or corrective action if TCM fails to operate as desired.
- There is no additional per-device license fee for using TCM.
- Once a partition hex file is created, it can be deployed to multiples devices reliably and with little chance of operator error.
- In SOME cases, it MAY be possible to “chain” cradles and transmit the same partition to multiple mobile devices at once, this increasing the speed of deployment.
- There is no Server required to use TCM.

Using TCM as a method of deployment has the following disadvantages:

- Building a partition can be a challenging and time consuming process.
- Transmitting a partition hex file to a device can be a VERY slow operation, depending on the size of the content being deployed.
- It is not possible to update part of a partition – even a SMALL content change requires the entire partition to be rebuilt and retransmitted to ever device.

AirBEAM Smart

AirBEAM Smart is a Motorola product that enables the deployment of collections of files called Packages to mobile devices via any IP connection. AirBEAM Smart is a deployment method whereby a device-resident client on the mobile device pulls files from a standard FTP Server. Many Motorola mobile devices ship with the AirBEAM Smart Client pre-installed.

AirBEAM Smart is a licensed product, so even though the AirBEAM Smart Client is pre-installed on many Motorola mobile devices, AirBEAM Smart licenses must be purchased for each mobile device before AirBEAM Smart can be used to deploy applications to those devices. In the AirBEAM Smart licensing model, either the client or the Package must be licensed before the client is permitted to deploy Packages. A licensed client can deploy either licensed or unlicensed Packages. Licensed Packages can be deployed using either a licensed or unlicensed client. But unlicensed Packages CANNOT be deployed using an unlicensed client. Whether clients or Packages are licensed, a sufficient quantity of AirBEAM Smart licenses must be purchased to cover the number of mobile devices that are used to deploy applications using AirBEAM Smart.

AirBEAM Smart requires that an IP connection be established on each device and that AirBEAM Smart be configured on each device to know how to reach a specific FTP Server. When using AirBEAM Smart, these configurations are typically performed manually, on the device, before initial deployment, via the user interface of the AirBEAM Smart Client. Once configured, these settings can be reconfigured through the use of AirBEAM Smart Packages.

Deployment in AirBEAM Smart works based on the concept of two lists of Packages: the DESIRED list and the PRESENT list. Before any deployment can occur, AirBEAM Smart must be configured on each device to set the list of DESIRED Packages that should be present on the device. AirBEAM can then be configured to “check in” with the FTP Server periodically, on reboot, on request by the device user, or programmatically by a command line executed by an application program on the device.

When the AirBEAM Smart Client checks-in, it examines the FTP Server to identify all Packages on the Server that are also contained in the DESIRED list. If the device does not have a Package or has a DIFFERENT version of the Package than that found on the FTP Server, then the AirBEAM Smart Client pulls the Package from the FTP Server and deploy it on the device. If a different version of the Package was on the device already, it is uninstalled before the desired package is installed. When a Package is installed, it is added to the PRESENT list, along with its version.

Using AirBEAM Smart as a method of deployment has the following advantages:

- Motorola can provide direct support or corrective action if AirBEAM Smart fails to operate as desired.
- AirBEAM Smart enables rapid and reliable deployment to devices over any IP connection.
- AirBEAM Smart enables controlled and repeatable deployment to mobile devices using AirBEAM Smart Packages.
- Once configured, AirBEAM Smart can enable remote update of devices without the need for manual intervention by a device user.
- There is no Motorola-supplied Server required to use AirBEAM Smart.
- The AirBEAM Smart Client ships pre-installed on many Motorola devices, thus simplifying the task of initial software deployment.
- AirBEAM Smart is designed for and integrates well with the unique persistent storage mechanisms of Motorola mobile devices.

Using AirBEAM Smart as a method of deployment has the following disadvantages:

- Before first use, manual configuration of the AirBEAM Smart Client is required to be performed on the device.
- One or more FTP Servers must be deployed within the enterprise, depending on the nature of connectivity, bandwidth, etc. These Servers are customer-supplied and any FTP Server can be used that complies with RFC 959 File Transfer Protocol (FTP), published in 1985.
- There is a per-device license fee for using AirBEAM Smart to deploy custom Packages to Motorola mobile devices.
- AirBEAM Smart generally supports only Motorola mobile devices.

A PC utility called the AirBEAM Smart Package Builder is provided to enable the creation of Packages for deployment using AirBEAM Smart. In general, an AirBEAM Smart Package is a set of files to be placed into various locations in a device and commands to be executed after installation and/or before uninstallation. Each Package is assigned a version number. The AirBEAM Smart Client skips deployment when the version of a Package on the FTP Server is IDENTICAL TO the version of the Package present on a device. The AirBEAM Client deploys a Package when the version of the Package is DIFFERENT FROM the version present on a device.

AirBEAM Smart Packages are most commonly used to deliver the files required to install applications, to configure applications for proper use, and to launch applications. AirBEAM Smart Packages are also used to deliver files related to device and driver patches.

A special variant of an AirBEAM Smart Package, called an AirBEAM Smart OS Update Package can also be built using a special utility called the AirBEAM Smart OS Update Package Builder. OS Update Packages can deliver full OS updates and can deliver one or more individual partition hex files as described earlier in the TCM deployment method.

Note that no matter how AirBEAM Smart Packages are built; the licensing requirements described earlier still exist. In some cases, Motorola provides pre-built and pre-licensed AirBEAM Smart OS Update Packages to enable customers to update the OS in Motorola devices without requiring an AirBEAM Smart license for every device. Once such an OS Update is complete, however, the AirBEAM Smart Client on the device remains unlicensed and no application deployment using AirBEAM Smart is permitted without purchasing the appropriate number of licenses.

Once AirBEAM Smart Packages are available, they can be installed to one or more FTP servers using the AirBEAM Smart Package Builder. Once on an FTP Server, Packages can be deployed by any mobile device that is configured to access that FTP Server and for which suitable licensing has been arranged.

The AirBEAM Smart Package Builder and documentation can be downloaded from the Motorola Support Central Web Site: <http://support.symbol.com>.

The AirBEAM Smart OS Update Package Builder and documentation can be downloaded from the Motorola Beta Zone Web Site:
http://devzone.symbol.com/login.cfm?return=/content.cfm?item_id=5798ADF0-ACB6-11D5-846400B0D04914DF.

AirBEAM Smart licenses can be purchased in whatever quantities desired via the standard Motorola product ordering process.

MSP

The Mobility Services Platform (MSP) is a Motorola product that enables the management of many Motorola mobile devices throughout an enterprise from various locations within the enterprise. In addition to at least one MSP Server, MSP requires that at least one FTP Server be available. This is because MSP leverages and extends the functionality of AirBEAM Smart. MSP uses AirBEAM Smart Packages and the MSP client software expands on the core capabilities of the AirBEAM Smart Client. Consequently, FTP Servers remain central to the Package deployment capabilities of MSP.

Many Motorola mobile devices now ship with the MSP client software pre-installed. Since the MSP client software is 100% backward compatible to the AirBEAM Smart Client, this does not interfere with customers continuing to use AirBEAM Smart.

Like AirBEAM Smart, MSP is a licensed product, so even though the MSP client software is pre-installed on many Motorola mobile devices, licenses must be purchased for each mobile device if MSP is used to deploy applications to those devices. In the MSP licensing model, devices are licensed for use by MSP by installing supplied license keys into the MSP Server via the MSP Console UI. MSP keeps a record of the number of licenses installed and in use. This makes it easy to know when to order and install more licenses.

MSP provides 2 key methods of deployment to address the life-cycle of Motorola mobile devices: Staging (sometimes called Rapid Deployment) and Provisioning. Both methods involve the deployment of AirBEAM Smart Packages by mobile devices from FTP Servers. The primary distinction is how the deployment is initiated and controlled.

Staging is most commonly defined as the process of preparing a set of “fresh out of the box” devices for use in a production environment. The term Rapid Deployment was coined to describe the fact that manually performing such actions is time-consuming, costly, and error-prone. By automating this process, the time and cost to prepare devices for first use can be significantly reduced and the reliability and repeatability can be significantly improved.

Provisioning is most commonly defined as the process of keeping a set of “in service” devices up-to-date by deploying updates as needed. By automating this process and allowing it to be controlled remotely, the need for manual intervention to perform updates can be entirely eliminated.

In MSP, both Staging and Provisioning are accomplished through the use of AirBEAM Smart Packages, but unlike the AirBEAM Smart Client, the MSP client software automates the underlying deployment actions. The method of automation varies between Staging and Provisioning.

In the case of Staging, deployment actions are initiated by a mobile device user launching the Rapid Deployment Client and selecting a deployment mode. The Rapid Deployment Client requires a Staging Profile that describes how to establish an IP connection, how to contact an FTP Server, and what Packages to deploy. Unlike the AirBEAM Smart Client, the Rapid Deployment Client requires little knowledge of how to set up the device. All the relevant information is determined by the Staging Profile.

In the case of Provisioning, deployment actions are initiated by a Provisioning Policy created and applied to one or more mobile devices from the MSP Console UI. It is generally assumed that Provisioning happens AFTER a device has been initially Staged and hence the device has already been configured to have an IP connection and periodically check in with an FTP Server. Unlike the AirBEAM Client, the MSP Agent has the ability to receive “Jobs” from MSP via the FTP Server that can trigger the deployment of any Packages without the need to pre-configure a DESIRED Package list. This means that once a device is properly configured, any Packages can be “logically pushed” to any or all devices when required, with no need for the intervention of the device user.

Using MSP as a method of deployment has the following advantages:

- Motorola can provide direct support or corrective action if MSP fails to operate as desired.
- MSP enables rapid and reliable deployment to devices over any IP connection.
- MSP enables controlled and repeatable deployment to devices to prepare them for their first use in a production environment with little device user interaction.
- MSP enables remote initiation of the controlled and repeatable deployment of updates or changes to mobile devices while they are in service with no need for device user intervention.
- The MSP client software ships pre-installed on many Motorola devices, thus simplifying the task of initial software deployment. For those where it does not, the AirBEAM Smart Client can be used to deploy the MSP client software.

- MSP is designed for and integrates well with the unique persistent storage mechanisms of Motorola mobile devices.
- MSP has deep knowledge of Motorola mobile devices and is designed to optimize the deployment experience for these devices.

Using MSP as a method of deployment has the following disadvantages:

- MSP requires Motorola-supplied software to be purchased and installed on a customer-supplied Windows Server.
- One or more FTP Servers must be deployed within the enterprise, depending on the nature of connectivity, bandwidth, etc.
- There is a per-device license fee for using MSP to manage Motorola mobile devices.
- MSP generally supports only Motorola mobile devices.

MSP can support any AirBEAM Smart Package produced for use with AirBEAM Smart. MSP also comes with an enhanced version of the Package Builder to allow Packages to be built to leverage the enhanced deployment capabilities of MSP.

Third-Party Device Management Tools

There are third-party tools available that offer capabilities to deploy applications on Motorola mobile devices. .

Using a Third-Party Device Management Tool as a method of deployment has the following advantages:

- A given Third-Party Device Management Tool MIGHT support a mixture of mobile devices, including those from Motorola as well as devices from other vendors, or even laptops or PCs.
- A given Third-Party Device Management Tool MIGHT provide enhanced functionality in selected focus areas compared to a solution offered by Motorola.

Using a Third-Party Device Management Tools as a method of deployment has the following disadvantages:

- Motorola cannot provide direct support or corrective action if a Third-Party Device Management Tool fails to operate as desired – this must be obtained from the third party.
- Third-Party Device Management Tools must be purchased separately from the Third-Party, including any requisite licensing and/or support.
- Third-Party Device Management Tools are most likely never be shipped pre-installed on Motorola mobile devices. Initial deployment may thus be complicated SINCE another deployment method may be needed to install the Third-Party agent before further deployment can be accomplished using the Third-Party Device Management Tool.
- In most cases, a Third-Party Device Management Tool do not provide the same breadth and depth of coverage for Motorola mobile devices. Such tools typically are more generic, and focus on “common denominator” features in order to cover a wider selection of devices.

Application Deployment for Mobile 5.0

Mobile 5.0 Deployment

This section lists the procedures for deploying applications onto a device with Windows Mobile 5.0.

Deployment

To install applications onto the device, developers package the application and all required files into a CAB file, then load the file onto the device using one of the following options:

- Microsoft ActiveSync 4.1 or greater
- Storage Card
- AirBEAM
- Image Update (for updating the operating system).
- Refer to the *Microsoft Windows Mobile 5.0 Help* file for information on CAB files.

Image Update

Windows Mobile 5.0 contains an Image Update feature that updates all operating system components. All updates are distributed as update packages. Update packages can contain either partial or complete updates for the operating system. The update packages are distributed on the Support Site, <http://support.symbol.com>.

To update an operating system component, copy the update package to the device using one of a variety of transports, including ActiveSync, an SD memory card, or Symbol AirBEAM.

Refer to the device's Integrator Guide for detailed information on installing image updates.

XML Provisioning

To configure the settings on a device XML provisioning should be used. To install an XML provisioning file on the device, create a Cabinet Provisioning File (CPF) file. A CPF file is similar to a CAB file and contains just one file: `_setup.xml`. Like a CAB file, the CPF extension is associated with `WCELoad.EXE`. Opening a CPF extracts the XML code and uses it to provision and configure the device. The user receives an e-mail notification indicating success or failure.

XML Provisioning provides the ability to configure various features of the device (i.e., registry and file system). However, some settings require security privileges. To change registry settings via a CPF file, you need to have certain privileges (roles). Some registry keys require you to simply be an *Authenticated User*, while other registry keys require you to be a *Manager*. Refer to the Windows Mobile 5.0 Help file for the default role settings in Windows Mobile 5.0.

For those registry settings that require the *Manager* role, the CPF file must be signed with a privileged certificate installed on the device. Refer to the *Microsoft Windows Mobile 5.0 Help* file and the *Windows Mobile 5.0 SDK* for instructions and sample test certificates.

Chapter 8 Application Lock-down

Introduction

This chapter provides information for an Application Developer who wishes to prevent users from accessing unauthorized applications.

Preventing User Access to Programs

The Windows Mobile operating system allows some User Interface customization, but does not provide support for preventing users from accessing "unauthorized" programs, such as the Solitaire game. This is a common complaint when a Windows Mobile device is used in an enterprise environment.

Motorola provides two solutions that prevent users from accessing unauthorized applications, keeping them focused on productivity: AppCenter and Symbol Pocket Browser.

AppCenter

AppCenter is an application that prevents the user from running "unauthorized" programs. AppCenter presents the user with a screen of icons representing approved applications. Only the programs launched from this screen are allowed to run. Any "unauthorized" application that attempts to start up (either automatically or by user control) is immediately closed. AppCenter can also be used to disable the Start menu, the on-screen keyboard and **Smart-Minimize** button (X). Refer to the *AppCenter for PPC v1.10 Administration Guide* for more detail.

AppCenter v1.10 includes enhanced support for Terminal Services Client and better support for .NET Compact Framework and MFC-based applications. Version 1.10 also provides Full Screen Mode capability to compliant applications, an Administration Guide, performance enhancements, and minor bug fixes.

✓ **NOTE** AppCenter is designed to run on approved Motorola enterprise mobility devices only.

Symbol PocketBrowser

The Symbol PocketBrowser integrates the core components of Pocket IE or IE6 with Symbol's unique features such as bar-code scanning. The Symbol PocketBrowser blocks the end user from the operating system, exposes the full real estate of the screen to the Web application designer and harnesses Symbol's unique features, so you can add more value to the device, while preserving the ease-of-use of the Web services programming model.



NOTE Symbol PocketBrowser requires a license for use. Although it can be used on a trial basis, free of charge, the recurring “nag” screen prevents the unlicensed version of PocketBrowser from being used in a production environment.

Microsoft SHFullScreen

The Microsoft SHFullScreen function can be used to take over certain areas of the screen. It is used to modify the taskbar, Input Panel button, or *Start* menu icon.

For more information about this function, go to the Microsoft website:

http://msdn.microsoft.com/library/default.asp?url=/library/en-us/apippc/html/ppc_aygshell_fsiq.asp.



Chapter 9 Application Persistence

Introduction

Motorola enterprise mobility devices were designed using a Flash File System architecture that supports application persistence. To survive a hard reset (also known as a cold boot), user applications and data should be installed in the non-volatile, flash memory of the device. The information presented below, applies to Motorola enterprise mobility devices running Windows CE v4.2, Windows CE 5.0 and Windows Mobile 2003.

Flash File System

The Flash File System contains two user accessible folders, Platform and Application. The Platform folder is reserved for system use and contains many of the system files and software drivers. Modification of the Platform folder should be avoided. The Application folder is available for customer use and may contain all of the files required for end-user applications.

RegMerge and CopyFiles

RegMerge and CopyFiles are system components that allow files to persist across a cold boot. These components run as a part of the hard reset process and copy files and registry settings from the flash folders to the volatile memory of the device. The volatile memory (or RAM) folders include: the root directory, Windows, My Documents and Program Files.

RegMerge and CopyFiles are Built-in drivers that are launched early in the boot sequence by the Windows CE Device.exe process.



NOTE With Windows Mobile 5.0, Motorola recommends using XML provisioning instead of RegMerge and CopyFiles. RegMerge and CopyFiles are supported for backward compatibility but Motorola may eliminate support in the future.

RegMerge

RegMerge is a built-in driver that allows entries found in .REG files to be stored in the RAM-based, Windows CE registry. RegMerge runs during a cold boot and looks for .REG files, first in the root of the Platform folder and then in the root of the Application folder. It merges the registry changes found in these files into the system registry. For example, what follows is an excerpt from one of the REG files found in the Application folder of an MC9000 device:

```
$ [HKEY_CURRENT_USER\Software\Symbol\Launcher\symsetup\Settings]
"DetailColumns"=dword:00000001
"ButtonColumns"=dword:00000002
"TitleBar"="Test Applications"
"Number"=dword:00000000
"Mode"="Large Icons"
"UseProgramIcons"=dword:00000001
"Return"="Back"
```

Besides creating keys and values in the registry, RegMerge is also capable of deleting them. To remove an existing registry key, precede its name with a minus character ('-'). For example:

```
$ [-HKEY_LOCAL_MACHINE\Software\Symbol\Appl]
```

To remove an existing registry value, assign a minus character ('-') to it. For example:

```
$ [HKEY_LOCAL_MACHINE\Software\Symbol\Appl]
"Option1"=-
```

CopyFiles

CopyFiles is a built-in driver that is used to copy files from non-volatile memory, such as the Platform and Application folders, to RAM-based folders such as the Windows and Program Files folders. During a cold boot, CopyFiles looks for files with a .CPY extension in the root of the Platform and Application folders. Much like the RegMerge process, this process looks for CPY files, first in the root of the Platform folder and then in the root of the Application folder. These text files contain a list of copy commands that specify a source and destination separated by a ">" character. For example:

```
\Application\ScanSamp2.exe > \Windows\ScanSamp2.exe
```

This line directs CopyFiles to copy the ScanSamp2.exe application from the \Application folder to the \Windows folder. If the destination folder does not exist, then CopyFiles creates it.

In addition to copying the program file, CopyFiles can also be used to place an application shortcut in the Start Menu. The following example illustrates this:

```
\application\MC50 Demo.Ink > \%WSM%\MC50 Demo.Ink
```

The "%WSM%" is a string substitution variable that is replaced at run time with the string "Windows\Start Menu" on English systems or the translation of this string on localized systems. The string substitution variables supported by CopyFiles are shown in [Table 9-1](#).

Table 9-1 String Substitution Variables

Variable	English String	Windows Mobile	CE .NET
%WSU%	Windows\Startup	X	X
%WSM%	Windows\Start Menu	X	
%WSMP%	Windows\Start Menu\Programs	X	
%WSMS%	Windows\Start Menu\Settings	X	
%WP%	Windows\Programs		X
%WDT%	Windows\Desktop		X

The Device Configuration Package (DCP) for each device contains examples of .REG and .CPY files. DCPs can be downloaded from the Symbol Support Central at <http://support.symbol.com/>.



NOTE Because the CopyFiles process runs early in the boot sequence, CPY files can not be processed from Compact Flash (CF) or Secure Digital (SD) memory.

Making the Application Persist

To make an application persist across a hard reset, a REG file must be created to set the program options in the registry and a CPY file must be created to copy all of the program and data files from the Application folder to RAM folders. Multiple REG and CPY files can be used for a single application. To test the installation, all of the program files, including the REG files and CPY files, must be transferred to the Application folder of the device using ActiveSync, and a hard reset performed on the device. The device is now ready to run the newly installed application.

Making Application Data Persist

Since the data collected by many applications is critical to the operation of the business, care should be taken to ensure the data does not get corrupted. One important way to avoid possible data loss is by storing the data collected by an application into non-volatile memory. By using non-volatile, or flash memory, the data persists even when power is removed from the device for extended periods of time or there is a power related failure. Storing application data into non-volatile memory is easily achieved by ensuring the data files created by the application reside on the \Application folder.

The \Application folder is accessed using a FAT file system. Because of the inherent vulnerabilities of the FAT file system, the amount of time that files are left open should be minimized. Programs should close or flush data files after each update. An alternate approach would be to keep data files in the RAM-based file system which is transaction based, and periodically copy the files to the flash file system.



NOTE The "\Platform" folder is reserved for system use. Application data files should not be stored under the "\Platform" folder.

Packaging and Deployment

Application files can be packaged into a HEX file using Terminal Configuration Manager (TCM), which is supplied with the device's DCP. Once packaged, this HEX file can be loaded onto the device using TCM.

An alternate method for application installation employs Windows CE CAB files. The application files and registry settings can be packaged as a CAB file and placed in the Application folder. Using a shortcut file placed in the "Application\Startup" folder and a CPY file placed in the Application folder, this CAB file can be automatically copied to the Windows folder and launched on each cold boot. Use the Microsoft CABWiz utility, or one of the other commercially available CAB file managers, to create the CAB package.

For mass deployments of packaged applications, Symbol AirBeam or Symbol Mobility Services Platform (MSP) should be considered.

Running Applications from RAM

As discussed earlier, Motorola recommends that applications be stored on non-volatile media, such as Flash File System or Compact Flash. This increases the integrity of the system by ensuring that applications are never lost due to a critical power failure. However, Motorola does not recommend running applications directly from non-volatile media due to delays imposed by Windows CE in accessing files following a resume from suspend. In particular, applications that are launched immediately following resume should not be run directly from non-volatile media.

Automatic Startup of Applications

Most mobile enterprise applications require a single program to take over the display and hide the distractions of the remainder of the device. To achieve this focus on productivity, the customer application needs to automatically launch when the device is rebooted. All Motorola enterprise mobility devices provide for the automatic launch of programs at boot time. This includes both a hard reset (cold boot) and a soft reset (warm boot). The methods are presented in the order in which they are processed by the system.

OS Launch Keys

The first method of auto-launch is achieved through the use of built-in OS launch keys, which are processed by the Windows CE operating system. These keys define the system processes to load and the order in which they are to be launched. The processes are defined in the registry at the following location:

```
$ HKEY_LOCAL_MACHINE\Init
```

The following list includes examples of OS launch keys:

- Launch20 Device.exe (starts RegMerge.dll and CopyFiles.dll)
- Launch30 GWES.exe (starts touch and display drivers)
- Launch50 Explorer.exe (launches programs in Windows startup folder)
- Launch60 Services.exe
- Launch70 Startup.exe (Symbol Startup process)



NOTE The OS launch keys are for system use only and must not be modified.

Windows Startup Folder

One of the OS launch keys defined above invokes Microsoft Windows Explorer. Part of the Explorer.exe startup process, is the launching of files located in the \Windows\Startup folder.

- ✓ **NOTE** The order of execution is not guaranteed for items launched from the \Windows\Startup folder. Programs are launched in the order that they are found in the file system.

Programs launched from this folder may run simultaneous with the Windows Welcome.exe process, sometimes referred to as the "Dentist Appointment" screen. To insure that your programs launch after the Welcome.exe process is complete, use the \Application\Startup folder described below.

- ✓ **NOTE** Use the Symbol \Application\Startup folder for launching applications that must start after the Welcome.exe process completes.

Symbol Startup Program Keys

The second method of auto-launch is achieved through the use of the Symbol Startup Program keys. One of the OS launch keys defined above (Launch70) invokes a Symbol created process called Startup.exe. The Symbol Startup process controls the launching of predefined programs when a device is re-booted. The Startup registry keys can be used to launch applications as well as install add-on components such as a printer driver package or the Microsoft .NET Compact Framework. The processes launched and the order in which they are launched are defined in the registry at the following location:

```
$ HKEY_CURRENT_USER\Software\Symbol\Startup\Programs
```

The following list includes examples of Symbol Startup Program Keys:

- Prog3 Microsoft .NET Compact Framework
- Prog4 Symbol .NET Class Libraries
- Prog5 Symbol AirBeam Abboot.exe
- Prog7 Symbol AirBeam Abstart.exe
- Prog10 Symbol Printer Drivers.

For each ProgN key, the following subkeys can be set:

- Name Application file name with full path spec (string)
- Command Application command line parameters (string)
- Continue Block or not block, 1 = do not wait for program to finish (dword)
- ColdBootOnly Run on cold boot, 1 = run on cold only (dword).

A total of 20 Startup Program Keys can be defined.

- ✓ **NOTE** Many of the Symbol Startup Program Keys are reserved for system use and caution should be used to avoid conflicts. Although these keys are not documented, customers have used them to install and launch applications. Before using a Symbol Startup Program Key, check the registry to see which of the 20 keys are defined.

Symbol Startup Folder

In addition to processing the "ProgN" registry keys defined above, the Symbol Startup process looks in a special folder for applications to launch. The Symbol Startup folder defines a set of applications to launch as well as packages to install. This is the preferred method for installing and launching customer applications. The items to launch are found in the following folder of the flash file system:

\Application\Startup

This location is user-definable in the registry as follows:

```
$ HKEY_CURRENT_USER\Software\Symbol\Startup\Path
```

The default value for this key is: "*\Application\Startup*"

The files placed in the Symbol Startup folder can be either executable (*.exe) or special Symbol defined run files (*.run). A ".RUN" is a text file with two lines. The first line is the complete name of the executable file (with full path information), and the second line is an optional command line argument to be passed to that executable file. For example, to launch OTL with the string "default" as the command line, create the OTL.run file with the following contents:

- \Application\Demo\Otl.exe
- default

Startup.exe also provides the ability on a Pocket PC device to delay the starting up of user-specified application until after the Welcome process has completed. The Welcome process is often referred to as the "Dentist Appointment Screen". Any application placed in the "\Application\Startup" folder does not launch until the Pocket PC Welcome process has completed.

✓ **NOTE** The order of execution is not guaranteed for items launched from the "\Application\Startup" folder. Programs are launched in the order that they are found in the file system.

Bypassing the Welcome Screen

A frequently requested feature from enterprise users of Pocket PC devices, is the ability to bypass the Welcome screens. The Welcome screens are a series of pages that are used to calibrate the touch screen, teach how to use a stylus and set the time zone. Beginning with the release of Windows Mobile 2003, the Welcome pages can be disabled using the following registry key:

```
$ [HKEY_LOCAL_MACHINE\Software\Microsoft\Welcome]
"Disable"=dword:FFFFFFFD
```

By setting "Disable" to 0xFFFFFFFF, all Welcome pages are disabled, except for Touch Screen Calibration. The value represents a bit mask, where each bit represents a configurable option. If the key does not exist, then all pages are enabled. The configurable bits are as follows:

- Disable Touch calibration 0x02
- Disable Stylus Help 0x04
- Disable Pop Up Menu Help 0x08
- Disable Time Zone Set 0x10
- Disable Complete Page 0x20

✓ **NOTE** Touch Calibration is critical to the proper operation of the device and must not be disabled.

Automatic CAB Installation

To use the startup folder to automatically install a CAB file, the CAB file needs to be placed in the \Application\Startup folder.

If some special processing is required, such as suppressing the UI prompts to the user, a RUN file needs to be created. This run file must launch the Microsoft utility called Wceload.exe with the CAB file name and the desired options on the command line.

The syntax for calling Wceload is as follows:

Wceload [Options] <CAB file>

where <CAB file> specifies the location of the cab file to install or remove

where [Options] specifies one of the following command line options

- /noaskdest -specifies that the user is not prompted for the installation directory.
- /askdest -specifies that the user is prompted for the installation directory.
- /delete 0 -does not remove the cab file.
- /delete 1 -removes the cab file. This is the default value.
- /delete 2 -treats the file as blocks of data, and dynamically deletes them.

The following options are new for Windows CE 5.0:

- /noui -user not prompted for input during the install. By default, prompts answered 'Yes'.
- /nouninstall-installed application cannot be removed. If option used, unload file not generated.

By default, an unload file is generated during installation with wceload.

The unload filename has the format <Software Provider Name> <Program Name>. This file is not generated if the nouninstall option is used.

Prior to Windows CE 5.0, a silent install could only be achieved using the headless version of "wceload", called Wceldcmd.exe.



NOTE Because Wceldcmd is not officially supported by Microsoft for devices that have displays, caution should be exercised when using it.

Windows Mobile 5.0

Microsoft Windows Mobile 5.0 introduces a new persistent storage model. Under this new model, the Windows CE Registry and the folders that were previously RAM-based, such as the Windows folder, are now located in non-volatile memory. Although the application deployment concepts presented in this paper is supported on Windows Mobile 5.0 devices for backward compatibility, customers should avoid using RegMerge and CopyFiles, and move toward a more standard CAB file deployment strategy.

✓ **NOTE** Support for RegMerge and CopyFiles is not guaranteed in future versions of Windows Mobile 5.0 devices from Motorola. Customers should begin using the more standard CAB file method of application deployment.

Although tools have been provided in the past, which allow customization of the Platform folder, most customers confine their changes to the Application folder. The Platform folder contains important system files, which if removed, could cause instability in the device.

✓ **NOTE** The first Windows Mobile 5.0 devices from Motorola does not contain a user accessible Platform folder, so customers should avoid using or even changing this folder.

XML Provisioning vs. RegMerge and CopyFiles

Prior to Windows Mobile 5.0, Symbol used two drivers (RegMerge and CopyFiles) to update the registry and to copy files during a cold boot. The following sections provide examples of how RegMerge and CopyFiles were used, and how to perform the same function using XML provisioning.

RegMerge

The following example uses RegMerge to set a registry key:

SampleReg.reg

```
$ [HKEY_LOCAL_MACHINE\Hardware\DeviceMap\Backlight]
$ "BacklightIntensity"=dword:00000036
```

The following example uses XML provisioning to perform the same task:

SampleReg.xml

```
$ <wap-provisioningdoc>
$ <characteristic type="Registry">
$ <characteristic type="HKLM\Hardware\DeviceMap\Backlight">
$ <parm name="BacklightIntensity" value="54" datatype="integer" />
$ </characteristic>
$ </characteristic>
$ </wap-provisioningdoc>
```

CopyFiles

The following example uses CopyFiles to copy a file from the \Application folder to the \Windows folder:

SampleCpy.cpy

```
$ \Application\example.txt > \Windows\example.txt
```

The following example uses XML provisioning to perform the same task:

```
$ SampleCpy.xml
$ <wap-provisioningdoc>
$ <characteristic type="FileOperation">
$ <characteristic type="\Windows" translation="filesystem">
$ <characteristic type="MakeDir"/>
$ <characteristic type="example.txt" translation="filesystem">
$ <characteristic type="Copy">
$ <parm name="Source" value="\Application\example.txt" translation="filesystem"/>
$ </characteristic>
$ </characteristic>
$ </characteristic>
$ </characteristic>
$ </wap-provisioningdoc>
```


Chapter 10 Advanced Programming

Introduction

This chapter provides a guideline for using the Symbol API functions in a useful application (e.g., WAN, Scanning, etc.) It also includes information about the proper use of 'Open/Close' statements, how to use Power Management effectively and snippets of code.

Advanced Programming

Learning the individual functions and how they operate is not enough to build a useful application. The functions must be used in the correct combination and order to get the most value from them. The following sections describe how to write useful applications for bar code data capture, image capture, Wireless Wide Area Network connectivity and mobile printing.

For additional information about the Symbol Windows CE C APIs, refer to the *SMDK Help* file for Motorola enterprise mobility devices.

Writing a Bar Code Data Capture Application



The Symbol Windows CE Scanner Driver C API provides a standard interface for developing scanning-aware applications. To develop applications capable of using the Scanner Driver, code is required to:

- Find a Scanner Device.
- Open a Scanner Device.
- Use Scanner Parameters.
- Start a Read Request.
- Trigger.
- Clean Up.

Finding a Scanner Device

The Symbol Windows CE Scanner Driver is capable of controlling more than one scanner device on the same device. Before an application can use a scanner, it must locate a scanner device. The *SCAN_FindFirst*, *SCAN_FindNext* and *SCAN_FindClose* functions provide the interface for enumerating all of the scanner devices attached to a device.

The *SCAN_FindFirst* function call begins the enumeration of scanner devices. The function returns the first device and a find handle for use in subsequent calls. Repeated calls to *SCAN_FindNext* return the next scanner device. When no more devices are available for enumeration, these functions return the *E_SCN_NOMOREITEMS* error.

The scanner device information is stored in the *SCAN_FINDINFO* structure. This structure contains the device name, a human-readable friendly name and the communication port used, if applicable. This information can be used to choose which scanner device to use. In most circumstances, a device only has one attached scanner device, named "SCN1:."

When a find session is complete, do not forget to call *SCAN_FindClose* to close the find handle and free resources used for the enumeration.

Opening a Scanner Device

In order to use the Scanner Driver to interface to a scanning device, an application must obtain a handle to that device. A scanner handle is obtained by a call to *SCAN_Open*. The *SCAN_Open* function takes the device name of a scanner device and returns a handle for subsequent function calls. The device name can be obtained using the *SCAN_FindFirst* device enumeration functions.

Once a handle to a scanner device is obtained, the version information should be verified. The *SCAN_GetVersionInfo* function takes a handle to a scanner device and returns the version information for the entire driver stack for that device.

The *SCAN_Enable* function initializes the Scanner Driver for use. Once the device is enabled, scanning parameters can be get and set, and read requests can be queued.

Using Scanner Parameters

The Scanner Driver is controlled by a large set of scanner parameters. These parameters include the enabled decoders, decoder specific parameters, and parameters governing the scanner hardware and interface.

Not all scanner devices support all of the scanner parameters. The *SCAN_GetDeviceInfo* function can be used to determine the functionality of the device being used. If the current scanner device does not support certain scanner parameters, the parameters are still validated even though they are ignored by the driver.

The most widely used parameters are the enabled decoders. These parameters control what types of bar codes the Scanner Driver decodes. The *SCAN_GetSupportedDecoders* function can be used to determine which decoders are supported on this scanner device. The enabled decoders can be controlled via the *SCAN_GetEnabledDecoders* and *SCAN_SetEnabledDecoders* functions.

Each decoder can have its own decoder specific parameters. These include the barcode lengths to be enabled and other parameters controlling things such as redundancy, check digits, and conversions of barcode types. These parameters are controlled via the *SCAN_GetDecoderParams* and *SCAN_SetDecoderParams* functions. The UPC and EAN barcode types have some common parameters that effect more than one decoder. Because of this, a separate category of UPC/EAN parameters exists and is controlled by the *SCAN_GetUPCEANParams* and *SCAN_SetUPCEANParams* functions.

The Scan Parameters are used to control the type of read being queued and the user feedback to be used when the read is satisfied. The scan type controls how a read request is viewed by the driver and effects the combination

of multiple reads. The feedback parameters can control the good decode LED and the beeper, and cause a WAV file to be played when a read is satisfied. These parameters are controlled by the *SCAN_GetScanParams* and *SCAN_SetScanParams* functions.

The reader and interface parameters control the scanner hardware. These parameters should generally not be modified by an application. These parameters can be accessed via the *SCAN_GetInterfaceParams*, *SCAN_SetInterfaceParams*, *SCAN_GetReaderParams*, and *SCAN_SetReaderParams* functions. Reader and interface parameters are not stored on a per-read basis as are the other parameters. Rather, they are passed directly to the driver and used as soon as they are accepted.

Starting a Read Request

When an application wants to scan a barcode and receive the data, it must submit a read request to the Scanner Driver. Read requests can be blocking or inform the application of completion via a message or an event. When a read request is submitted, it is queued with the current scanner parameters. The barcode data and all information regarding the read request are stored in a scan buffer.

To submit a read request, an application first must allocate a scan buffer. The scan buffer consists of a *SCAN_BUFFER* structure followed by a data buffer. The *SCAN_AllocateBuffer* function allocates such a structure. The function takes a flag describing whether the data is binary or text and the size of the data buffer in bytes (for binary) or characters (for text). It returns a pointer to a properly initialized scan buffer.

A read request can be submitted in three ways. The *SCAN_ReadLabelWait* function takes a scan buffer and a timeout and blocks until the read request is completed. The *SCAN_ReadLabelEvent* function returns immediately and signals an event (passed as a parameter) when the read request is completed. The *SCAN_ReadLabelMsg* function returns immediately and sends a message to a window when the read request is completed.

A read request is queued with the current scanner parameters. These include the enabled decoders, decoder-specific parameters, UPC/EAN parameters, and scan parameters. Any subsequent changes to these parameters does not effect the queued read. The reader and interface parameters are not stored on a per-read basis. These parameters are used immediately after they are accepted.

When a read request is completed, the *SCAN_BUFFER* holds information about the read. The status can be found in the *dwStatus* field. Other fields contain information such as the label type, the time stamp, and the direction in which the barcode was scanned. The barcode data can be found in the data buffer immediately following the *SCAN_BUFFER*.

Triggering

When a read request is submitted to the Scanner Driver, the scanner is controlled by triggers. Two types of triggers exist, soft and hard.

Hard triggers are physical buttons or switches on a device or scanner. In general, if a read request is present and the trigger is pressed, the scanner laser turns on. Some triggers are two-stage, containing two positions. When the trigger is pressed to the first stage, it is considered an aim trigger, and when it is pressed to the second stage, it is considered a scan trigger. The function of these two trigger inputs is governed by the aim type being used. This parameter is specified in *LASER_PARAMS.dwAimType* and applies to laser scanners and imager scanners capable of aiming.

Soft triggers are available for an application to simulate hard triggers. An application can control the soft trigger via the *SCAN_GetSoftTrigger* and *SCAN_SetSoftTrigger* functions. One soft trigger exists per open instance of the scanner driver. When a read request submitted on that open instance, or handle, is successfully completed, the soft trigger is cleared. The soft trigger is equivalent to a hard scan trigger.

Cleaning Up

The Symbol Scanner Driver C API contains functions to cancel submitted read requests, deallocate scan buffers, and shut down an open instance of the driver.

The *SCAN_CancelRead* function can be used to cancel a read submitted via *SCAN_ReadLabelEvent* or *SCAN_ReadLabelMsg*. The cancel function takes the scan request ID returned by the read call, and cancels that read. An application can cancel all reads submitted on the same scanner handle by calling the *SCAN_Flush* function.

When an application is finished using a scan buffer allocated with *SCAN_AllocateBuffer*, it should be deallocated by calling the *SCAN_DeallocateBuffer* function. This function frees the memory used by the structure.

When the application is finished using a scanning device, it should disable and close the device. The *SCAN_Disable* function disables scanning on the device, and the *SCAN_Close* function closes the scanner handle.

Combining Reads

The Symbol Scanner Driver allows multiple applications to share the use of a single scan engine. If each application can form its own read request, all of these requests must be combined into a single request to be sent to the scanner. After the combined read request has been satisfied, the driver must be able to determine which application's request was satisfied, and supply the data to that application only. A set of rules has been created for combining multiple read requests so that the driver can unambiguously determine which scan request has been satisfied by the scanner. The device can then perform the user feedback for that read request, and return the data to the proper application. If a read request cannot be combined with the read requests already present in the system, it is rejected.

Reads are combined based upon the enabled decoders and their lengths. If two read requests contain a disjoint set of enabled decoders, they may be combined. When there is an overlap in enabled decoders, the reads can still be combined if the decoder lengths are disjoint and the decoder specific parameters are identical. When a combined read is satisfied by the scanner, the Scanner Driver can determine the request to which the data belongs, based upon the label type and length.

As expected, there are some exceptions to these rules. The UPC/EAN common parameters apply to all UPC and EAN decoders (UPCE0, UPCE1, UPCA, EAN8, and EAN13). Therefore, these parameters must be identical in order to combine read requests that enable any of these decoders. Because of an ambiguity in the symbology definition, reads cannot contain both Code 39 with full ASCII enabled and Trioptic 39. Finally, code conversions affect read combination. If a conversion flag is set in a read request, the read cannot be combined with a request containing the original or converted decoder type of the same length. For example, if a read request enables UPCE0 and sets the convert to UPCA flag, the read cannot be combined with any other request including UPCE0 or UPCA.

Foreground, Background and Monitor Reads

Not all applications want to use the scanner in the same manner. Some applications may want to have exclusive use of the scanner while they are active. Other applications may be willing to share the scanner, or even want to eavesdrop on the scan data requested by other applications. To handle the various types of applications requiring the use of a scanner, the Symbol Scanner Driver implements several modes of scanner operation: foreground reads, background reads, and monitor reads. The type of read submitted is determined by the *dwScanType* field of the scan parameters.

An application that posts a foreground read request has complete control over the scanner while it has the focus. If the application is minimized, or the user begins to use another application, the application should cancel its

foreground reads to allow another application to gain control of the Scanner Driver. If one or more foreground read requests are present in the system, background reads are ignored.

The background read mode allows several applications to post simultaneous scan requests. An application is not required to have the focus in order to have its background scan request satisfied. This mode provides the ability to have an application automatically gain the focus when a barcode of a certain type is scanned. For example, when a UPCA barcode on a product is scanned, it can automatically cause an inventory application to get the focus. The applications need not have any knowledge of the other applications running in the system, because the device handles the cooperative use of the scanner.

The third and final scanner read mode is a monitor read. A monitor read capability is provided for applications that may want to monitor the scanner operation, without affecting the other applications in the system. Whenever a read request is satisfied, the list of current monitor reads is consulted, and any monitor read that can be satisfied by the current scan is satisfied as well. The monitor reads are not considered when creating the single read request sent to the scanner, so they have no effect on the scanning operation of the device.

Writing an Image Capture Application

 The Image Capture C API allows an application to access the imaging device(s) on a system for the purpose of acquiring still images. This API provides applications with a Win32 C callable interface.

Using the API an application can:

- Enumerate the imaging devices that are available on the system.
- Open one or more of the available devices.
- Query for supported capabilities.
- Modify capabilities.
- Enable the viewfinder function.
- Acquire still images.

Imaging Devices

There may be more than one imaging device available on a system. The *IMAGE_FindFirst*, *IMAGE_FindNext* and *IMAGE_FindClose* functions may be used to enumerate the available devices.

Once a device has been selected the *IMAGE_Open* and *IMAGE_Close* functions must be called before and after any of the other Image Capture C API functions are called. The *IMAGE_Open* call returns a handle that must be used by all subsequent calls to the API. The handle is used to identify the target device. This allows an application to use the Image Capture C API to access multiple imaging devices simultaneously.

After opening an imaging device the *IMG_VERCAP* capabilities may be queried to determine the hardware, driver and API versions. The *IMG_DEVCAP* capabilities returns basic information about the imaging device such as its resolution.

Device Sharing

The imaging devices on a system may be accessed by multiple applications and for multiple purposes. When an application opens an imaging device, that device is not automatically locked. The imaging device is only locked while actually acquiring images or if explicitly locked using the *IMAGE_LockImager* function.

In order ensure maximum availability of the imaging devices applications should only lock the imager when necessary.

Image Acquisition

Image acquisition is the process of obtaining image data from an imaging device. When an application makes a successful call to *IMAGE_StartAcquisition* the driver begins to transfer consecutive images from the imager to an internal buffer. When the application calls *IMAGE_GetImage* the driver formats and returns the most recently acquired image.

In order for an application to read every image acquired from the imager without missing any, the current call to *IMAGE_GetImage* must return, and the application must make another image read call within one acquisition period. The ability to read every acquired image is not guaranteed and depends on the following factors:

- Acquisition time: This time is made up of the exposure time and the image data transfer time. The data transfer time is fixed. The exposure time can be modified but must be set for the current lighting conditions.
- Formatting: Accepting RAW image data in the format as is it is read from the imager gives the best performance. This can be done by using all default values for the *IMG_IMGCAP_** capabilities.
- Viewfinder: Disabling the viewfinder function improves performance.
- Overall system loading: Avoid any unnecessary processing between image reads. Loading by other applications may be unpredictable and unavoidable.

The *IMG_ACQCAP* capabilities control the parameters effecting acquisition such as exposure and gain. These capabilities may be updated at any time whether or not image acquisition has been started with *IMAGE_StartAcquisition*. The driver puts new parameters into effect at the earliest opportunity. The values of the *IMG_ACQCAP* capabilities that were used on the last call to *IMAGE_GetImage* may be retrieved using the *IMAGE_GetCapImgAcqValue* function.

Image Formatting

When an image is read using *IMAGE_GetImage* the returned buffer contains the image formatted as specified by the *IMG_IMGCAP* capabilities. These capabilities may be modified at any time and the driver puts them in to effect at the earliest opportunity. The values of the *IMG_IMGCAP* capabilities that were in used on the last call to *IMAGE_GetImage* may be retrieved using the *IMAGE_GetCapImgAcqValue* function.

Image Composition

Image composition is the means by which a user can determine what is in the field of view of the imaging device. The Image Capture C API supports two mechanisms to assist in image composition: aiming and viewfinder.

Aiming is achieved by shining a laser through a Diffractive Optical Element (DOE) thereby creating a targeting pattern on the object(s) in the field of view. The *IMG_ACQCAP_AIMING* capability (if supported) may be used to enable or disable the aiming function. Aiming may be enabled or disabled at any time but this only become active when acquisition is started using *IMAGE_StartAcquisition*.

Viewfinder works by displaying to the screen, consecutive images captured by the imaging device. The *IMG_VFCAP_WINHANDLE* capability specifies the window handle that the driver should use to display the

viewfinder images and must be set by the application. Once set the application may start and stop the viewfinder using the *IMAGE_StartViewfinder* and *IMAGE_StopViewfinder* functions, respectively.

Image acquisition must be started using *IMAGE_StartAcquisition* in order for viewfinder updates to take place. *IMAGE_StartViewfinder* may be called before image acquisition is started but viewfinder updates does not begin until the next successful call to *IMAGE_StartAcquisition* is made.

The *IMG_VFCAP* capabilities are used to control the viewfinder and may be modified at anytime. The driver puts new capability values in to effect the next time *IMAGE_StartViewfinder* is called.

The images acquired for the viewfinder function use the acquisition parameters that are defined by the *IMG_ACQCAP* capabilities.

Writing a WWAN Application



The Radio Interface Layer (RIL) driver of Pocket PC Phone Edition devices provides applications the ability to: send and receive voice calls; establish data connections (GPRS or 1xRTT); send and receive messages via Short Message Service (SMS); and, retrieve radio and network information, such as radio ESN, radio firmware, signal strength and roaming status.

There are various APIs and mechanisms available from Microsoft for the WAN application, including Telephony Application Programming Interface (TAPI), Phone API, SMS API, Remote Access Service (RAS), Connection Manager (CM) and Messaging Application Programming Interface (MAPI).

Line Device

TAPI views the WAN modem as a line device. If using TAPI, the application must initialize the handle for the line device. There are call back functions and line device messages associated with the device.

The TAPI functions involved are: *lineInitialize()*, *lineNegotiateAPIVersion()*, *lineGetDevCaps()*, *lineOpen()*, *lineSetStatusMessages()*.



NOTE Refer to the WANSample application in the SMDK for the usage.

Voice Call

The application can send/receive voice calls by using the TAPI or Phone API. In the WANSample application, the voice call is handled by the TAPI. There are list of APIs related to dialing, dropping and answering the voice call. The call status can be monitored or notified in the line device's call back function, by the line device message: *LINE_CALLSTATE*.

Data Call

The data call can be established and/or dropped by the RAS functions or CM functions. The WANSample application demonstrates both.

SMS

Microsoft provides SMS APIs to send a message via SMS. To receive it in the application, the developer must implement a mail rule client with the `ImailRuleClient` interface for the old `SMSReadMessage()` function to be deprecated. The mail rule client is registered as a COM object and must be registered to the OS by the following registry key:

```
$ [HKEY_LOCAL_MACHINE\Software\Microsoft\Inbox\Svc\SMS\Rules]
$ <CLSID> = dword: 1
```

The `WANSample` application includes a COM object called `Mapirule.dll`. This DLL is a mail rule client. If there is an incoming SMS, it looks for the existence of a named window ("FavoriteWndClass"); if it is present, it notifies the owner of that window of the incoming SMS. The `WANSample` application includes a named window to receive the SMS.

Radio/Network Information

Radio/Network information can be retrieved by the following TAPI functions:

- `lineGetCurrentSystemType()` - gets the system type (CDMA or GPRS).
- `lineGetGeneralInfo()` - gets the modem information. For GPRS and CDMA devices, the information is allocated differently in the `LINEGENERALINFO` data structure.
- `lineGetCurrentOperator()` - gets the operator of the GSM network. For CDMA, the application must get it from the REG key:
[HKEY_LOCAL_MACHINE\SOFTWARE\Sierra Wireless\EM3400] <CarrierSuffix>.
- `lineGetRegisterStatus()` - gets the network status.
- `lineGetLineDevStatus()` - gets the signal status.

Writing a Mobile Printing Application



The Symbol Windows CE Printer Drivers provide the ability for applications to print bar codes, text, bitmaps and lines. The bar codes that can be printed depend on the support provided by the printer. A set of `PRINT` functions allows an application developer to access all of the printing functionality defined in the drivers. What follows is a summary of the available printer API functions:

- *PRINT_FindFirst*, *PRINT_FindNext* and *PRINT_FindClose* allow an application to enumerate the available printers for a particular device.
- *PRINT_CreateDC* allows an application to create a Windows Device Context (DC) and associate it with one of the available printers.
- *PRINT_StartDoc* and *PRINT_EndDoc* allow an application to control the beginning and ending of a print job.
- *PRINT_StartPage* and *PRINT_EndPage* allow an application to control the printer driver to accept data and to inform the device that the application has finished writing to a page.
- *PRINT_GetDeviceCaps* allows an application to retrieve information about the capabilities of a printer.
- *PRINT_BarCode*, *PRINT_BitBlt*, *PRINT_DrawText* and *PRINT_Polyline* allow an application to print bar code data, bitmaps, text and polylines respectively.
- *PRINT_AbortDoc* allows an application to stop the current print job.

- *PRINT_DeleteDC* allows an application to delete the specified device context (DC).
- *PRINT_GetVersion* allows an application to get the version information of the printer, including the API, MDD, PDD, and TLD versions.

How to Print

The elements supported by the print drivers are as follows:

- Text
- Bar codes
- Polylines
- Bitmaps.

The following steps should be followed when writing mobile printing applications:

- i. Before any elements can be printed, a document must be started. This is done using the *PRINT_StartDoc* function.
- ii. Next a page must be started using the *PRINT_StartPage* command.
- iii. Once a page is started, any of the element commands can be used to print text, barcodes, polylines and bitmaps.
- iv. When there are no more elements to print, the page must be closed using the *PRINT_EndPage* command.
- v. A document can contain one or more pages. Each must be started by *PRINT_StartPage* and closed by *PRINT_EndPage*. Every new page starts with coordinates at origin
- vi. Once the last page is closed, the document must be ended with the *PRINT_EndDoc* command.



Appendix A Additional Learning

Introduction

This chapter provides the learning resources necessary for the effective use the SMDK series of developer kits.



NOTE Please note that all Web sites provided in this chapter are subject to change.

Additional Learning

In order to get the most value from the SMDK series of developer kits, there are certain prerequisite topics that need to be understood. These topics are not specific to products from Motorola, but are important concepts that should be learned. The concepts are separated into four groups, one for each SMDK.

SMDK for C

This section includes prerequisite subject matter for the effective use the SMDK for C.

Topics

- C/C++ Language Programming
- Windows CE Application Programming
- Windows CE API
- Windows Mobile 2003 API (Pocket PC)
- Windows Mobile 5.0 SDK
- CAB file packaging and Installation
- Using Windows Mobile 2003 devices
- Using Windows CE (CE.NET) devices
- Microsoft eMbedded Visual C++ 4.0
- Microsoft Pocket PC 2003 SDK
- Microsoft Visual Studio 2005
- Microsoft ActiveSync.

Training, Books, Learning Resources

To find training, books and other learning resources that target the topics listed about, visit the Microsoft Developer Network web sites below.

MSDN - Visual Studio Team System Developer Center

<http://msdn.microsoft.com/vstudio/teamsystem/default.aspx>

MSDN - Mobile Developer Center

<http://msdn.microsoft.com/mobility>

MSDN - Embedded Developer Center

<http://msdn.microsoft.com/embedded>

MSDN - Embedded Visual C++

<http://msdn.microsoft.com/mobility/gettingstarted/evc/default.aspx>

SMDK for .NET

This section includes prerequisite subject matter for the effective use the SMDK for .NET.

Topics

- C# Language Programming
- VB.NET Language Programming
- Microsoft Visual Studio.NET 2003
- Microsoft Visual Studio 2005
- Microsoft .NET Compact Framework.

Training, Books, Learning Resources

To find training, books and other learning resources that target the topics listed about, visit the Web sites below.

MSDN - Visual Studio Team System Developer Center

<http://msdn.microsoft.com/vstudio/teamsystem/default.aspx>

Microsoft Developer Network - .NET Compact Framework

<http://msdn2.microsoft.com/netframework/Aa497273>

Microsoft Developer Network - Microsoft Visual Studio Developer Center

<http://msdn.microsoft.com/vstudio/>

OpenNETCF.org

Provides information and shared-source projects targeting the Microsoft .NET Compact Framework.

<http://www.opennetcf.org>

Symbol Pocket Browser

This section includes prerequisite subject matter for the effective use the Symbol Pocket Browser.

Topics

- JavaScript
- VBScript
- HTML
- Ajax
- ActiveX
- COM
- Web Browsers
- Internet Explorer for Windows CE
- Pocket Internet Explorer
- Microsoft FrontPage.

Training, Books, Learning Resources

To find training, books and other learning resources that target the topics listed about, visit the Web sites below.

MSDN - Web Development

<http://msdn2.microsoft.com/en-us/library/Aa286532.aspx>

HTML

<http://www.htmlgoodies.com/>

SMDK for Java

This section includes prerequisite subject matter for the effective use the SMDK for Java.

Topics

- Java Language Programming
- Java Virtual Machine (JVM)
- IBM J9 JVM
- IBM WebSphere Studio Device Developer
- Java 2 Micro Environment (J2ME)
- Eclipse Plug-ins
- JAR Files
- JavaDocs.

Training, Books, Learning Resources

To find training, books and other learning resources that target the topics listed about, visit the Web sites below.

IBM DeveloperWorks - New to Java Technology

<http://www-128.ibm.com/developerworks/java/newto/>

Eclipse

<http://www.eclipse.org/>

Glossary

802.11/802.11b.

A radio protocol that may be used by the Symbol Spectrum24 radio card. Symbol radio cards that use the 802.11 protocol also have an ESS_ID.

ActiveX.

A set of technologies that enables software components to interact with one another in a networked environment, regardless of the language in which the components are created. ActiveX is built on Microsoft's Component Object Model (COM). Currently, ActiveX is used primarily to develop interactive content for the World Wide Web, although it can be used in desktop applications and other programs. ActiveX controls can be embedded in Web pages to produce animation and other multimedia effects, interactive objects, and sophisticated applications.

AirBEAM[®] Manager.

AirBEAM[®] Manager is a comprehensive wireless network management system that provides essential functions that are required to configure, monitor, upgrade and troubleshoot the Spectrum24[®] wireless network and its components (including networked devices). Some features include event notification, access point configuration, diagnostics, statistical reports, auto-discovery, wireless proxy agents and monitoring of access points and mobile units.

AirBEAM® Smart Client.

AirBEAM® Smart Client is part of Motorola's AirBEAM® suite, which also includes AirBEAM® Safe and AirBEAM® Manager. The AirBEAM® Smart Client system uses the network accessible host server to store software files that are to be downloaded to the devices. The AirBEAM® Smart Client provides the devices with the "smarts" to request software from the host. It allows them to request, download and install software, as well as to upload files and status data. The AirBEAM® Smart Client uses the industry standard FTP or TFTP file transfer protocols to check the host system for updates, and if necessary, to transfer updated software. Most often, AirBEAM® Smart Client is used with wireless networks, but any TCP/IP connection can be used. For more information, refer to the AirBEAM® Smart Windows® CE Client Product Reference Guide (p/n 72-63060-xx).

API.

Application Programming Interface. An interface by means of which one software component communicates with or controls another. Usually used to refer to services provided by one software component to another, usually via software interrupts or function calls

**Application Programming Interface.
ASCII.**

See **API**.

American Standard Code for Information Interchange. A 7 bit-plus-parity code representing 128 letters, numerals, punctuation marks and control characters. It is a standard data transmission code in the U.S.

Bar.

The dark element in a printed bar code symbol.

Bar Code.

A pattern of variable-width bars and spaces which represents numeric or alphanumeric data in machine-readable form. The general format of a bar code symbol consists of a leading margin, start character, data or message character, check character (if any), stop character, and trailing margin. Within this framework, each recognizable symbology uses its own unique format. See **Symbology**.

Bar Code Density.

The number of characters represented per unit of measurement (e.g., characters per inch).

Bar Coding.	A method of encoding a data identifier.
Bar Height.	The dimension of a bar measured perpendicular to the bar width.
Bar Width.	Thickness of a bar measured from the edge closest to the symbol start character to the trailing edge of the same bar.
Baud Rate.	A measure of the data flow or number of signaling events occurring per second. When one bit is the standard "event," this is a measure of bits per second (bps). For example, a baud rate of 50 means transmission of 50 bits of data per second.
Binary Digit.	See Bit .
BIOS.	Basic Input/Output System. A collection of ROM-based code with a standard API used to interface with standard PC hardware.
Bit.	Binary digit. One bit is the basic unit of binary information. Generally, eight consecutive bits compose one byte of data. The pattern of 0 and 1 values within the byte determines its meaning.
Bits per Second (bps).	Bits transmitted or received.
boot or boot-up.	The process a computer goes through when it starts. During boot-up, the computer can run self-diagnostic tests and configure hardware and software.
BOOTP.	A protocol for remote booting of diskless devices. Assigns an IP address to a machine and may specify a boot file. The client sends a bootp request as a broadcast to the bootp server port (67) and the bootp server responds using the bootp client port (68). The bootp server must have a table of all devices, associated MAC addresses and IP addresses.
bps.	See Bits Per Second .
Bps.	bytes per second
Byte.	On an addressable boundary, eight adjacent binary digits (0 and 1) combined in a pattern to represent a specific character or numeric value. Bits are numbered from the right, 0 through 7, with bit 0 the low-order bit. One byte in memory is used to store one ASCII character.
CAD.	Computer-Aided Design
CD.	Compact Disk
CD-ROM.	Compact Disk Read-Only Memory

Character.	A pattern of bars and spaces which either directly represents data or indicates a control function, such as a number, letter, punctuation mark, or communications control contained in a message.
Code Length.	Number of data characters in a bar code between the start and stop characters, not including those characters.
Cold Boot.	A cold boot restarts the device and erases all user stored records and entries.
COM.	See Component Object Model .
COM port.	Communication port; ports are identified by number, e.g., COM1, COM2.
Component Object Model.	A software architecture that allows applications to be built from binary software components. The underlying architecture that forms the foundation for higher-level software services, like those provided by Object Linking and Embedding (OLE) and ActiveX. Allows COM objects to be accessed by any COM-compliant application. Developed by Microsoft.
Continuous Code.	A bar code or symbol in which all spaces within the symbol are parts of characters. There are no intercharacter gaps in a continuous code. The absence of gaps allows for greater information density.
Cradle.	A cradle is used for charging the terminal battery and for communicating with a host computer, and provides a storage place for the terminal when not in use.
CRT.	Cathode Ray Tube
CRU .	Customer-Replaceable Unit
Data Communications Equipment (DCE).	A device (such as a modem) which is designed to attach directly to a DTE (Data Terminal Equipment) device.
DBP.	Digital Bar code Pattern
DCE.	See Data Communications Equipment .
DCP.	See Device Configuration Package .
Dead Zone.	An area within a scanner's field of view, in which specular reflection may prevent a successful decode.
Decoded Scanner.	A scanner that capable of reading and <i>decoding</i> (translating) a scanned bar code. (Also see decoded scanner.)

Decryption.	Decryption is the decoding and unscrambling of received encrypted data. Also see, Encryption and Key .
Device Configuration Package.	The Motorola enterprise mobility device Configuration Package provides the Product Reference Guide (PRG), flash partitions, Terminal Configuration Manager (TCM) and the associated TCM scripts. With this package hex images that represent flash partitions can be created and downloaded to the device.
DHCP.	Dynamic Host Configuration Protocol. Software that automatically assigns IP addresses to client stations logging onto a TCP/IP network. Similar to BOOTP, but also permits the leasing of an IP address. It eliminates having to manually assign permanent IP addresses. DHCP software typically runs in servers and is also found in network devices such as routers that allow multiple users access to the Internet.
DHCP Server.	A server in the network or a service within a server that assigns IP addresses.
Diffuse Reflections.	The scattered light reflected from a bar code as a result of scanning.
Digital Bar Pattern.	An enhanced pattern representing the width of the bars and spaces within the scanned symbol.
DIMM.	Dual In-line Memory Module
Discrete Code.	A bar code or symbol in which the spaces between characters (intercharacter gaps) are not part of the code.
DLL.	Dynamic Link Library
DNS Server.	The Control Panel allows you to set the IP address for a DNS Server, if used. This allows users to use server names, rather than IP addresses. It is set on the Network tab of the Control Panel.
Docking Station.	The device used to place a portable computer into for uploading, downloading and battery charging operations.
Domain Name.	The Control Panel allows you to set a Domain Name for the DNS Server, if used (e.g., symbol.com). It is set on the Network tab of the Control Panel.
DOS.	Disk Operating System. This is basic software that allows you to load and use software applications on your computer.
dpi.	dots per inch

DRAM.	Dynamic Random Access Memory
DSR.	Data Set Ready
EEPROM.	Electrically Erasable Programmable Read-Only Memory
Encode.	To translate data into machine-readable form using the format and conventions of a specific bar code symbology.
Encoded Area.	Total linear dimension occupied by all characters of a code pattern, including start/stop characters and data.
Encryption.	Encryption is the scrambling and coding of data, typically using mathematical formulas called algorithms, before information is transmitted over any communications link or network. A key is the specific code used by the algorithm to encrypt or decrypt the data. Also see, Decryption and Key .
ESD.	Electro-Static Discharge
ESS_ID.	Extended Service Set Identifier, defines the coverage area. Prior to the release of the 802.11 specification the ESS_ID was called the Net_ID or Network Identifier. For terminals using Spectrum24 radios with the 802.11 protocol, an ESS_ID allows facilities to limit which Access Points a device can communicate with. It is set on the Network tab of the Control Panel. The terminal can only communicate with Spectrum24 Access Points that have matching ESS_IDs. Also see NetID .
Ethernet.	Ethernet communication port. Allows a wired interface to a radio network.
FIFO.	First-In, First-Out memory buffer
File Transfer Protocol (FTP).	A TCP/IP application protocol governing file transfer via network or telephone lines. See TCP/IP .
Flash Disk.	Additional non-volatile memory for storing application and configuration files.
Flash Memory.	Flash memory is responsible for storing the system firmware and is non-volatile. If the system power is interrupted the data is not be lost.

Frequency Hopping.	The use of a random sequence of frequency channels to achieve spread spectrum compliance. Stations that use frequency hopping change their communications frequency at regular intervals. A hopping sequence determines the pattern at which frequencies are changed. Messages take place within a hop. See Hopping Sequence and Spread Spectrum .
FRU.	Field-Replaceable Unit
FTP.	See File Transfer Protocol .
Gateway Address.	An IP address for a network gateway or router. A device may be part of a subnet as specified by its IP address and Netmask. It can send packets directly to any node on the same subnet. If the destination node is on a different subnet, then the terminal sends the packet to the gateway first. The gateway determines how to route the packet to the destination subnet. This field is an option used by networks that require gateways.
Gbyte, GB.	gigabyte
GUI.	Graphical User Interface
Hard Reset.	See Cold Boot .
Hopping Sequence.	A set of random frequencies designed to minimize interference with other sets of random frequencies. A hopping sequence determines the pattern with which a station that uses frequency hopping changes its communications frequency. See Frequency Hopping .
Host Computer.	A computer that serves other terminals in a network, providing such services as computation, database access, supervisory programs and network control.
HTML.	HyperText Markup Language. The coding language used to create Hypertext documents for use on the World Wide Web.
HTTP.	HyperText Transfer Protocol
HyperText Markup Language.	See HTML .
Hz.	Hertz; A unit of frequency equal to one cycle per second.
I/O.	Input/Output

I/O Ports.	The connection between two devices, defined by common physical characteristics, signal characteristics, and signal meanings. Types of interfaces include RS-232 and PCMCIA.
	I/O ports are primarily dedicated to passing information into or out of a device's memory.
IDE.	Intelligent drive electronics. Refers to the solid-state hard drive type.
IEEE.	Institute of Electrical and Electronics Engineers
IEEE Address.	See MAC Address .
imaging scanning .	devices with an integrated imager use digital camera technology to take a digital picture of a bar code, store the resulting image in memory and execute state-of-the-art software decoding algorithms to extract the data from the image.
IMEI.	International Mobile Equipment Identity. A 15 digit unique code that is used to identify the GSM/DCS/PCS phone to a GSM/DCS/PCS network. When a phone is switched on, this unique IMEI number is transmitted and checked against a database of blacklisted or greylisted phones in the network's EIR (Equipment ID Register). If the EIR and IMEI numbers match, the networks can do a number of things. For example, the can greylist or blacklist a phone: Greylisting allows the phone to be used, but it can be tracked to see who has it (via the SIM info). Blacklisting bars the phone from being used on any network where there is an EIR match.
Input/Output Ports.	See I/O Ports .
International Mobile Equipment Identity.	See IMEI .
Internet Protocol Address.	See IP .
IOCTL.	Input/Output Control.

IP.	Internet Protocol. The IP part of the TCP/IP communications protocol. IP implements the network layer (layer 3) of the protocol, which contains a network address and is used to route a message to a different network or subnetwork. IP accepts "packets" from the layer 4 transport protocol (TCP or UDP), adds its own header to it and delivers a "datagram" to the layer 2 data link protocol. It may also break the packet into fragments to support the maximum transmission unit (MTU) of the network.
IP Address.	Internet Protocol Address. The address of a computer attached to an IP network. Every client and server station must have a unique IP address. A 32-bit address used by a computer on a IP network. Client workstations have either a permanent address or one that is dynamically assigned to them each session. IP addresses are written as four sets of numbers separated by periods; for example, 204.171.64.2.
IPX/SPX.	Internet Package Exchange/Sequential Packet Exchange. A communications protocol for Novell. IPX is Novell's Layer 3 protocol, similar to XNS and IP, and used in NetWare networks. SPX is Novell's version of the Xerox SPP protocol.
IR.	infrared
IRQ.	interrupt request
IS-95.	Interim Standard 95. The EIA/TIA standard that governs the operation of CDMA cellular service. Versions include IS-95A and IS-95B. See CDMA.
ISDN.	Integrated Services Digital Network
ISO.	International Standards Organization

J2ME (Java 2 Platform, Micro Edition).	The edition of the Java platform that is targeted at small, standalone or connectable consumer and embedded devices. The J2ME technology consists of a Java Virtual Machine and a set of APIs suitable for tailored runtime environments for these devices. The J2ME technology has two primary kinds of components - configurations and profiles. J2ME configurations have profiles associated to them. For example, a Connected Device Configuration (CDC) is available in two profiles: Foundation and Personal while Connected Limited Device Configuration (CLDC) is available in Mobile Information Device Profile (MIDP).
J9 JVM.	A J2ME Java Virtual Machine from IBM. Competing J2ME JVMs are Sun PersonalJava, Insignia Jeode, and NSIcom CrEme).
Java.	Developed by Sun Microsystems, Java is a programming language and a technology that is specifically designed for writing programs that can be safely downloaded to a computer through the Internet and immediately run without fear of viruses or other harm to the computer or files. Using small Java programs (called "Applets"), Web pages can include functions such as animations, calculators and other fancy tricks. Java is a simple, robust, object-oriented, platform-independent multi-threaded, dynamic general-purpose programming environment. It is best for creating applets and applications for the Internet, intranets and any other complex, distributed network. Applications created by using the Java language are compiled into bytecode, a somewhat interpreted code unlike native machine code. A Java Virtual Machine (JVM) executes this bytecode. The JVM is designed to talk to the "native" hardware acting as the "translator" of the bytecode. Java comes in different implementations such as J2SE, J2EE, and J2ME.
Java Virtual Machine.	See JVM .

JavaScript.	A scripting language from Netscape that is only marginally related to Java. JavaScript was designed to resemble Java, which in turn looks a lot like C and C++. The difference is that Java was built as a general-purpose object language, while JavaScript is intended to provide a quicker and simpler language for enhancing Web pages and servers. JavaScript is embedded as a small program in a web page that is interpreted and executed by the Web client. The scriptor controls the time and nature of the execution, and JavaScript functions can be called from within a Web document, often executed by mouse functions, buttons, or other actions from the user. JavaScript can be used to fully control Netscape and Microsoft Web browsers, including all the familiar browser attributes.
JVM (Java Virtual Machine).	A platform-independent programming language that converts Java bytecode into machine language and executes it.
kbyte, kb, k.	kilobyte
Kerberos.	Kerberos is a network authentication protocol. It is designed to provide strong authentication for client/server applications by using secret-key cryptography. A free implementation of this protocol is available from the Massachusetts Institute of Technology. Kerberos is available in many commercial products as well.
Key.	A key is the specific code used by the algorithm to encrypt or decrypt the data. Also see, Encryption and Decrypting .
kg.	kilogram
kHz.	kilohertz
kW.	kilowatt
LAN .	Local Area Network. A radio network that supports data communication within a local area, such as within a warehouse of building.
LASER.	Light Amplification by Stimulated Emission of Radiation. The laser is an intense light source. Light from a laser is all the same frequency, unlike the output of an incandescent bulb. Laser light is typically coherent and has a high energy density.

Laser Diode.	A gallium-arsenide semiconductor type of laser connected to a power source to generate a laser beam. This laser type is a compact source of coherent light.
Laser Scanner.	A type of bar code reader that uses a beam of laser light.
lb.	pound
LCD.	See Liquid Crystal Display.
LDT.	Laser Data Terminal.
LED.	Light-Emitting Diode.
LED Indicator.	A semiconductor diode (LED - Light Emitting Diode) used as an indicator, often in digital displays. The semiconductor uses applied voltage to produce light of a certain frequency determined by the semiconductor's particular chemical composition.
Light Emitting Diode.	See LED.
Liquid Crystal Display (LCD).	A display that uses liquid crystal sealed between two glass plates. The crystals are excited by precise electrical charges, causing them to reflect light outside according to their bias. They use little electricity and react relatively quickly. They require external light to reflect their information to the user.
Local Area Network.	See LAN.
LRT.	Laser Radio Terminal
LS.	Laser Scanner
m.	meter, micro, milli
M.	mega
mA.	milliampere
MA.	megampere
MAC Address (also called IEEE Address).	Spectrum24 [®] devices, like other Ethernet devices, have unique, hardware-encoded MAC (also called IEEE addresses). MAC addresses determine the device sending or receiving data. The MAC address is a 48-bit number written as six hexadecimal bytes separated by colons.
Magnetic Stripe Reader.	Scanner that reads the magnetic tape attached to items such as credit cards.
Mb.	megabit
Mbps.	one million bits per second
Mbyte, MB.	megabyte
MC.	Mobile Computer.

MDN.	Mobile Directory Number. The directory listing telephone number that is dialed (generally using POTS) to reach a mobile unit. The MDN is usually associated with a MIN in a cellular telephone -- in the US and Canada, the MDN and MIN are the same value for voice cellular users. International roaming considerations often result in the MDN being different from the MIN.
mg.	milligram
MHz.	megahertz
MIL.	1 mil = 1 thousandth of an inch.
MIN.	Mobile Identification Number. The unique account number associated with a cellular device. It is broadcast by the cellular device when accessing the cellular system.
Misread (Mis-decode).	A condition which occurs when the data output of a reader or interface controller does not agree with the data encoded within a bar code symbol.
mm.	millimeter
device.	In this text, <i>device</i> refers to the Symbol Series 9000 wireless portable computer. It can be set up to run as a stand-alone device, or it can be set up to communicate with a network, using wireless radio technology.
ms.	milli-seconds
MSR.	See Magnetic Stripe Reader.
MTBF.	Mean Time Between Failures
MTTR.	Mean Time To Repair
NCU.	Network Control Unit.
NetBeui.	A non-routable LAN protocol that is an extension to NetBIOS. Used for IBM's OS/2-based LAN Manager and Microsoft's LAN Manager and Windows for Workgroups.
NetBIOS.	Network Basic Input/Output System
NetID.	For terminals using Spectrum24 radios with the Spring protocol, a NetID allows facilities to limit which Access Points a device can communicate with. It is set on the Network tab of the Control Panel. The terminal can only communicate with Spectrum24 Access Points that have matching NetIDs. Also see ESS_ID.

Nominal.	The exact (or ideal) intended value for a specified parameter. Tolerances are specified as positive and negative deviations from this value.
Nominal Size.	Standard size for a bar code symbol. Most UPC/EAN codes are used over a range of magnifications (e.g., from 0.80 to 2.00 of nominal).
Null Modem Cable.	An RS-232 cable used to connect two personal computers together in close proximity for file transfer. It attaches to the serial ports of both machines and simulates what would occur naturally if modems and the phone system were used. It crosses the sending wire with the receiving wire.
NVM.	Non-Volatile Memory.
NVRAM.	Non-Volatile Random Access Memory
ODI.	See Open Data-Link Interface .
Open Data-Link Interface (ODI).	Novell's driver specification for an interface between network hardware and higher-level protocols. It supports multiple protocols on a single NIC (Network Interface Controller). It is capable of understanding and translating any network information or request sent by any other ODI-compatible protocol into something a NetWare client can understand and process.
Open System Authentication.	Open System authentication is a null authentication algorithm.
OS.	Operating System
oz.	ounce
p.	pico
PAN .	Personal Area Network. Using Bluetooth wireless technology, PANs enable devices to communicate wirelessly. Generally, a wireless PAN consists of a dynamic group of less than 255 devices that communicate within about a 33-foot range. Only devices within this limited area typically participate in the network.
Parameter.	A variable that can have different values assigned to it.

PC Card.	A plug-in expansion card for laptop computers and other devices, also called a PCMCIA card. PC Cards are 85.6mm long x 54 mm wide, and have a 68 pin connector. There are several different kinds: Type I; 3.3 mm high; use - RAM or Flash RAM Type II; 5 mm high; use - modems, LAN adaptors Type III; 10.5 high; use - Hard Disks
PCB.	Printed Circuit Board
PCMCIA.	Personal Computer Memory Card Interface Association. See PC Card .
PDT.	Portable Data Terminal.
Percent Decode.	The average probability that a single scan of a bar code would result in a successful decode. In a well-designed bar code scanning system, that probability should approach near 100%.
PING.	(Packet Internet Groper) An Internet utility used to determine whether a particular IP address is online. It is used to test and debug a network by sending out a packet and waiting for a response.
Pitch.	The angle on the horizontal plane between a scanner's beam and a perpendicular line directly out from the bar code surface.
pixel.	picture element
Print Contrast Signal (PCS).	Measurement of the contrast (brightness difference) between the bars and spaces of a symbol. A minimum PCS value is needed for a bar code symbol to be scannable. $PCS = (RL - RD) / RL$, where RL is the reflectance factor of the background and RD the reflectance factor of the dark bars.
Programming Mode.	The state in which a scanner is configured for parameter values. See Scanning Mode .
PROM.	Programmable Read-Only Memory.
PSDK.	Platform SDK.
Quiet Zone.	A clear space, containing no dark marks, which precedes the start character and follows the stop character of a bar code symbol.
Radio Frequency Identification.	See RFID .

RAM.	Random Access Memory. Data in RAM can be accessed in random order, and quickly written and read.
Raster Mode.	A scanning pattern of parallel lines that form the display of an image.
Reflectance.	Amount of light returned from an illuminated surface.
Resolution.	The narrowest element dimension which is distinguished by a particular reading device or printed with a particular device or method.
RF.	Radio Frequency.
RFID.	Radio Frequency Identification is a system for tagging and identifying mobile objects such as store merchandise, postal packages and sometimes living organisms (like pets). Using a special device called an RFID reader, RFID allows objects to be labeled and tracked as they move from place to place.
RFID Reader.	Device used to read RFID tags.
ROM.	Read-Only Memory. Data stored in ROM cannot be changed or removed.
ROM BIOS.	Read-Only Memory Basic Input/Output System
ROM-DOS.	The name of the licensed Disk Operating System loaded into the terminal's flash file system.
Router.	A device that connects networks and supports the required protocols for packet filtering. Routers are typically used to extend the range of cabling and to organize the topology of a network into subnets. See Subnet .
RS-232.	An Electronic Industries Association (EIA) standard that defines the connector, connector pins, and signals used to transfer data serially from one device to another.
Scan Area.	Area intended to contain a symbol.

Scanner.	An electronic device used to scan bar code symbols and produce a digitized pattern that corresponds to the bars and spaces of the symbol. Its three main components are: 1. Light source (laser or photoelectric cell) - illuminates a bar code. 2. Photodetector - registers the difference in reflected light (more light reflected from spaces). 3. Signal conditioning circuit - transforms optical detector output into a digitized bar pattern.
Scanning Mode.	The scanner is energized, programmed and ready to read a bar code.
Scanning Sequence.	A method of programming or configuring parameters for a bar code reading system by scanning bar code menus.
SDK.	Software Development Kit
Self-Checking Code.	A symbology that uses a checking algorithm to detect encoding errors within the characters of a bar code symbol.
SGML.	Standard Generalized Markup Language
Shared Key.	Shared Key authentication is an algorithm where both the AP and the MU share an authentication key.
SHIP.	Symbol Host Interface Program.
SID.	System Identification code. An identifier issued by the FCC for each market. It is also broadcast by the cellular carriers to allow cellular devices to distinguish between the home and roaming service.
SIMM.	Single In-line Memory Module
Skew .	The angle on the vertical plane between the scanner's beam and a perpendicular line directly out from the bar code surface.
SMDK.	Symbol Mobility Developer Kit.
Soft Reset.	See Warm Boot .
Space.	The lighter element of a bar code formed by the background between bars.
Spectrum One.	Motorola's implementation of the Spread Spectrum wireless network, utilizing direct sequencing.
Spectrum24.	Motorola's frequency-hopping, spread spectrum cellular network.

Specular Reflection.	The mirror-like direct reflection of light from a surface, which can cause difficulty decoding a bar code.
Spot(s).	The presence of ink in a bar code's spaces or clear areas.
Spread Spectrum.	A technique for uniformly distributing the information content of a radio signal over a frequency range larger than normally required for robust transmission of data. Spreading the signal without adding additional information adds significant redundancy, which allows the data to be recovered in the presence of strong interfering signals such as noise and jamming signals. The primary advantage of spread spectrum technology is its ability to provide robust communications in the presence of interfering signals.
Spring Radio Protocol.	A radio protocol that may be used by the Symbol Spectrum24 radio card. Symbol Radio cards that use the Spring protocol also have an Net ID.
SRAM.	Static Random Access Memory
Start/Stop Character.	A pattern of bars and spaces that provides the scanner with start and stop reading instructions and scanning direction. The start and stop characters are normally to the left and right margins of a horizontal code.
STEP.	Symbol Terminal Enabler Program.
Subnet.	A subset of nodes on a network that are serviced by the same router. See Router .
Subnet Mask.	A 32-bit number used to separate the network and host sections of an IP address. A custom subnet mask subdivides an IP network into smaller subsections. The mask is a binary pattern that is matched up with the IP address to turn part of the host ID address field into a field for subnets. Default is often 255.255.255.0.
Substrate.	A foundation material on which a substance or image is placed.
SVTP.	Symbol Virtual Terminal Program.
Symbol.	A scannable unit that encodes data within the conventions of a certain symbology, usually including start/stop characters, quiet zones, data characters and check characters.
Symbol Aspect Ratio.	The ratio of symbol height to symbol width.

Symbol Height.	The distance between the outside edges of the quiet zones of the first row and the last row.
Symbol Length.	Length of symbol measured from the beginning of the quiet zone (margin) adjacent to the start character to the end of the quiet zone (margin) adjacent to a stop character.
Symbology.	The structural rules and conventions for representing data within a particular bar code type (e.g. UPC/EAN, Code 39, PDF417, etc.).
TCM.	See Terminal Configuration Manager.
TCP/IP.	(Transmission Control Protocol/Internet Protocol) A communications protocol used to internetwork dissimilar systems. This standard is the protocol of the Internet and has become the global standard for communications. TCP provides transport functions, which ensures that the total amount of bytes sent is received correctly at the other end. UDP is an alternate transport that does not guarantee delivery. It is widely used for real-time voice and video transmissions where erroneous packets are not retransmitted. IP provides the routing mechanism. TCP/IP is a routable protocol, which means that all messages contain not only the address of the destination station, but the address of a destination network. This allows TCP/IP messages to be sent to multiple networks within an organization or around the world, hence its use in the worldwide Internet. Every client and server in a TCP/IP network requires an IP address, which is either permanently assigned or dynamically assigned at startup.
Telnet.	A terminal emulation protocol commonly used on the Internet and TCP/IP-based networks. It allows a user at a terminal or computer to log onto a remote device and run a program.
Terminal.	See device.
Terminal Emulation.	A "terminal emulation" emulates a character-based mainframe session on a remote non-mainframe terminal, including all display features, commands and function keys. The MC9000 Series supports Terminal Emulations in 3270, 5250 and VT220.

Terminate and Stay Resident (TSR).	A program under DOS that ends its foreground execution to remain resident in memory to service hardware/software interrupts, providing background operation. It remains in memory and may provide services on behalf of other DOS programs.
TFTP.	(Trivial File Transfer Protocol) A version of the TCP/IP FTP (File Transfer Protocol) protocol that has no directory or password capability. It is the protocol used for upgrading firmware, downloading software and remote booting of diskless devices.
Tolerance.	Allowable deviation from the nominal bar or space width.
Transmission Control Protocol/Internet Protocol.	See TCP/IP .
Trivial File Transfer Protocol.	See TFTP .
TSR.	See Terminate and Stay Resident .
UDP.	User Datagram Protocol. A protocol within the IP protocol suite that is used in place of TCP when a reliable delivery is not required. For example, UDP is used for real-time audio and video traffic where lost packets are simply ignored, because there is no time to retransmit. If UDP is used and a reliable delivery is required, packet sequence checking and error notification must be written into the applications.
Undecoded Scanner.	A scanner that is only capable of reading a scanned bar code. The scanner is connected to a host (e.g., terminal) that is equipped with the necessary programming to <i>decode</i> (translate) the laser bar code. (Also see decoded scanner.)
UPC.	Universal Product Code. A relatively complex numeric symbology. Each character consists of two bars and two spaces, each of which is any of four widths. The standard symbology for retail food packages in the United States.
URL.	Uniform Resource Locator
VGA.	Video Graphics Adapter
Visible Laser Diode (VLD).	A solid state device which produces visible laser light.

VK Code.	Virtual key codes are symbolic constant names, hexadecimal values, and keyboard equivalents for the virtual key codes used by Windows.
VRAM.	Video Random Access Memory
WAIS.	Wide-Area Information Server
WAN.	Wide-Area Network. A radio network that supports data communication beyond a local area. That is, information can be sent across a city, state, or even nationwide.
Wand.	A non-automatic, hand-held scanning device containing a photo-electric cell used as a contact bar code reader (sometimes called a "light pen").
Warm Boot.	A warm boot restarts the device by closing all running programs. All data that is not saved to flash memory is lost.
WEP.	Wired Equivalent Privacy, is specified by IEEE for encryption and decryption of RF (wireless) communications.
WEP Encryption.	(Wired Equivalent Privacy encryption) The conversion of data into a secret code for transmission over a public network. The original text, or plaintext, is converted into a coded equivalent called ciphertext via an encryption algorithm. The ciphertext is decoded (decrypted) at the receiving end and turned back into plaintext. The encryption algorithm uses a key, which is a binary number that is typically from 40 to 128 bits in length. The greater the number of bits in the key (cipher strength), the more possible key combinations and the longer it would take to break the code. The data is encrypted, or "locked," by combining the bits in the key mathematically with the data bits. At the receiving end, the key is used to "unlock" the code and restore the original data.
Wh.	watt-hour
Wireless Local Area Network (WLAN).	See LAN .
Wireless Wide Area Network (WWAN).	See WAN .
WNMP.	(Wireless Network Management Protocol) This is Motorola's proprietary MAC layer protocol used for inter access point communication and other MAC layer communication.
WNMS (renamed to AirBEAM® Manager).	See AirBEAM® Manager .

Working Distance.

The distance from the scanner to the bar code within the scanner's depth of field.

WSDD.

WebSphere Studio Device Developer

WWAN.

Wireless Wide-Area Network

WYSIWYG.

What You See Is What You Get

Index

A

ActiveSync i-ix, 3-13, 6-8, 7-1, 9-3, A-2
ActiveSync Remote Display 3-16
AirBEAM 3-13, 9-4
API Groups 2-2, 3-5
architecture
 SMDK for .NET 5-2
 SMDK for C 3-1
 SMDK for Java 6-1
 Symbol Pocket Browser 4-2

B

browsers
 IE6 8-2
 internet explorer A-4
 pocket internet explorer 8-2, A-4
 Symbol PocketBrowser 8-2

C

Class Libraries 2-8, 5-4, 6-6, 9-5
Class Namespaces 2-6
compact framework, Microsoft .NET 5-3
components
 SMDK for .NET 5-5
 SMDK for C 3-7
 SMDK for Java 6-6
connection manager 10-7
CopyFiles 9-2, 9-8

D

DCP 7-3
Device Configuration Packages 7-3
DLLs 3-1, 5-1, 5-8, 6-6, 6-8, 10-8

dynamic link libraries 3-1, 5-1, 5-8, 6-6, 6-8, 10-8

E

eMbedded Visual Basic 3-15
Enterprise Mobility 1-1
eVB 3-15

F

flash file system 9-1
Function Groups 2-4

I

initial program loader 7-3
internet explorer A-4
internet explorer 6 8-2
IPL 7-3

J

j2me 6-3
java virtual machine 6-3
JavaPOS 6-19
JVM 6-3
JVM See java virtual machine

L

learning resources A-2, A-3, A-4, A-5

M

MAPI 10-7
mass deployment
 ActiveSync 7-1

messaging application programming interface 10-7
 Microsoft .NET CF 5-3
 MSP 3-13, 9-4

P

platform integrator 3-8
 platform SDK 3-7, 3-8
 Pocket Browser 2-10
 pocket internet explorer 8-2, A-4
 programming libraries
 SMDK for .NET 5-4
 SMDK for C 3-5
 SMDK for Java 6-4
 PSDK 3-7, 3-8

R

radio interface layer driver 10-7
 RAS 10-7
 RegMerge 9-2, 9-8
 remote access service 10-7
 RIL driver 10-7

S

sample apps
 C 3-12
 C# 5-9
 Java 6-11
 VB.NET 5-10
 SDK 2-10, 3-7, 3-8, 5-1, 5-8, 6-1, A-2
 SHFullScreen 8-2
 SMDK 1-2
 platform integrator 3-8
 SMDK for .NET i-ix, 2-6, 2-10, 5-1, A-3
 SMDK for C i-ix, 2-2, 2-10, A-2
 SMDK for eVC4 2-10
 SMDK for Java i-ix, 2-8, 2-10, A-5
 SMDK help file i-ix
 SMS API 10-7
 Software Tools for Application Development . . 3-13, 5-7, 6-12
 Symbol Pocket Browser i-ix, 2-4, A-4
 Symbol PocketBrowser 8-2
 Symbol PocketBrowser Features 4-5

T

TAPI 10-7
 TCM 7-3
 telephony application programming interface 10-7
 Terminal Configuration Manager 7-3
 tools

all SMDKs 2-10
 alternate development 2-11
 Odyssey 2-11
 SMDK for .NET 2-6
 SMDK for Java 2-8
 Wavelink 2-12
 training A-2, A-3, A-4, A-5

X

XML Provisioning 9-8



MOTOROLA

Motorola, Inc.
One Motorola Plaza
Holtsville, New York 11742, USA
1-800-927-9626
<http://www.symbol.com>

MOTOROLA and the Stylized M Logo and Symbol and the Symbol logo are registered in the U.S. Patent and Trademark Office.
All other product or service names are the property of their registered owners.
© Motorola, Inc. 2007